

1- فصل اول (درک مدل‌های زبانی)

سیر تکامل هوش مصنوعی زبان: از شمارش کلمات تا درک عمیق متن



درک مدل‌های زبانی

در این فصل به این پرسش‌های کلیدی پاسخ خواهیم داد:

- هوش مصنوعی زبانی (Language AI) چیست؟
- مدل‌های زبانی بزرگ (LLMs) چه هستند؟
- مدل‌های زبانی بزرگ چه کاربردها و موارد استفاده‌ای دارند؟
- چگونه می‌توانیم خودمان از مدل‌های زبانی بزرگ استفاده کنیم؟

هوش مصنوعی زبانی چیست؟

واژه‌ی هوش مصنوعی (Artificial Intelligence - AI) اغلب برای توصیف سیستم‌های کامپیوتری به کار می‌رود که برای انجام وظایفی مشابه به هوش انسانی طراحی شده‌اند، مانند شناسایی صدا، ترجمه زبان و درک بصری. این نوع هوش متعلق به نرم‌افزار است و برخلاف هوش انسان‌ها است.

هوش مصنوعی زبانی (Language AI) به زیرشاخه‌ای از هوش مصنوعی اطلاق می‌شود که بر توسعه‌ی فناوری‌هایی متمرکز است که قادر به درک، پردازش و تولید زبان انسانی هستند. واژه‌ی هوش مصنوعی زبانی به‌طور معمول می‌تواند به‌جای NLP (Natural Language Processing) به کار رود، با توجه به موفقیت‌های مستمر روش‌های یادگیری ماشین در حل مشکلات پردازش زبان.

ما از واژه‌ی هوش مصنوعی زبانی برای اشاره به فناوری‌هایی استفاده می‌کنیم که از نظر فنی ممکن است مدل‌های زبانی بزرگ (LLMs) نباشند، اما همچنان تأثیر قابل توجهی در این حوزه دارند، مانند اینکه چگونه

سیستم‌های جستجو می‌توانند قدرت‌های فوق‌العاده‌ای به LLM‌ها بدهند (این موضوع در فصل ۸ توضیح داده خواهد شد).

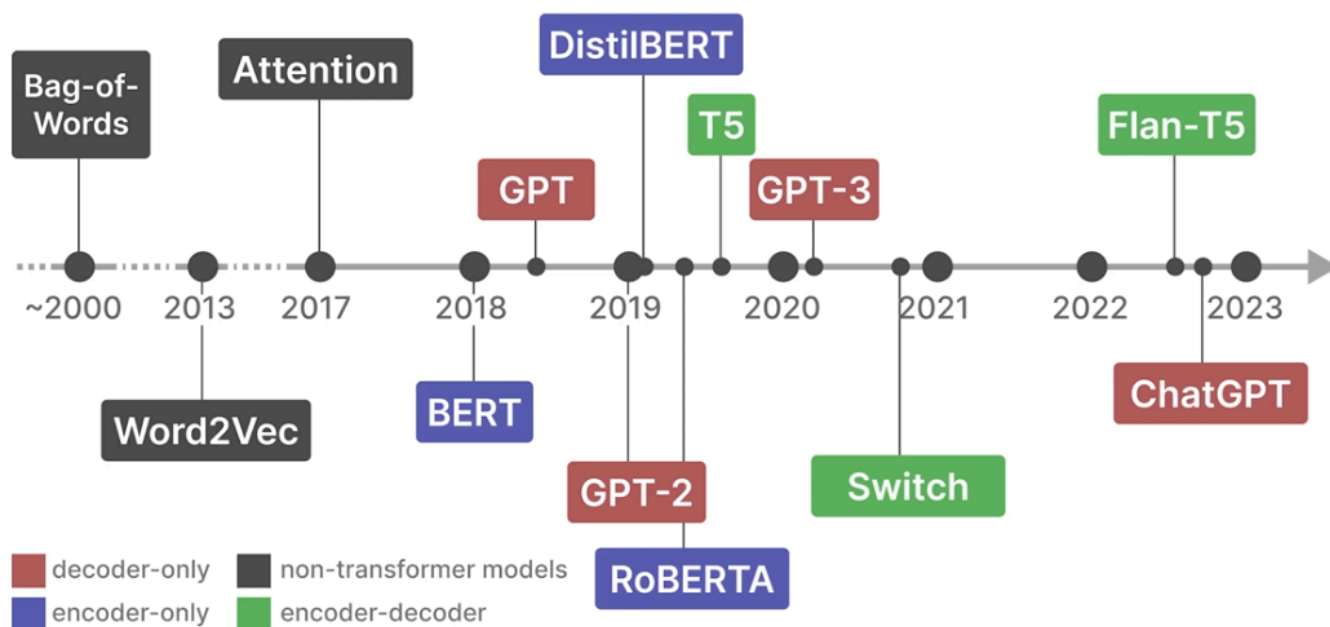
مدل‌های زبانی بزرگ چه هستند؟

برای شروع پاسخ به این سوال، ابتدا به تاریخچه‌ی هوش مصنوعی زبانی خواهیم پرداخت.

تاریخچه‌ی اخیر هوش مصنوعی زبانی

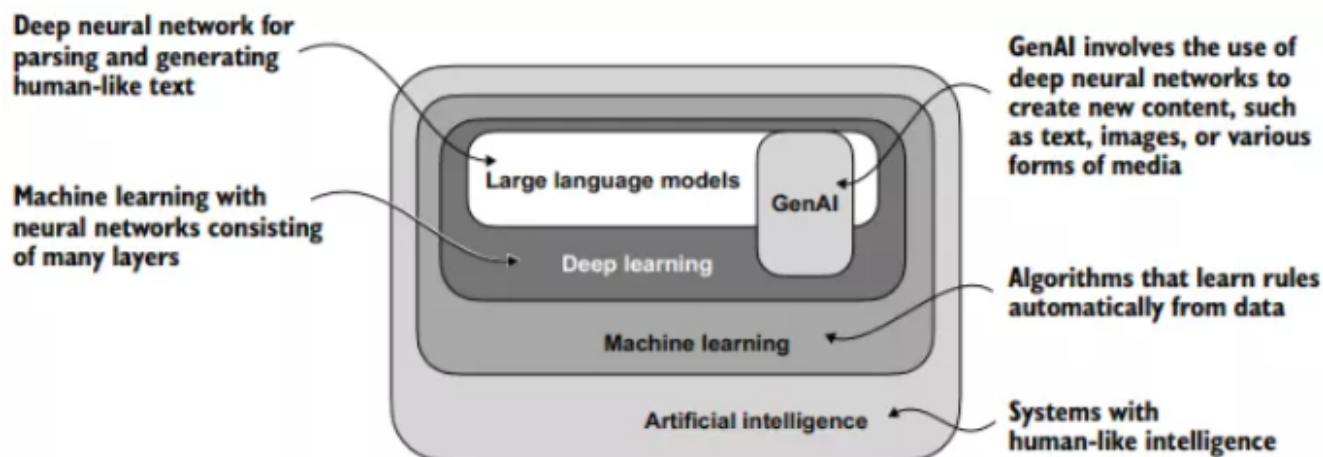
تاریخچه‌ی هوش مصنوعی زبانی (Language AI) شامل پیشرفت‌ها و مدل‌های متعددی است که با هدف نمایش و تولید زبان توسعه یافته‌اند. نمونه‌ای از این روند تاریخی در شکل زیر نشان داده شده است.

A Recent History of Language AI

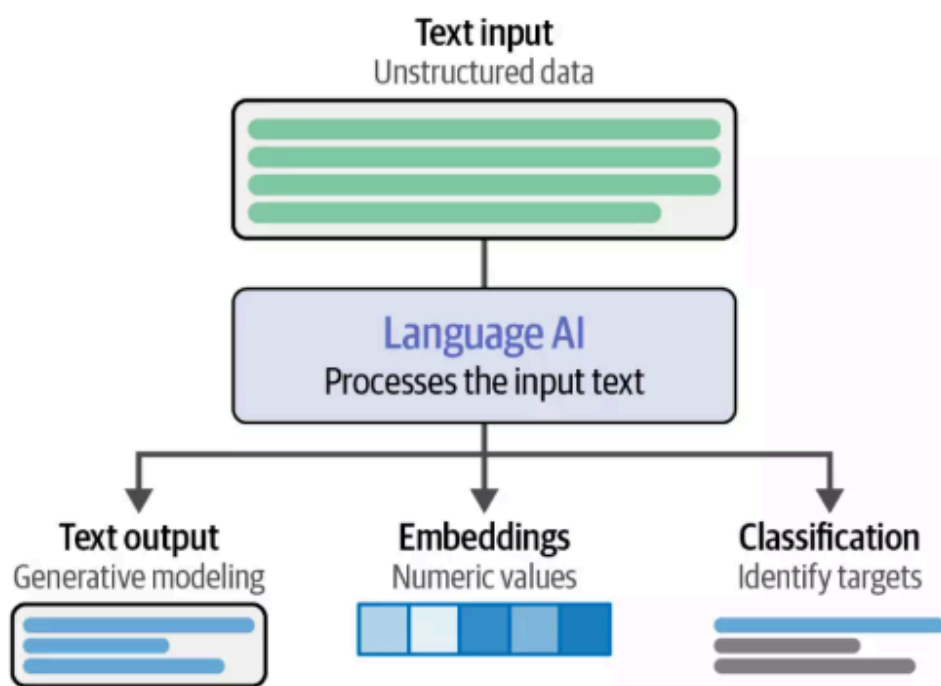


زبان، مفهومی پیچیده برای کامپیوترها محسوب می‌شود. متن، ذاتاً ساختار نیافته (Unstructured Data) است و هنگامی که صرفاً با صفر و یک (کاراکترهای مجزا) نمایش داده شود، معنای خود را از دست می‌دهد.

مدل‌های زبانی بزرگ (LLMs) کاربرد خاصی از تکنیک‌های یادگیری عمیق (Deep Learning) هستند که از قابلیت پردازش و تولید متن شبیه به انسان بهره می‌برند. یادگیری عمیق شاخه‌ای تخصصی از یادگیری ماشین (Machine Learning) است که بر استفاده از شبکه‌های عصبی چندلایه تمرکز دارد. به طور کلی، یادگیری ماشین و یادگیری عمیق حوزه‌هایی هستند که هدف آن‌ها طراحی الگوریتم‌هایی است که به کامپیوترها امکان می‌دهند از داده‌ها بیاموزند و وظایفی را انجام دهند که معمولاً به هوش انسانی نیاز دارند.

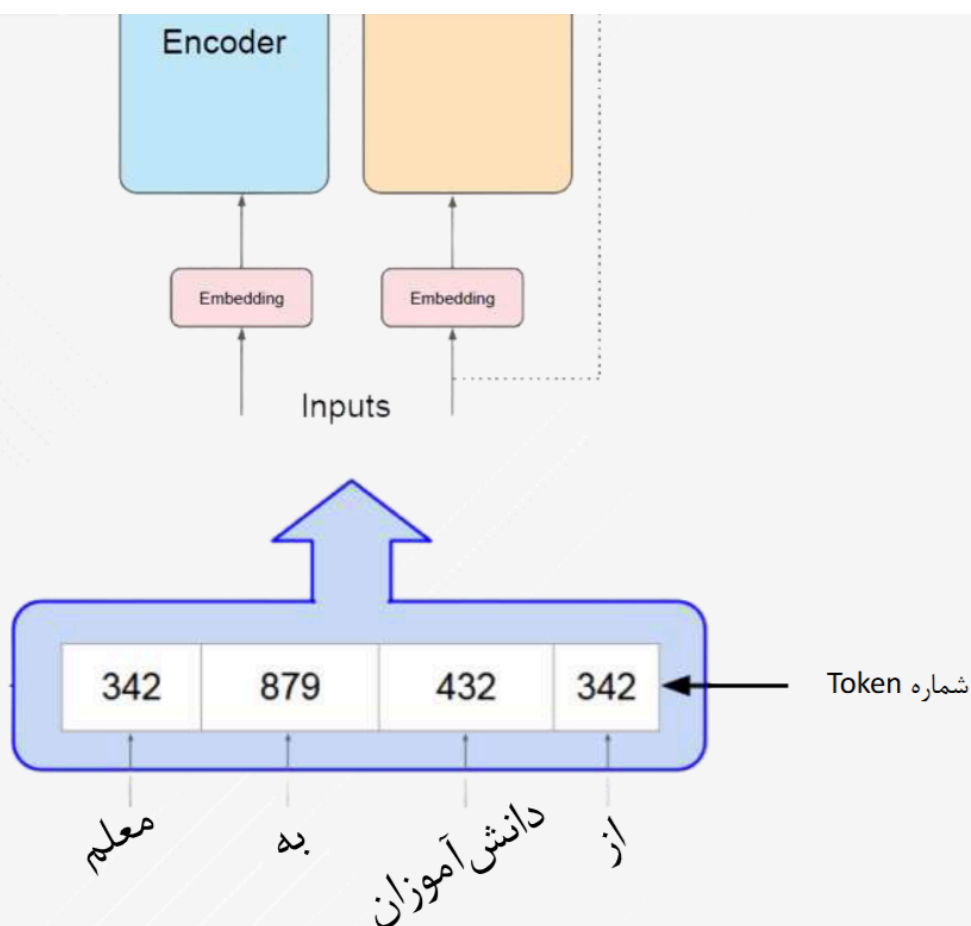


در نتیجه، در سراسر تاریخ هوش مصنوعی زبانی، تمرکز عمده‌ای بر ارائه‌ی زبان در قالب‌های ساختاریافته صورت گرفته است تا امکان پردازش آن برای کامپیوترها تسهیل شود. برخی از وظایفی که هوش مصنوعی زبانی قادر به انجام آن‌ها است، در شکل زیر نمایش داده شده است.



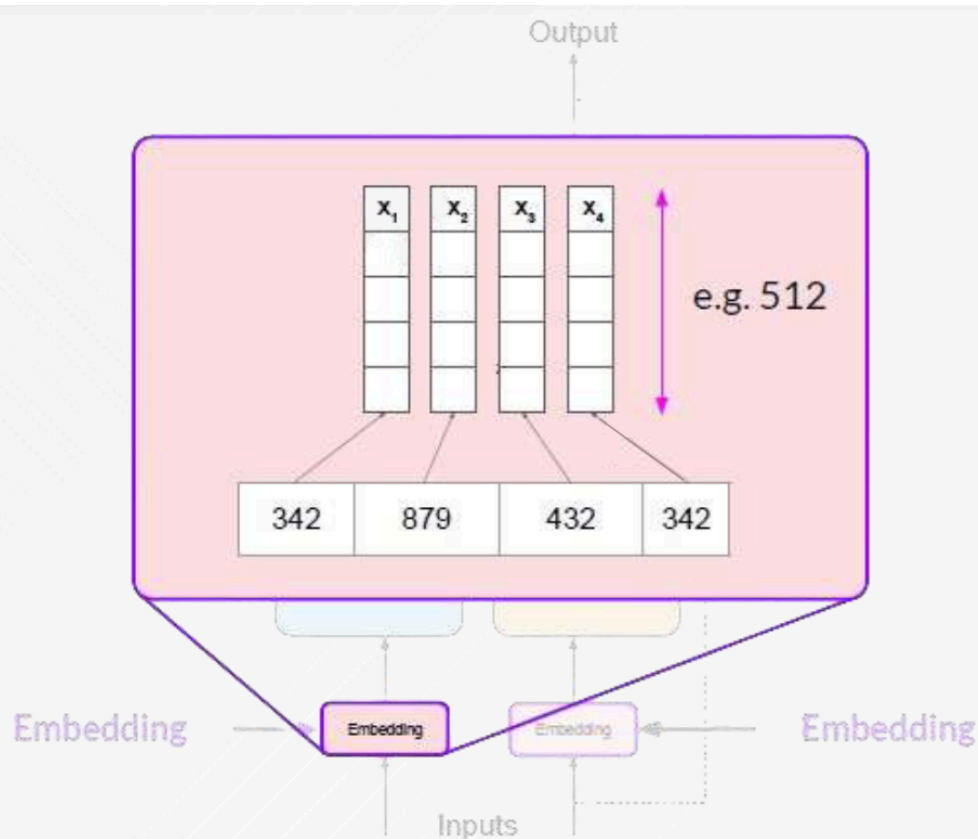
نمایش زبان با ایندکس

ترنسفورمر

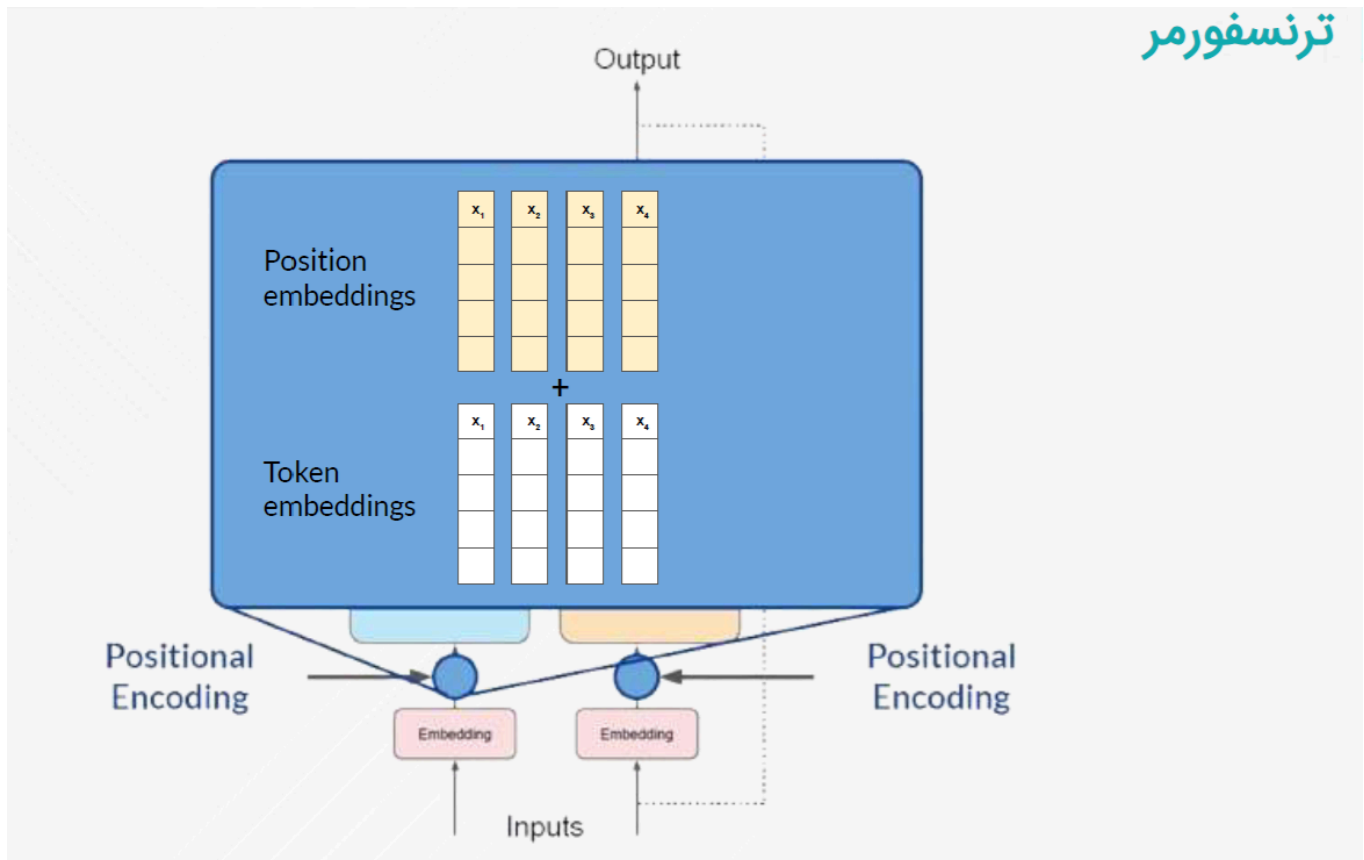


نمایش زبان با بردار

ترنسفورمر



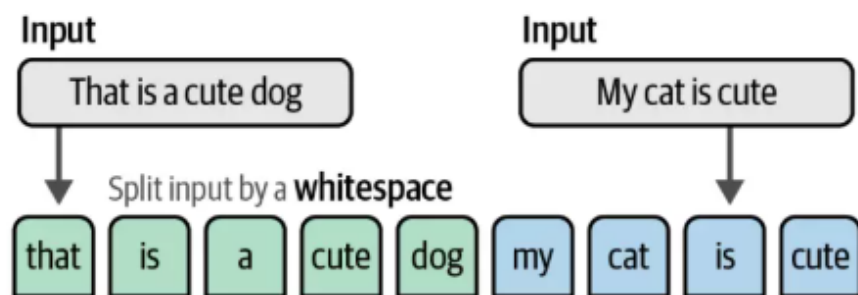
افزودن ترتیب به بردار

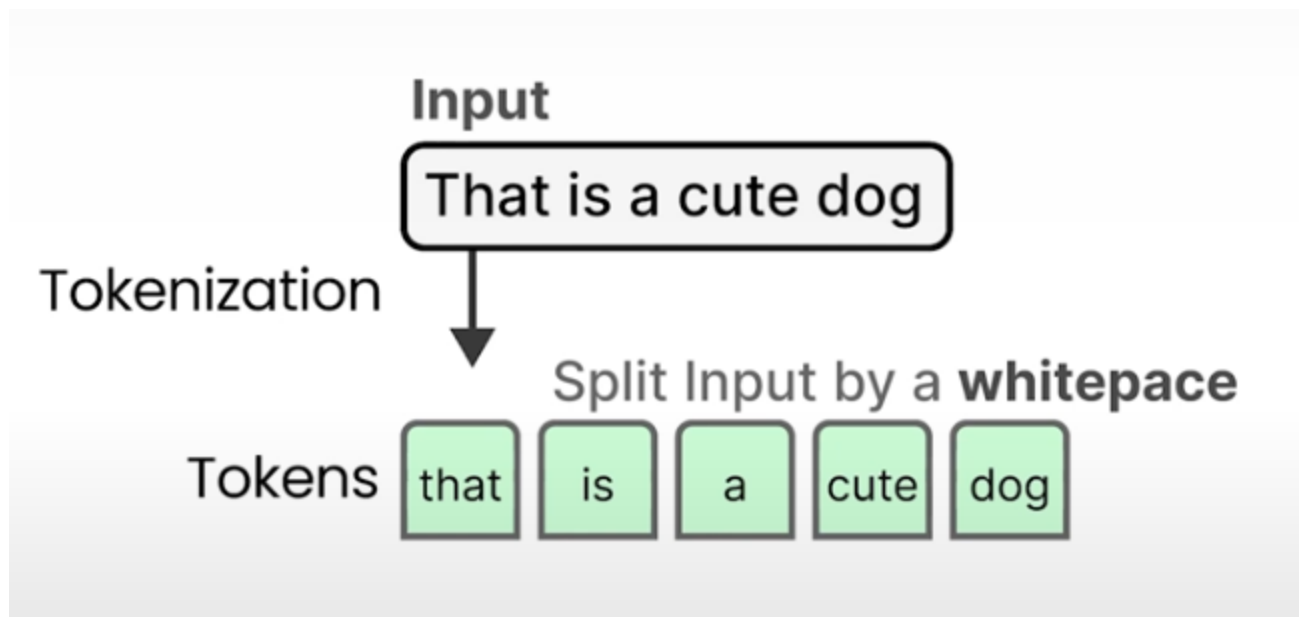


نمایش زبان به صورت کیسه کلمات (bag of words)

تاریخچه‌ی هوش مصنوعی زبانی (Language AI) با تکنیکی به نام کیسه کلمات (Bag-of-Words) آغاز می‌شود، که روشی برای نمایش متن‌های غیرساختاریافته (Unstructured Text) است. این روش برای نخستین بار در دهه‌ی 1950 مطرح شد، ولی محبوبیت آن به اوایل دهه 2000 باز می‌گردد.

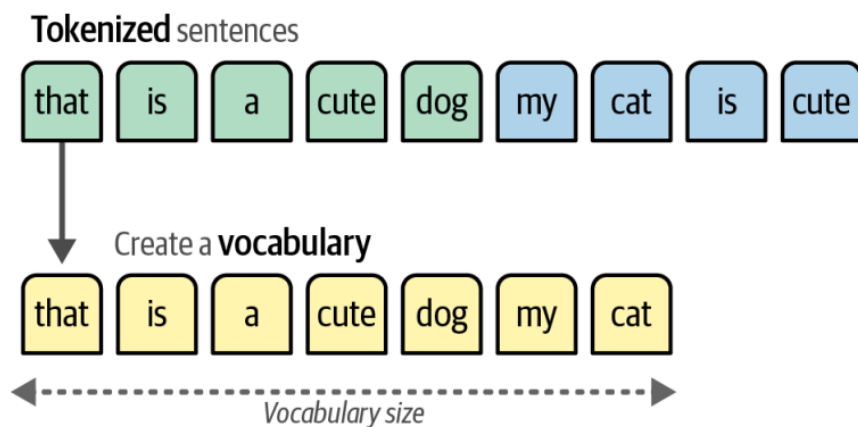
روش کیسه کلمات به این صورت عمل می‌کند: فرض کنید که دو جمله داریم و می‌خواهیم نمایش‌های عددی برای آن‌ها ایجاد کنیم. اولین گام در مدل کیسه کلمات تجزیه به کلمه‌ها (Tokenization) است؛ فرآیندی که در آن جملات به کلمات یا زیرکلمات (توکن‌ها) تقسیم می‌شوند، همانطور که در شکل زیر نشان داده شده است.



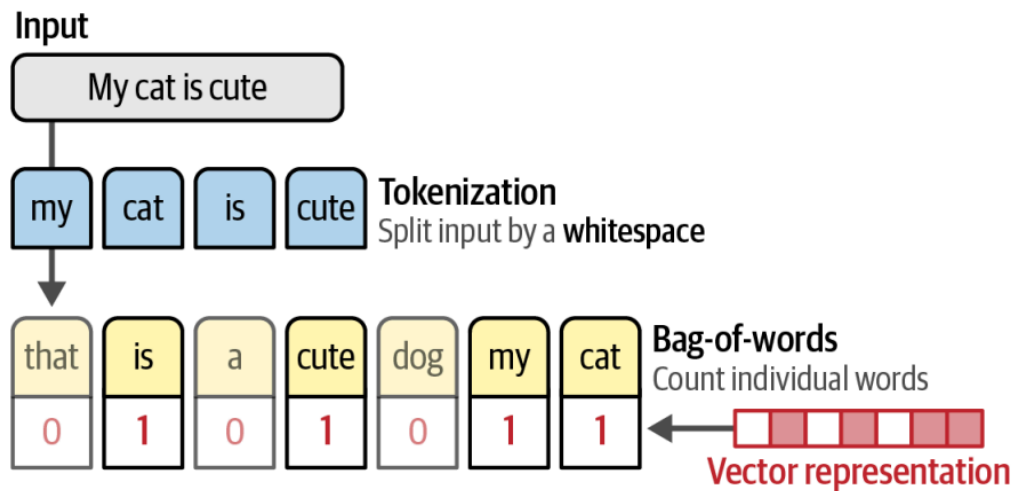


رایج‌ترین روش برای تجزیه به کلمه‌ها، تقسیم جملات بر اساس فاصله‌های خالی است تا کلمات جداگانه به دست آید. با این حال، این روش دارای معایبی است؛ برای مثال، برخی از زبان‌ها مانند چینی به طور طبیعی فاصله‌ای بین کلمات ندارند. در فصل بعد، به طور عمیق‌تر به فرآیند تجزیه به کلمات پرداخته و بررسی خواهیم کرد که این تکنیک چگونه بر مدل‌های زبانی تأثیر می‌گذارد.

همانطور که در شکل زیر نشان داده شده است، پس از تجزیه به کلمه‌ها، تمام کلمات منحصر به فرد از هر جمله جمع‌آوری می‌شوند تا یک واژگان (Vocabulary) بسازیم که می‌توانیم از آن برای نمایش جملات استفاده کنیم.



با استفاده از این واژگان، به سادگی تعداد دفعاتی که هر کلمه در هر جمله ظاهر می‌شود را می‌شماریم، که عملاً یک کیسه‌کلمات ایجاد می‌شود. به این ترتیب، هدف مدل کیسه‌کلمات ایجاد نمایش‌هایی از متن به صورت اعداد است، که به طور معمول به آن‌ها نمایش‌های برداری (Vector Representations) یا لیست بردارها (Vectors) گفته می‌شود، همانطور که در شکل زیر مشاهده می‌کنید. از این به بعد، ما به این نوع مدل‌ها به عنوان مدل‌های نمایش (Representation Models) اشاره خواهیم کرد.



اگرچه مدل کیسه‌کلمات یک روش کلاسیک است، اما به هیچ وجه منسوخ نشده است. در فصل‌های آتی، بررسی خواهیم کرد که چگونه هنوز می‌توان از آن برای تکمیل مدل‌های زبانی جدیدتر استفاده کرد.

نمایش‌های بهتر با بردارهای متراکم (Embedding)

.12 .78 .33 .29 .88 .09 .73

Vector Embeddings

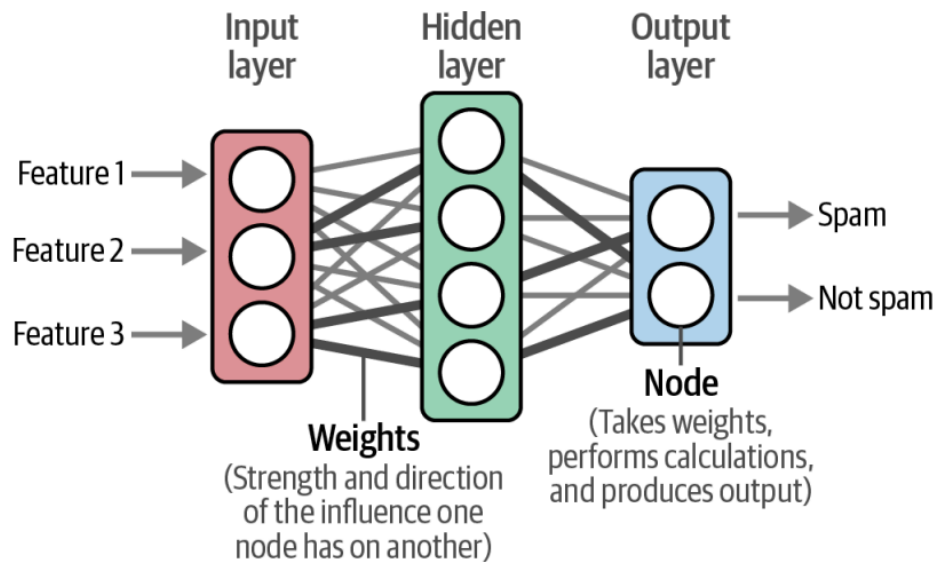
روش کیسه‌کلمات، اگرچه روشی زیباست، اما یک نقص دارد. این روش زبان را چیزی جز یک کیسه‌ی تقریباً لغوی از کلمات نمی‌داند و ماهیت معنایی یا معنی متن را نادیده می‌گیرد.

- **Bag-of-Words** does not consider the semantic nature of text
- **Word2Vec** could capture meaning of words in vector embeddings through neural networks.

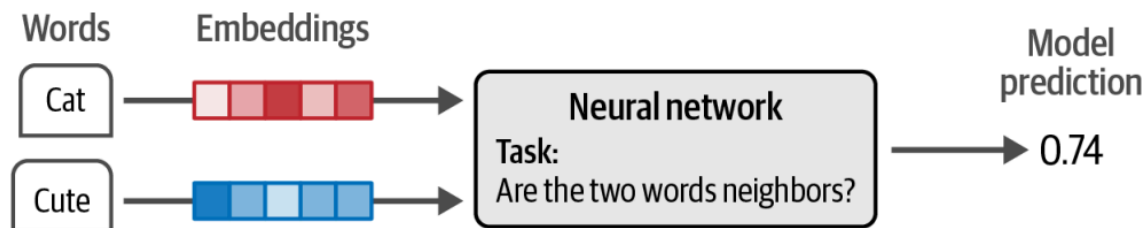
در سال 2013، word2vec یکی از نخستین تلاش‌های موفق برای درک معنای متن و نمایش آن در قالب بردارها (Embeddings) بود. بردارها در واقع نمایش‌های برداری از داده‌ها هستند که تلاش می‌کنند معنی آن‌ها را به تصویر کشند. برای این کار، word2vec با استفاده از داده‌های متنی فراوان، مانند تمام مطالب ویکی‌پدیا، نمایش‌های معنایی کلمات را یاد می‌گیرد.

برای تولید این نمایش‌های معنایی، word2vec از شبکه‌های عصبی (Neural Networks) بهره می‌برد. این شبکه‌ها از لایه‌های متصل به هم و از گره‌هایی تشکیل شده‌اند که اطلاعات را پردازش می‌کنند. همانطور که در

شکل زیر نشان داده شده است، شبکه‌های عصبی می‌توانند لایه‌های زیادی داشته باشند که هر اتصال در آن‌ها دارای وزن خاصی است که بسته به ورودی تغییر می‌کند. این وزن‌ها معمولاً به عنوان پارامترهای مدل (Model Parameters) شناخته می‌شوند.



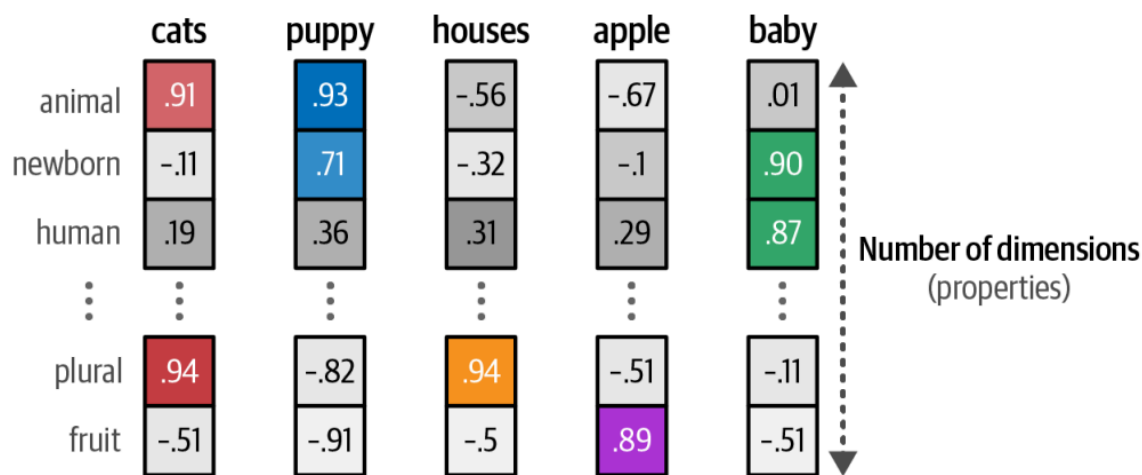
با استفاده از این شبکه‌های عصبی، word2vec بردارهای کلمات (Word Embeddings) را از طریق مشاهده کلماتی که تمایل دارند در کنار یکدیگر در یک جمله ظاهر شوند، تولید می‌کند. ما ابتدا به هر کلمه در واژگان خود یک بردار به طول 50 اختصاص می‌دهیم که به طور تصادفی مقداردهی اولیه می‌شود. سپس در هر مرحله از آموزش، همانطور که در شکل زیر نشان داده شده است، جفت کلماتی از داده‌های آموزشی انتخاب می‌کنیم و مدل سعی می‌کند پیش‌بینی کند که آیا آن‌ها احتمالاً همسایه‌های یکدیگر در جمله خواهند بود یا خیر.



در طول این فرآیند آموزش، word2vec رابطه بین کلمات را می‌آموزد و این اطلاعات را به صورت بردار در می‌آورد. اگر دو کلمه تمایل داشته باشند که همسایگان یکسانی داشته باشند، بردارهای آن‌ها به هم نزدیک‌تر خواهند بود و برعکس. در فصل بعدی، به طور دقیق‌تر به فرآیند آموزش word2vec خواهیم پرداخت.

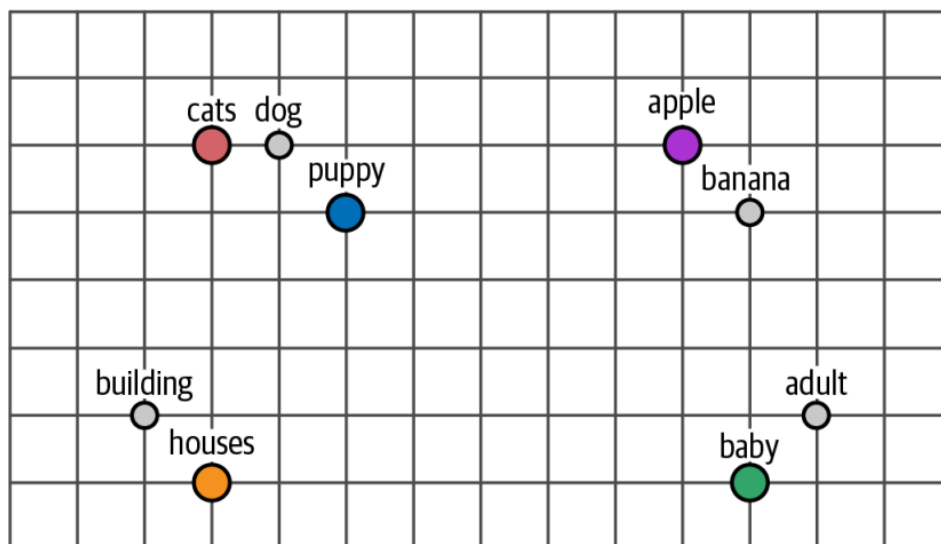
اما بردارهای حاصل، معنی کلمات را چگونه نمایان می‌سازند؟ برای روشن‌تر کردن این پدیده، فرض کنیم که بردارهای چندین کلمه، مانند "سیب" (Apple) و "کودک" (Baby) را داریم. بردارها سعی دارند معنی را با نمایش ویژگی‌های کلمات بیان کنند. برای مثال، کلمه "کودک" ممکن است در ویژگی‌های "نوزاد" (Newborn) و "انسان" (Human) امتیاز بالایی کسب کند، در حالی که کلمه "سیب" در این ویژگی‌ها امتیاز پایین‌تری خواهد داشت.

همانطور که در شکل زیر نشان داده شده است، بردارها می‌توانند ویژگی‌های زیادی برای نشان دادن معنی یک کلمه داشته باشند. از آنجایی که اندازه بردارها ثابت است، ویژگی‌های آن‌ها به گونه‌ای انتخاب می‌شوند که یک نمایه ذهنی از کلمه ایجاد کنند.



در عمل، این ویژگی‌ها معمولاً نسبت به مفاهیم واحد یا شناسایی‌شده‌ی انسانی نه‌چندان واضح هستند. اما با این حال، مجموع این ویژگی‌ها به‌طور معنی‌داری برای کامپیوترها قابل‌فهم است و به‌عنوان روشی خوب برای ترجمه زبان انسانی به زبان کامپیوتر عمل می‌کند.

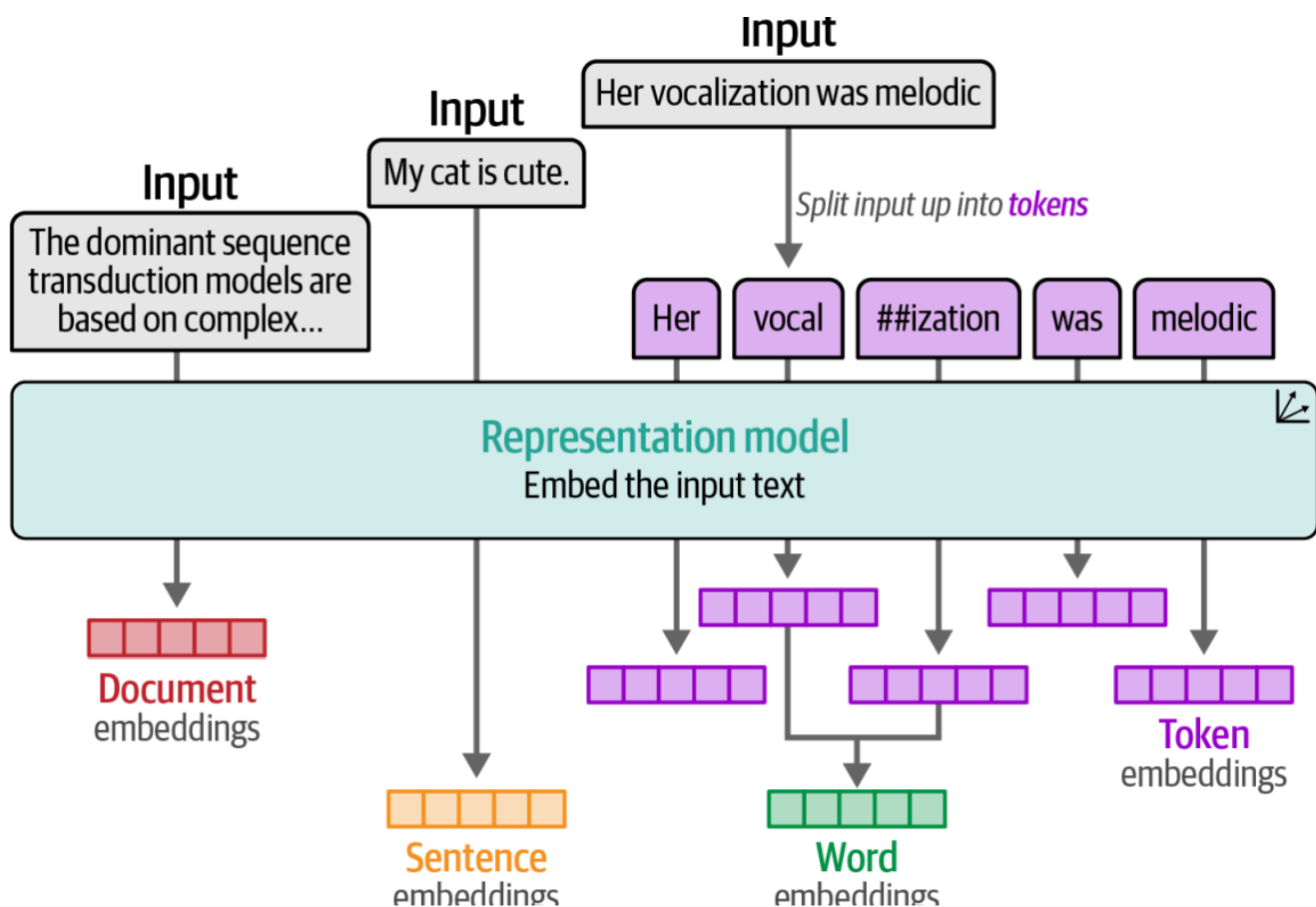
بردارها از این رو بسیار مفید هستند که به ما این امکان را می‌دهند تا شباهت معنایی بین دو کلمه را اندازه‌گیری کنیم. با استفاده از معیارهای مختلف فاصله، می‌توانیم قضاوت کنیم که یک کلمه چقدر به کلمه‌ای دیگر نزدیک است. همانطور که در شکل زیر نشان داده شده است، اگر این بردارها را به فضای دو بعدی فشرده کنیم، متوجه خواهیم شد که کلمات با معنای مشابه تمایل دارند که به یکدیگر نزدیک‌تر باشند. در فصول آتی، به‌طور مفصل‌تر بررسی خواهیم کرد که چگونه می‌توان این بردارها را به فضای چندبعدی فشرده کرد.



بردارهای کلمات مشابه در فضای ابعادی به یکدیگر نزدیک‌تر خواهند بود.

انواع بردارهای تعبیه‌شده (Embeddings)

بردارهای تعبیه‌شده (Embeddings) انواع مختلفی دارند که شامل بردارهای کلمات (Word Embeddings) و بردارهای جملات (Sentence Embeddings) می‌شوند. این انواع، سطوح مختلفی از انتزاع (کلمه در مقابل جمله) را نمایش می‌دهند که در شکل زیر نشان داده شده است.



برای مثال، روش کیسه کلمات (Bag-of-Words) بردارهایی را در سطح سند ایجاد می کند، چراکه کل سند را نمایش می دهد. در مقابل، word2vec فقط بردارهای کلمات را تولید می کند و معنای هر کلمه را به صورت جداگانه مدل سازی می کند.

در طول این نوشتار، بردارهای تعبیه شده نقشی محوری خواهند داشت، چراکه در بسیاری از موارد کاربردی مورد استفاده قرار می گیرند، از جمله:

- دسته بندی متن (Classification)
- خوشه بندی متن (Clustering)
- جستجوی معنایی (Semantic Search)
- تولید تقویت شده با بازایی اطلاعات (Retrieval Augmented Generation)

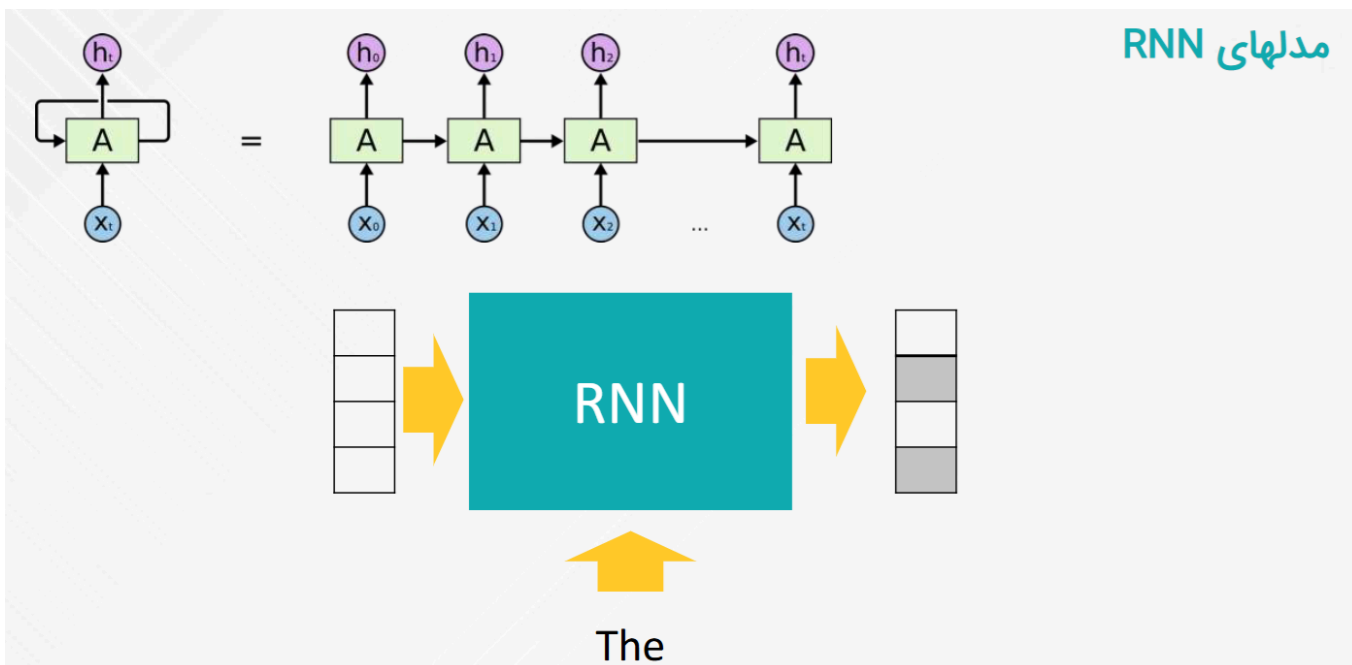
در فصل بعد، به طور عمیق تر به بررسی بردارهای تعبیه شده توکن (Token Embeddings) خواهیم پرداخت.

رمزگذاری و بازسازی (رمزگشایی) متن با استفاده از مکانیسم توجه

فرآیند آموزش word2vec، بازنمایی های ثابتی از کلمات ایجاد می کند. به عنوان مثال، کلمه ی "bank" همیشه دارای یک بردار تعبیه شده (Embedding) ثابت خواهد بود، صرف نظر از اینکه در چه متنی استفاده شده است. اما "bank" می تواند به بانک مالی (Financial Bank) یا کرانه ی رودخانه (River Bank) اشاره داشته باشد. بنابراین، معنای آن، و در نتیجه بردار تعبیه شده ی آن، باید بسته به متن تغییر کند.

در ادامه گامی در جهت رمزگذاری این مفهوم از طریق شبکه های عصبی بازگشتی (Recurrent Neural Networks - RNNs) برداشته شد. RNN ها گونه ای از شبکه های عصبی هستند که می توانند دنباله های متنی را مدل سازی کنند.

مدلهای RNN



مشکل فراموشی RNN ها

? my tea tastes Great

درک زبان در مدل‌های هوش مصنوعی چالش دارد...

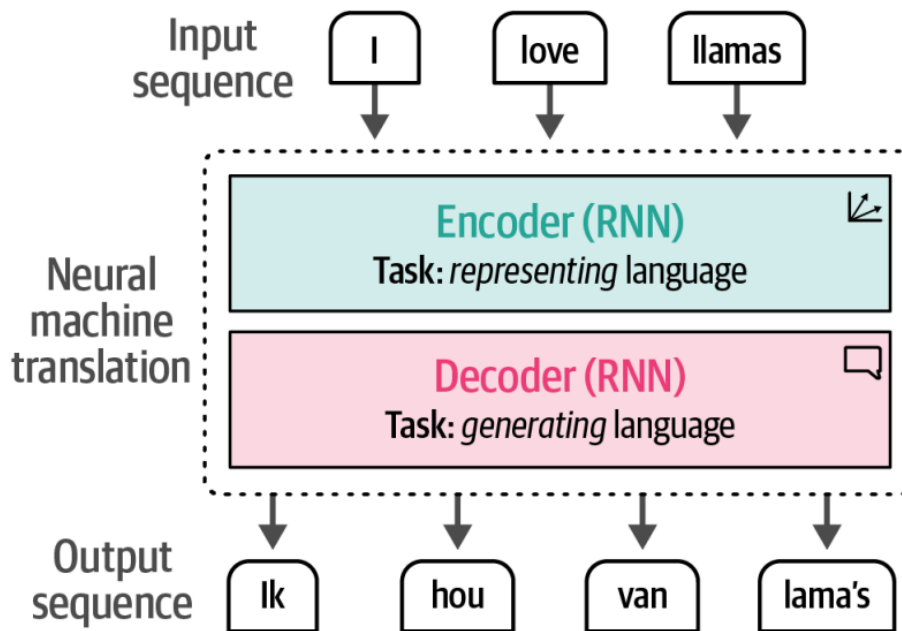
I took my money to the bank.

River bank?

برای انجام این کار، از RNN‌ها در دو وظیفه اصلی استفاده می‌شود:

- رمزگذاری (Encoding): ایجاد یک شهود یا بازنمایی از جمله‌ی ورودی
- رمزگشا (Decoding): تولید جمله‌ی خروجی با استفاده از شهود قبلی

در شکل زیر این مفهوم نشان داده شده است، که نحوه‌ی ترجمه‌ی جمله‌ی "I love llamas" از انگلیسی به "Ik hou van lama's" در زبان هلندی را نمایش می‌دهد.

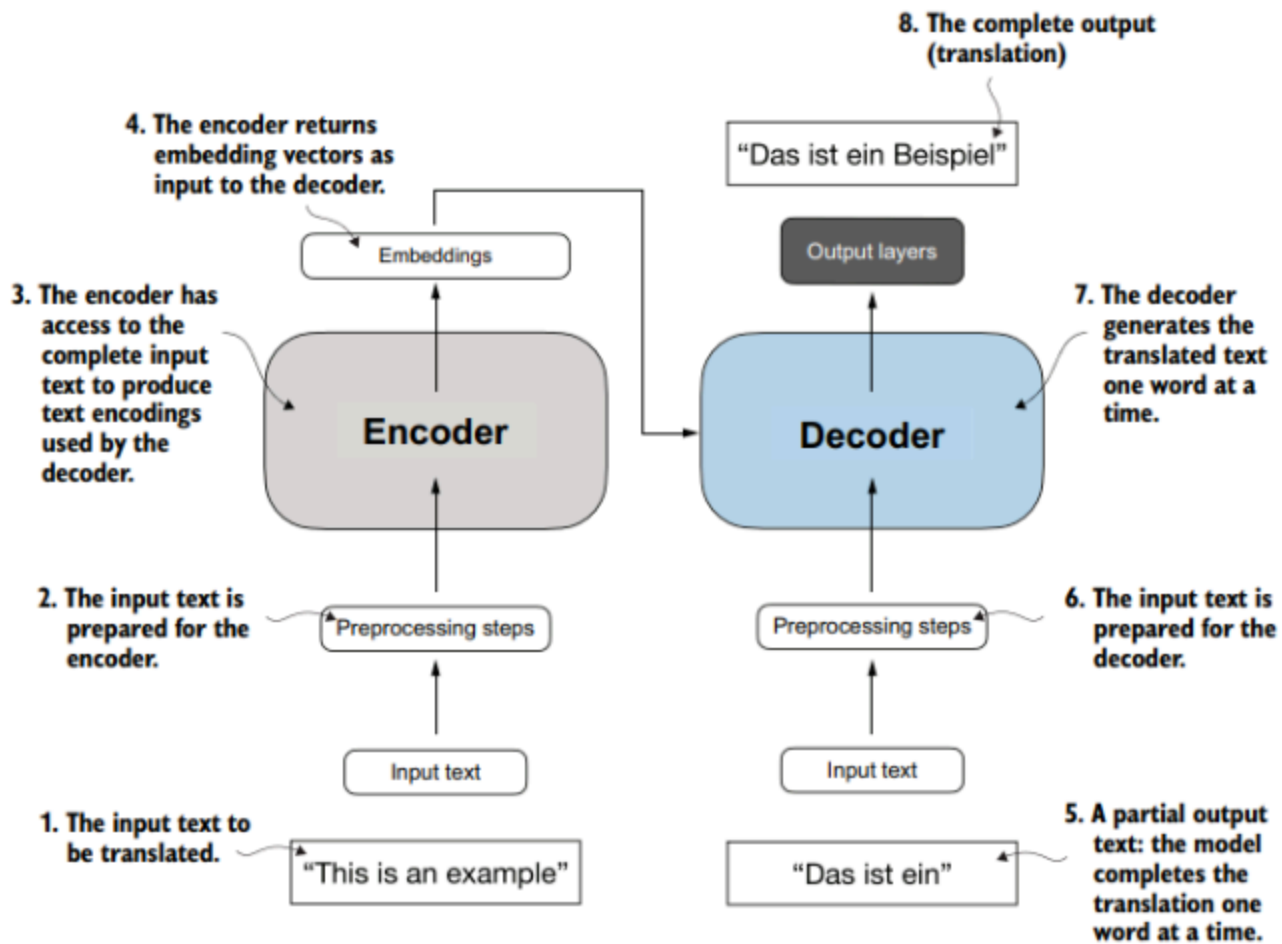


برای مثال از این معماری میتوان در وظیفه ترجمه زبانی استفاده کرد در این تسک:

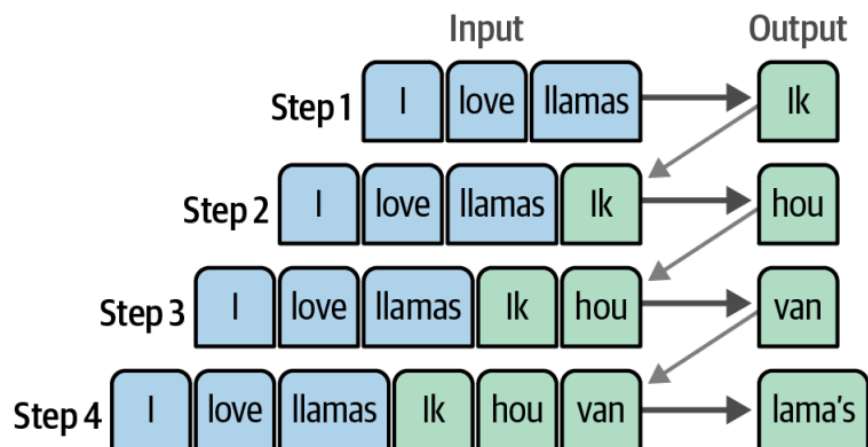
(الف) رمزگذار (Encoder) متن ورودی را پردازش کرده و یک نمایش بردار تعبیه شده (Embedding Representation) ایجاد می‌کند (در واقع بردار امبدینگ شامل چند عدد است که هریک عوامل مختلفی را در

ابعاد گوناگون در برمی گیرد) تولید می کند

(ب) رمزگشا (Decoder) که از این نمایش برای تولید متن ترجمه شده، به صورت کلمه به کلمه، استفاده می کند.

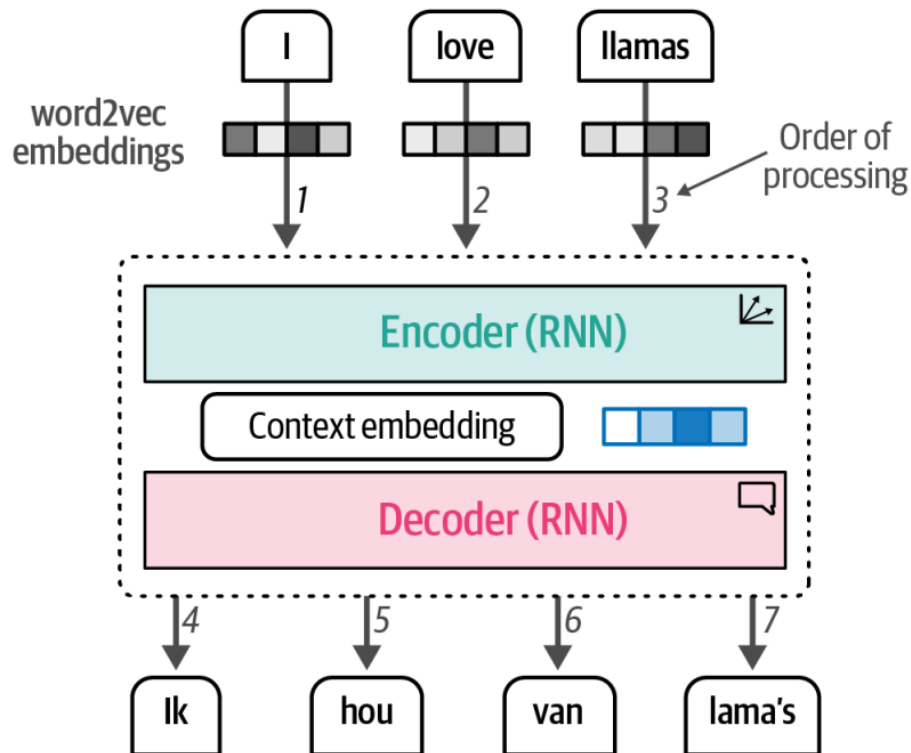


در مدل های بازگشتی، تولید هر کلمه یا هر گام وابسته به خروجی گام قبلی است. هنگام تولید کلمه ی بعدی، مدل باید تمامی کلمات تولیدشده ی قبلی را دریافت کند، همان طور که در شکل زیر نشان داده شده است.



هدف مرحله ی رمزگذاری (Encoding)، ارائه ی نمایشی (بازنمایی) مناسب از جمله ی ورودی است که زمینه ی معنایی (ccntext vector) را در قالب یک بردار تعبیه شده فراهم کند. این بردار ورودی مرحله ی رمزگشایی (Decoding) خواهد بود. برای تولید این نمایش، مدل ابتدا بردارهای تعبیه شده ی کلمات

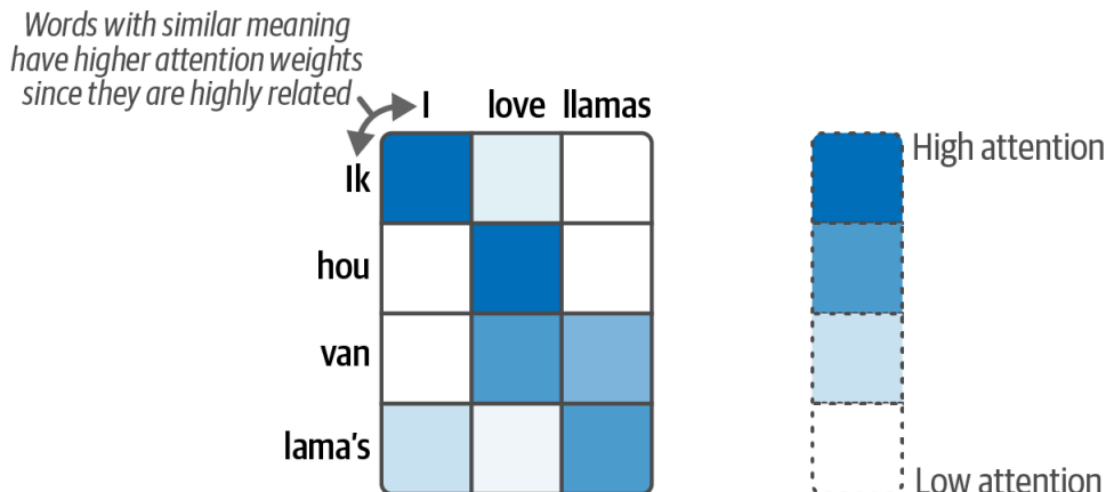
(Embeddings) را دریافت می‌کند، که می‌توانند از word2vec استخراج شوند. این فرآیند در شکل زیر نمایش داده شده است.



اما این بردار زمینه‌ای (context vector) مشکلاتی دارد، زیرا فقط یک نمایش کلی از جمله را ارائه می‌دهد، که باعث دشواری پردازش جملات بلند می‌شود. در سال 2014، راه‌حلی به نام مکانیسم توجه (Attention Mechanism) معرفی شد که به طور قابل توجهی این معماری را بهبود بخشید.

مکانیسم توجه (Attention Mechanism): تمرکز بر قسمت‌های مهم متن

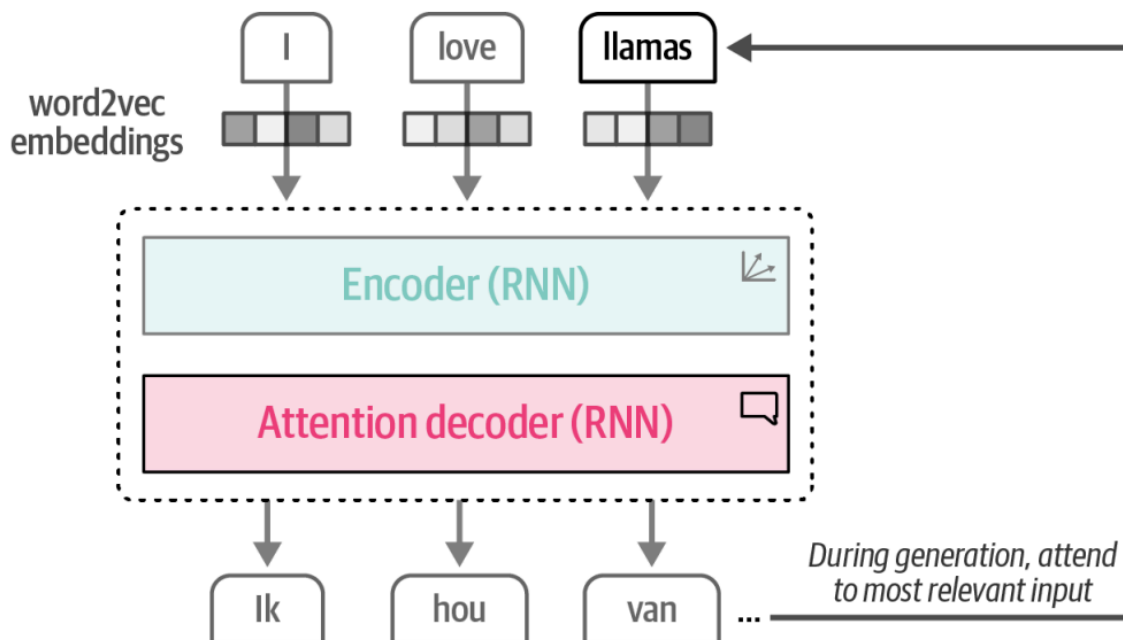
مکانیسم توجه به مدل این امکان را می‌دهد که بر بخش‌های مرتبط‌تر جمله تمرکز کند و سیگنال‌های معنایی مهم را تقویت کند. این ایده در شکل زیر نشان داده شده است. مثلاً در تسک ترجمه در حین تولید هر کلمه، مدل فقط کافی است تا به کلمه متناظر بنگرد.



برای مثال، کلمه‌ی خروجی "lama's" در زبان هلندی معادل "llamas" در انگلیسی است، به همین دلیل میزان توجه بین این دو کلمه بالا است. در مقابل، کلمات "I" و "love" ارتباط معنایی کمتری دارند، لذا مقدار توجه آن‌ها کمتر خواهد بود. در فصول آتی، به طور عمیق‌تر به مکانیسم توجه خواهیم پرداخت.

ادغام مکانیسم توجه در فرآیند رمزگشایی (Decoding)

با افزودن مکانیسم توجه به مرحله‌ی رمزگشایی (Decoding)، مدل (در اینجا RNN) می‌تواند سیگنال‌های معنایی هر کلمه را نسبت به خروجی احتمالی تقویت کند. به جای ارسال تنها بردار زمینه‌ای (Context Vector) به مرحله‌ی رمزگشایی (Decoding)، بردارهای مخفی (Embedding) تمامی کلمات ورودی ارسال می‌شوند. این فرآیند در شکل زیر نمایش داده شده است.



بهبود رمزگشایی متن با مکانیسم توجه

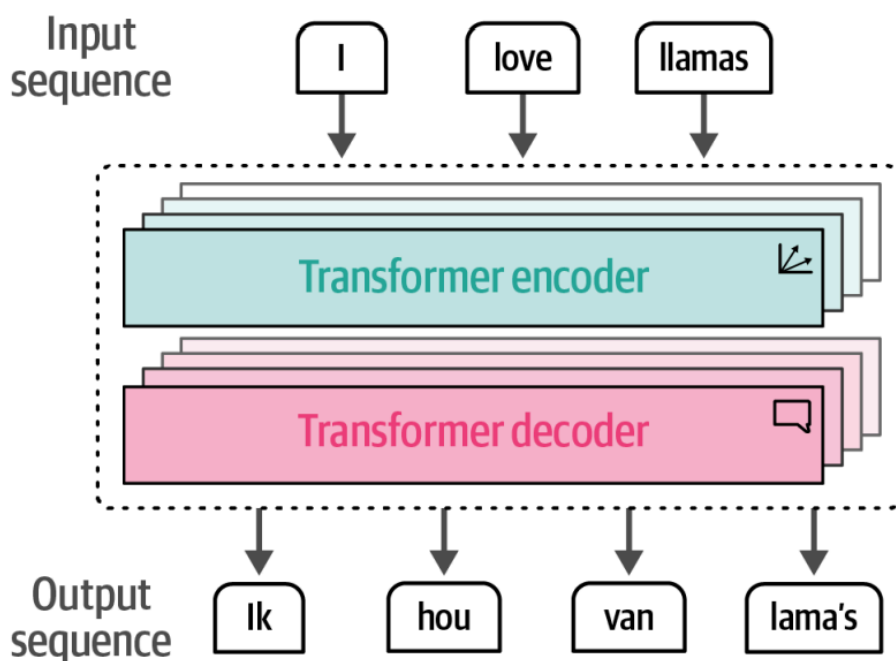
با استفاده از مکانیسم توجه، مدل در هنگام تولید عبارت "lk hou van lama's" می‌تواند تشخیص دهد که هنگام تولید "lama's"، باید بیشتر بر روی کلمه‌ی "llamas" در جمله‌ی ورودی تمرکز کند. در مقایسه با word2vec، این معماری قادر است ساختار دنباله‌ای زبان و متناظر بودن واژگان را بهتر مدل‌سازی کند.

با این حال، ماهیت دنباله‌ای (بازگشتی) این مدل‌ها مانع از اجرای موازی آن‌ها در حین آموزش می‌شود، که یکی از چالش‌های اصلی در این نوع شبکه‌ها محسوب می‌شود.

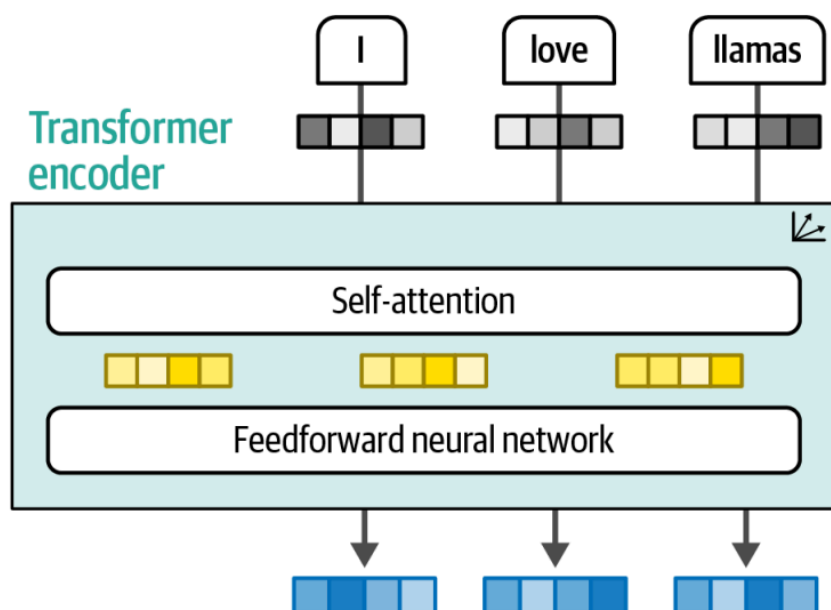
توجه همه چیز است: معرفی مدل ترنسفورمر

قدرت واقعی مکانیسم توجه و آنچه که توانایی‌های شگفت‌انگیز مدل‌های زبانی بزرگ را به وجود می‌آورد، اولین بار در مقاله‌ی معروف "Attention is all you need" که در سال 2017 منتشر شد، مورد بررسی قرار گرفت. در این مقاله، نویسندگان معماری شبکه‌ای به نام ترنسفورمر (Transformer) را معرفی کردند که کاملاً مبتنی بر مکانیسم توجه بود و سبب حذف معماری شبکه‌های بازگشتی که قبلاً مشاهده کرده بودیم شد. در مقایسه با شبکه‌های بازگشتی، ترنسفورمر قادر بود به طور موازی آموزش داده شود، که این امر باعث تسریع قابل توجه در فرآیند آموزش شد.

در مدل ترنسفورمر، اجزای رمزگذار و رمزگشا به صورت لایه‌لایه بر روی یکدیگر قرار گرفته‌اند، همان‌طور که در شکل زیر نشان داده شده است. این معماری هنوز هم وابسته به خروجی گام‌های قبلی است و برای تولید هر کلمه جدید، نیاز به مصرف کلمات تولیدشده قبلی دارد.

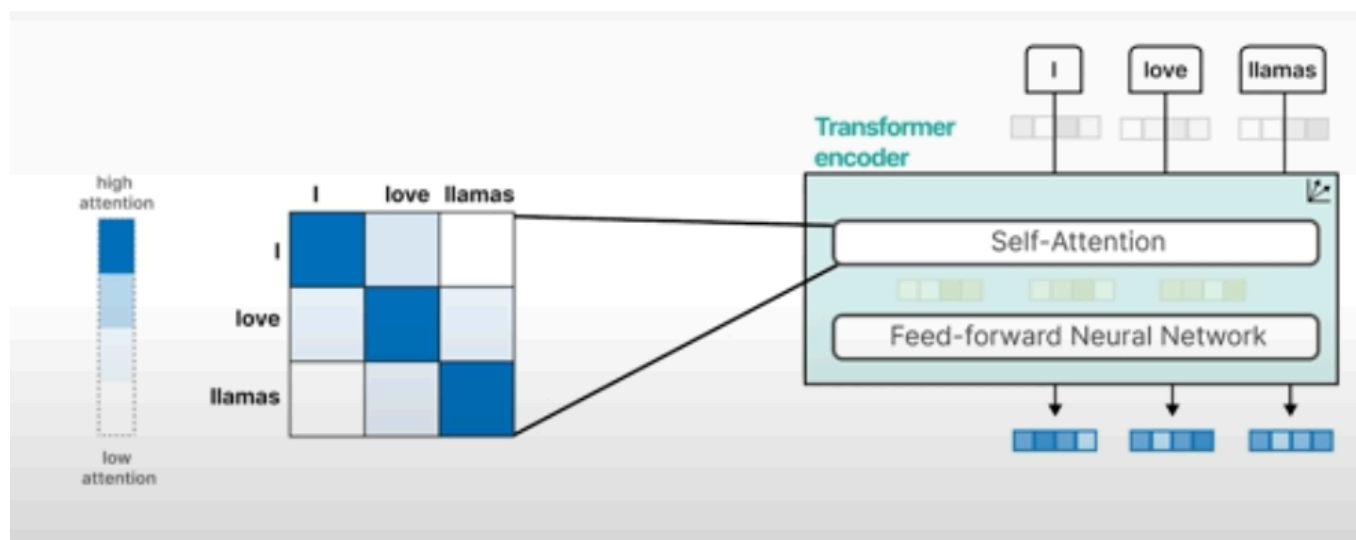
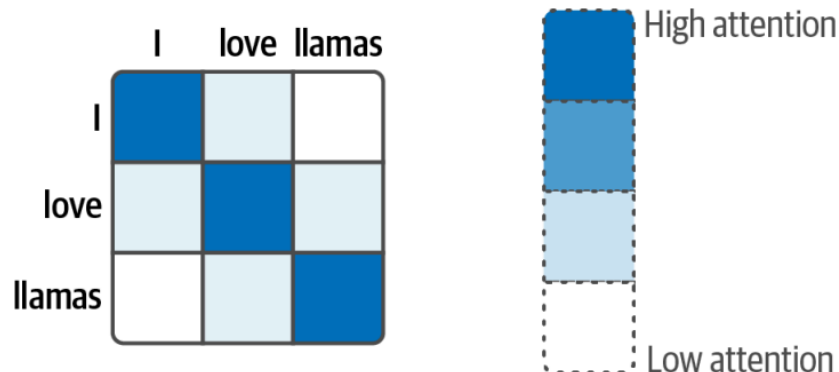


حالا، هر دو بلوک رمزگذار (Encoder) و رمزگشا (Decoder)، به جای استفاده از مدل RNN با ویژگی‌های توجه، حول مکانیسم توجه می‌چرخند. بلوک رمزگذار در ترنسفورمر شامل دو قسمت است: توجه به خود (Self-Attention) و یک شبکه عصبی متراکم (Feedforward Neural Network) که در شکل زیر نشان داده شده است.

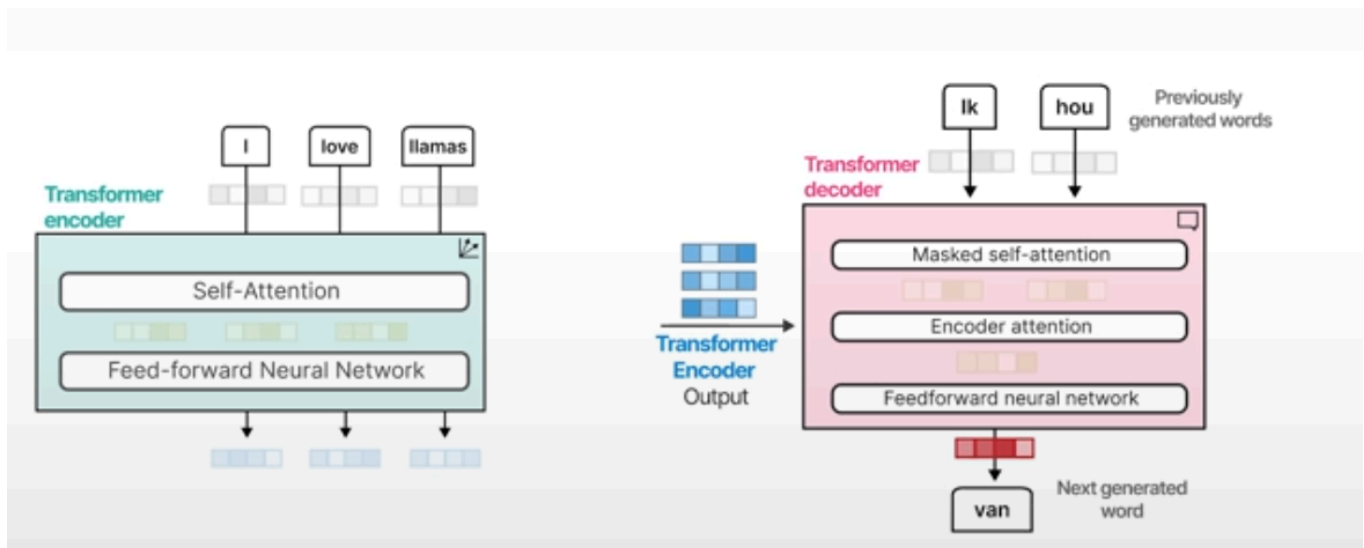
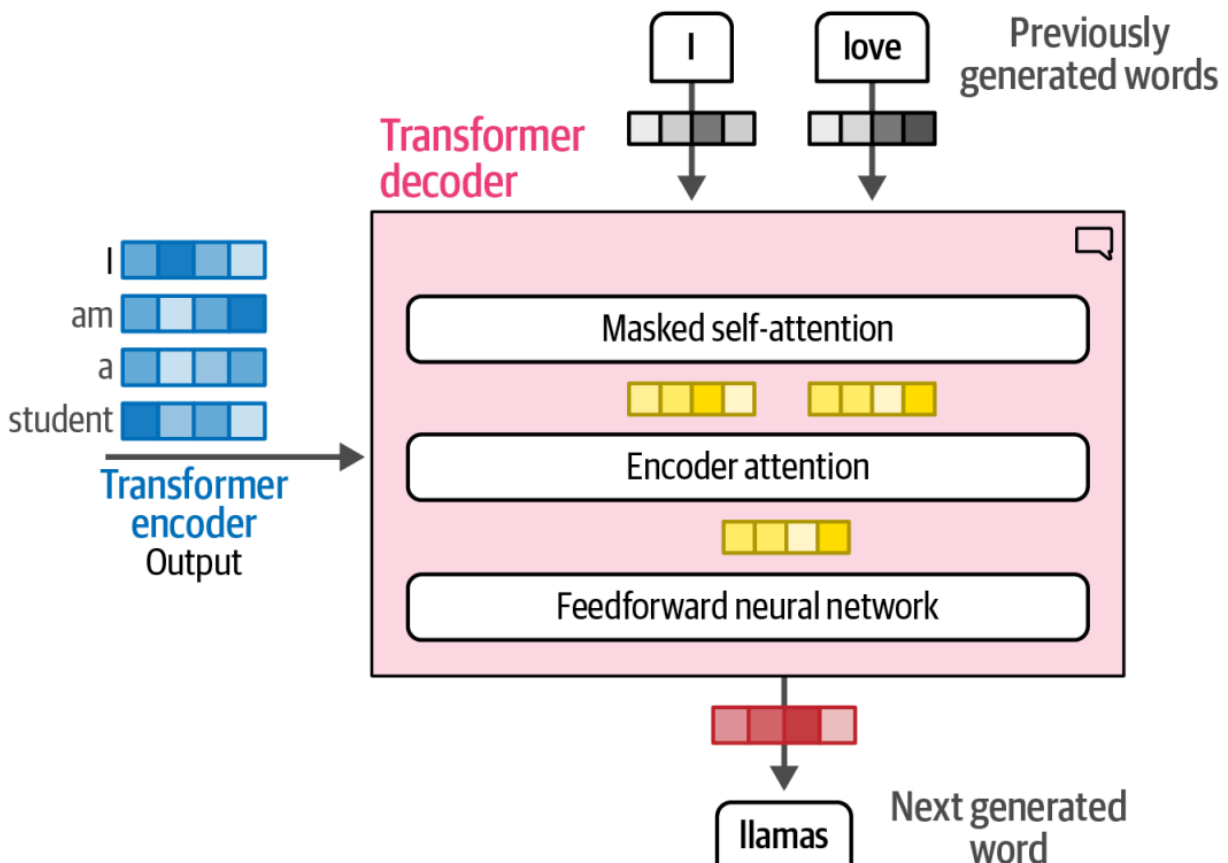


در مقایسه با روش‌های قبلی مکانیسم توجه، مدل **توجه به خود**، به جای توجه به دو متن مختلف (مثلاً در ترجمه)، فقط به موقعیت‌های مختلف در متن ورودی (دنباله) توجه کند (و آنرا با خودش مقایسه می‌کند)، که به این ترتیب، قادر است نمایش دقیق‌تر و بهتری از دنباله ورودی ایجاد کند، همان‌طور که در شکل زیر نشان

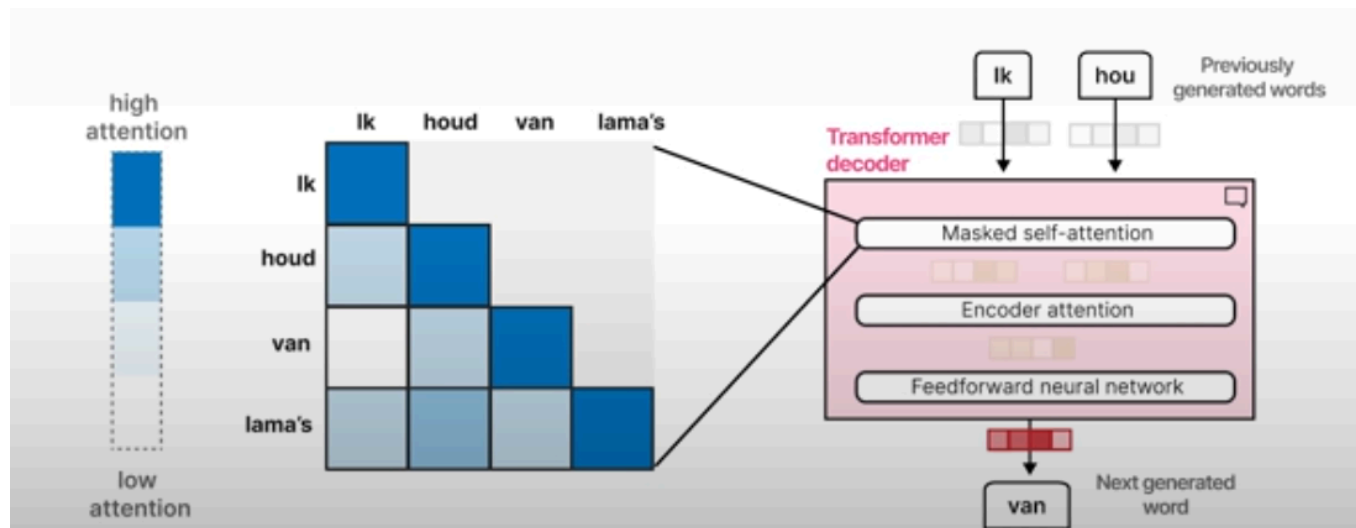
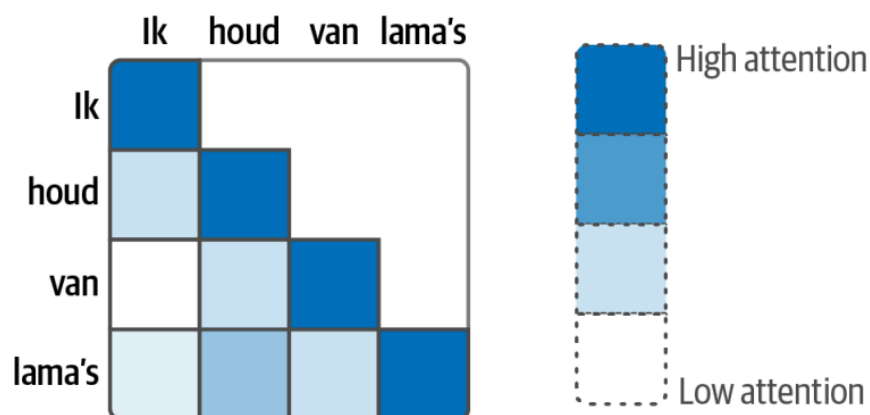
داده شده است. برخلاف پردازش توکن‌ها به صورت گام به گام، این مکانیسم قادر است به طور همزمان به کل دنباله نگاه کند.



در مقایسه با رمزگذار، رمزگشا یک لایه اضافی دارد که به خروجی رمزگذار توجه می‌کند تا بخش‌های مرتبط از ورودی را پیدا کند. همان‌طور که در شکل زیر نشان داده شده است، این فرآیند مشابه به مدل توجه در رمزگشا شبکه‌های RNN است که قبلاً توضیح داده شد.



همان‌طور که در شکل زیر نشان داده شده است، لایه توجه خودی در رمزگشا موقعیت‌های آینده را ماسک می‌کند تا فقط به موقعیت‌های قبلی توجه کند و از نشت اطلاعات به آینده در حین تولید خروجی جلوگیری کند.



تمامی این بلوک‌ها با هم معماری ترنسفورمر را تشکیل می‌دهند و پایه‌گذار بسیاری از مدل‌های تاثیرگذار در هوش مصنوعی زبان مانند BERT و GPT-1 هستند که در بخش‌های بعدی این فصل بررسی خواهند شد. در این نوشتار، بیشتر مدل‌هایی که استفاده خواهیم کرد، مدل‌های مبتنی بر ترنسفورمر خواهند بود.

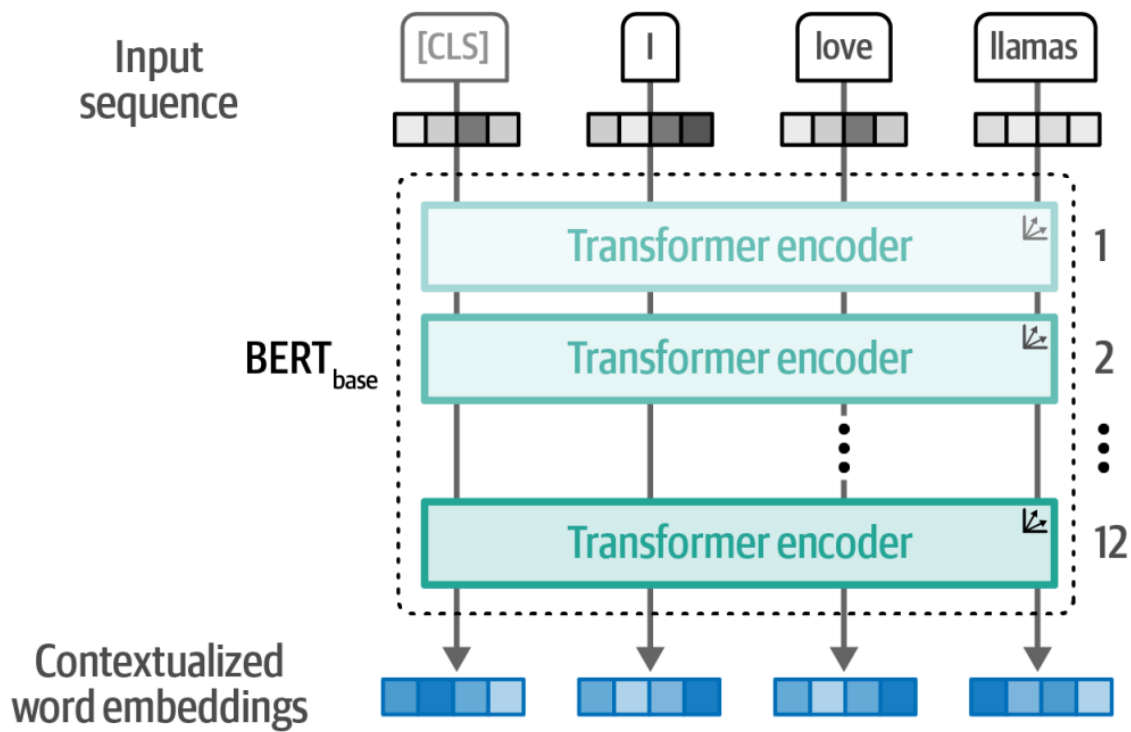
ترنسفورمر معماری پیچیده‌تری نسبت به آنچه که تاکنون بررسی کردیم دارد. در فصل‌های آتی، دلایل بسیاری که باعث موفقیت این مدل‌ها می‌شود، از جمله توجه چندگانه (Multi-Head Attention)، گنجاندن موقعیت (Positional Embeddings)، و نرمال‌سازی لایه‌ها (Layer Normalization) را به تفصیل بررسی خواهیم کرد.

مدل‌های نمایشی: مدل‌های فقط رمزگذار

مدل ترنسفورمر اولیه (Original Transformer Model) یک معماری رمزگذار-رمزگشا (Encoder-Decoder Architecture) بود که به خوبی برای وظایف ترجمه (Translation Tasks) عمل می‌کرد، اما به راحتی برای وظایف دیگر، مانند دسته‌بندی متن (Text Classification)، قابل استفاده نبود.

در سال 2018، معماری جدیدی به نام نمایش‌های رمزگذاری دوطرفه از ترنسفورمرها (Bidirectional Encoder Representations from Transformers - BERT) معرفی شد که امکان استفاده از آن در طیف گسترده‌ای از وظایف وجود داشت و در سال‌های آینده، پایه‌ی هوش مصنوعی زبانی شد.

مدل BERT یک معماری فقط رمزگذار (Encoder-Only Architecture) است که بر نمایش زبان تمرکز دارد، همان‌طور که در شکل زیر نشان داده شده است. این بدان معناست که فقط از رمزگذار استفاده می‌کند و رمزگشا را کاملاً حذف کرده است.



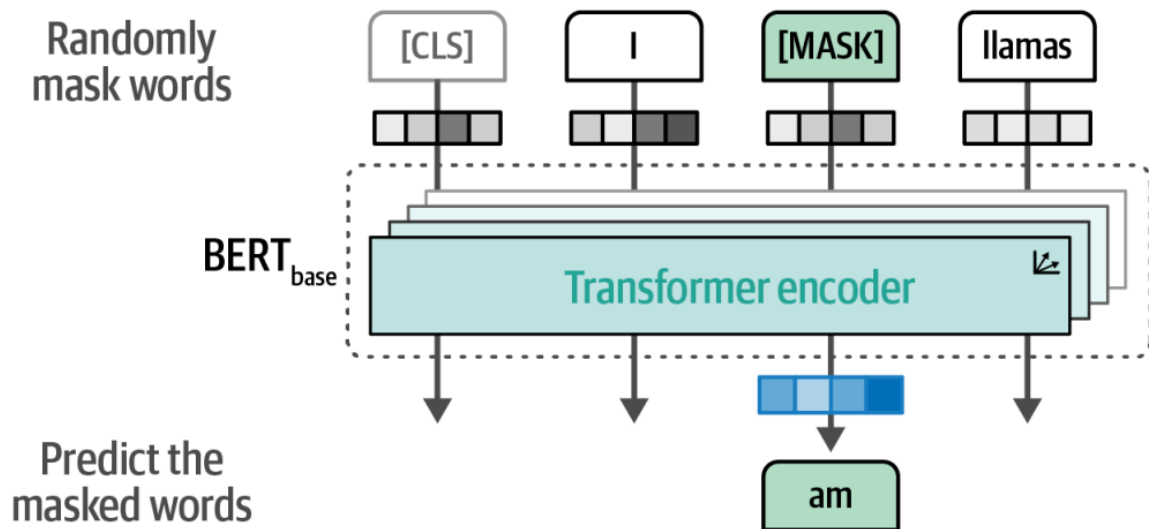
ساختار مدل BERT

بلوک‌های رمزگذار در BERT مشابه آنچه در مدل ترنسفورمر دیده‌ایم، از دو بخش اصلی تشکیل شده‌اند:

- مکانیسم توجه به خود (Self-Attention)
- شبکه عصبی پیشخور (Feedforward Neural Network)

ورودی مدل شامل یک توکن اضافی به نام [CLS] (Classification Token) است که نماینده‌ی کل ورودی متن است. معمولاً، این توکن [CLS] برای تنظیم دقیق مدل در وظایف خاص مانند دسته‌بندی متن استفاده می‌شود.

آموزش این پشته‌های رمزگذار (encoder stacks) می‌تواند کار دشواری باشد، که BERT با استفاده از تکنیکی به نام **مدل‌سازی زبان با ماسک (masked language modeling)** با آن مقابله می‌کند. همان‌طور که در شکل زیر نشان داده شده است، این روش بخشی از ورودی را پنهان (ماسک) می‌کند تا مدل آن را پیش‌بینی کند. این کار پیش‌بینی، دشوار است، اما به BERT کمک می‌کند تا نمایش‌های میانی (intermediate representations) دقیق‌تری از ورودی ایجاد کند. (در واقع این بخش معادل همان bidirectional LSTM است)

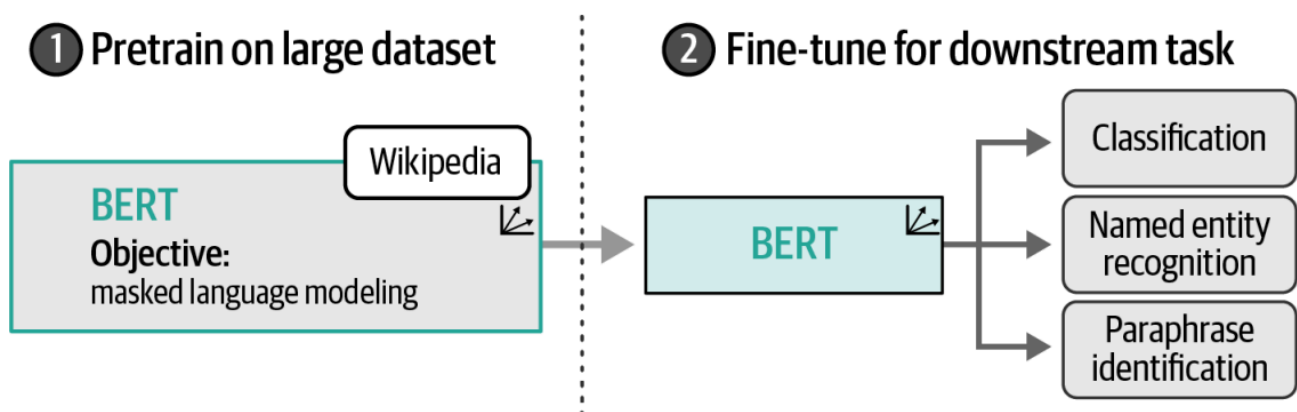


آموزش و تنظیم دقیق BERT

این معماری و روش آموزش، باعث می‌شود BERT و معماری‌های مشابه آن در بازنمایی زبان زمینه‌ای بسیار قدرتمند باشند.

مدل‌های مشابه BERT معمولاً برای یادگیری انتقالی (Transfer Learning) استفاده می‌شوند. این فرآیند شامل دو مرحله است:

1. پیش‌آموزش (pre train) مدل برای مدل‌سازی زبان (بر روی مجموعه داده‌هایی مانند ویکی‌پدیا)
 2. تنظیم دقیق (fine-tune) مدل بر روی وظایف خاص (مانند دسته‌بندی متن)
- برای مثال، هنگامی که BERT بر روی کل مجموعه‌ی ویکی‌پدیا آموزش داده شود، می‌تواند ماهیت معنایی و متنی جملات را یاد بگیرد. سپس، همان‌طور که در شکل زیر نشان داده شده است، می‌توانیم این مدل پیش‌آموزش‌یافته را برای یک وظیفه‌ی خاص مانند دسته‌بندی متن تنظیم دقیق کنیم.



مزایای مدل‌های پیش‌آموزش‌یافته

یکی از مزایای بزرگ مدل‌های پیش‌آموزش‌یافته (pretrained models) این است که بخش عمده‌ای از فرآیند آموزش از قبل انجام شده است. این ویژگی باعث می‌شود که تنظیم دقیق (Fine-tuning) مدل بر روی وظایف خاص عموماً نیاز به منابع محاسباتی کمتری داشته باشد و به داده‌های کمتری نیاز باشد.

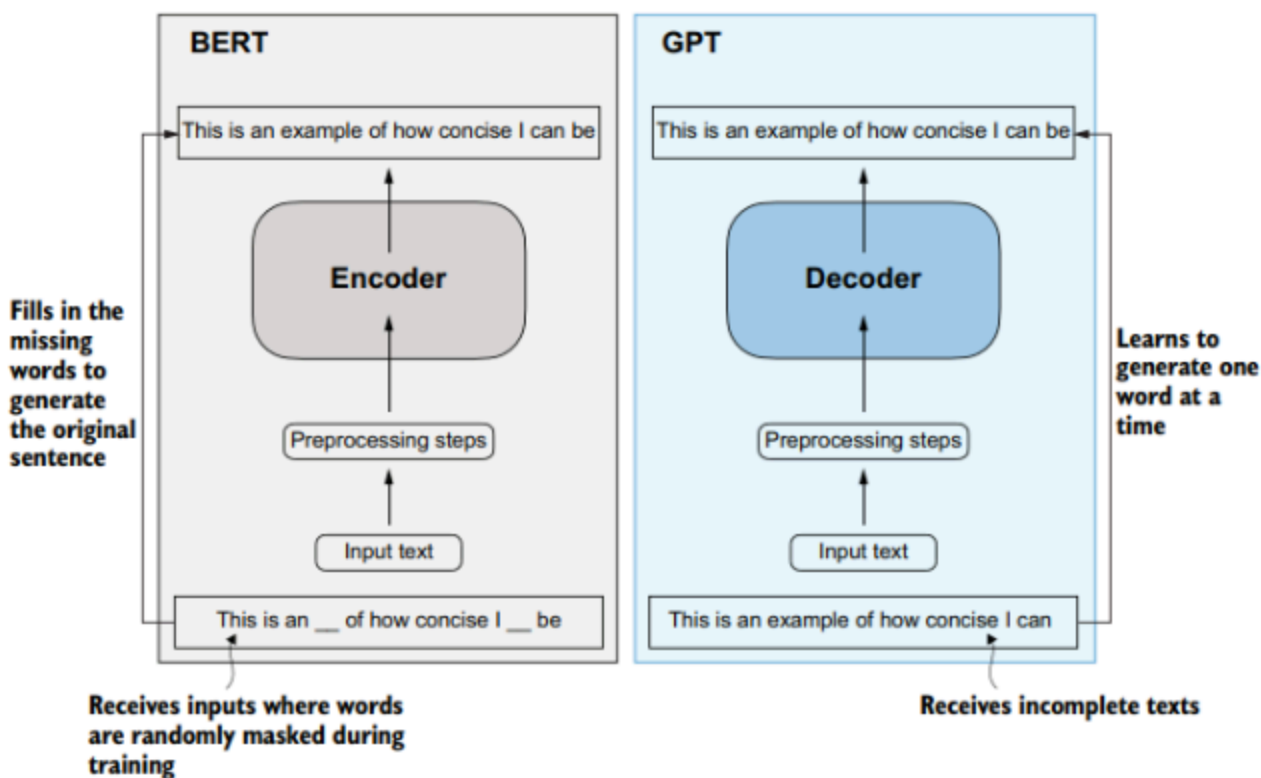
علاوه بر این، مدل‌های مشابه BERT در تقریباً تمام مراحل معماری خود، بردارهای تعبیه‌شده (Embedding) تولید می‌کنند. این امر باعث می‌شود که مدل‌های BERT به‌عنوان ابزار استخراج ویژگی بدون نیاز به تنظیم دقیق (fine tune) برای وظایف خاص، قابل استفاده باشند.

مدل‌های فقط رمزگذار (Encoder-Only Models)، مانند BERT، در بخش‌های زیادی از این نوشتار مورد استفاده قرار خواهند گرفت. برای سال‌ها، این مدل‌ها برای وظایف رایج مانند دسته‌بندی (Classification Tasks)، خوشه‌بندی (Clustering Tasks)، و جستجوی معنایی (Semantic Search) مورد استفاده قرار گرفته و هنوز هم استفاده می‌شوند.

در سرتاسر این نوشتار، ما به مدل‌های فقط رمزگذار به‌عنوان مدل‌های نمایشی (Representation Models) اشاره خواهیم کرد تا آنها را از مدل‌های فقط رمزگشا (Decoder-Only Models) که به‌عنوان مدل‌های تولیدی (Generative Models) شناخته می‌شوند، متمایز کنیم.

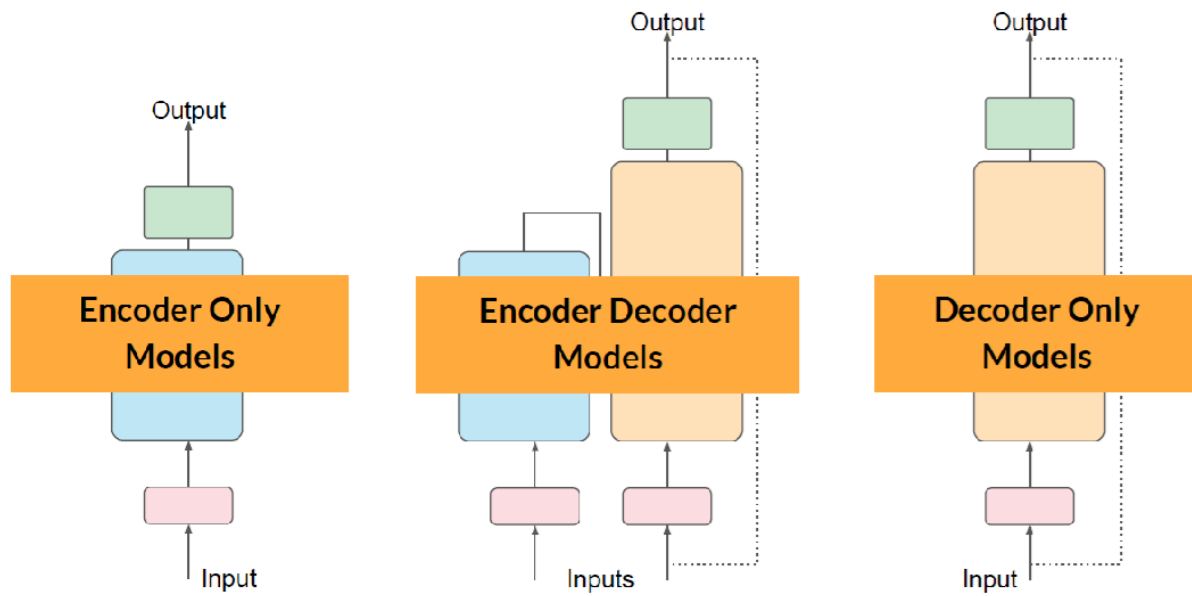
توجه داشته باشید که تمایز اصلی در اینجا نه در معماری این مدل‌ها و نحوه‌ی عملکرد آنهاست، بلکه در هدف و کاربرد آنها است. مدل‌های نمایشی عمدتاً بر نمایش زبان تمرکز دارند، به‌طور مثال با ایجاد بردارهای تعبیه‌شده، و معمولاً متن تولید نمی‌کنند. در حالی که مدل‌های تولیدی عمدتاً بر تولید متن تمرکز دارند و معمولاً برای تولید بردارهای تعبیه‌شده آموزش داده نمی‌شوند.

در سمت چپ شکل زیر، بخش رمزگذار نمایانگر مدل‌های زبانی بزرگی همچون برت (BERT) است که بر پیش‌بینی کلمات پوشیده‌شده تمرکز دارند و عمدتاً در وظایفی مانند طبقه‌بندی متن (Text Classification) به کار می‌روند. در مقابل، بخش رمزگشا که در سمت راست دیده می‌شود، نشان‌دهنده مدل‌هایی نظیر جی‌پی‌تی (GPT) است که برای تولید متن طراحی شده‌اند و قادرند توالی‌های متنی روان و منسجم ایجاد کنند.



این تمایز میان مدل‌های نمایشی و مدل‌های تولیدی در بیشتر تصاویر نوشتار نیز نمایش داده خواهد شد. مدل‌های نمایشی به رنگ آبی فیروزه‌ای با یک آیکون کوچک بردار نشان داده می‌شوند تا بر تمرکز آنها بر

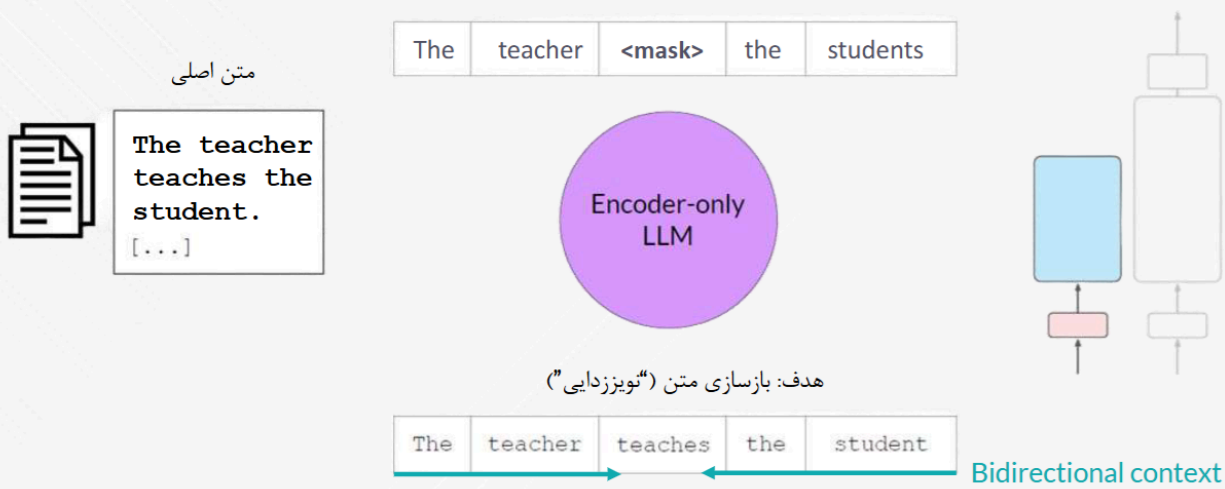
بردارها و بردارهای تعبیه‌شده (Embeddings) تأکید شود، در حالی که مدل‌های تولیدی به رنگ صورتی با یک آیکون کوچک چت نشان داده می‌شوند تا بر قابلیت‌های تولید متن آنها تأکید شود.



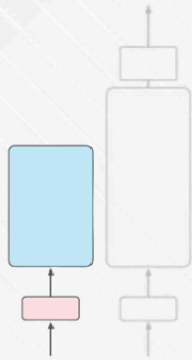
مدلهای فقط رمز گذار

مدل‌های خودرمزگذار (Autoencoding Models)

مدل‌سازی زبان ماسک‌شده
Masked Language Modeling (MLM) یا



مدل‌های خودرمزگذار (Autoencoding Models)



❑ موارد استفاده مناسب:

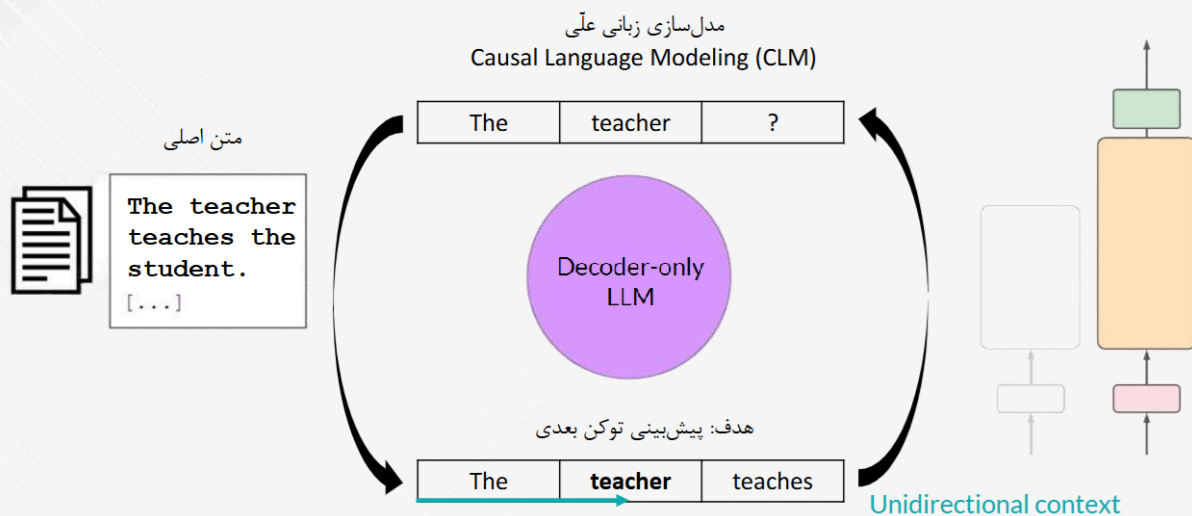
- ✓ تحلیل احساسات
- ✓ تشخیص موجودیت‌های نامدار (NER)
- ✓ طبقه‌بندی کلمات

❑ نمونه مدل‌ها:

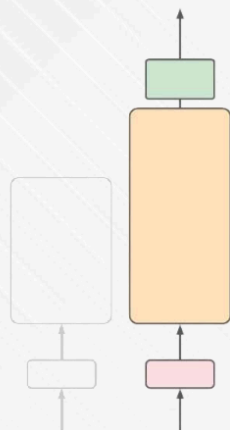
- (Bidirectional Encoder Representations from Transformers) BERT
- (Robustly Optimized BERT Pretraining Approach) RoBERTa

مدل‌های فقط رمز گشا

مدل‌های خودهمبسته (Autoregressive)



مدل‌های خودهمبسته (Autoregressive)



❑ موارد استفاده مناسب:

✓ تولید متن (Text generation)

❑ سایر کاربردهای جدید و نوظهور:

✓ بستگی به اندازه مدل دارد

❑ نمونه مدل‌ها:

○ (Generative Pre-trained Transformer) GPT

○ (BigScience Large Open-science Open-access Multilingual Language Model) BLOOM

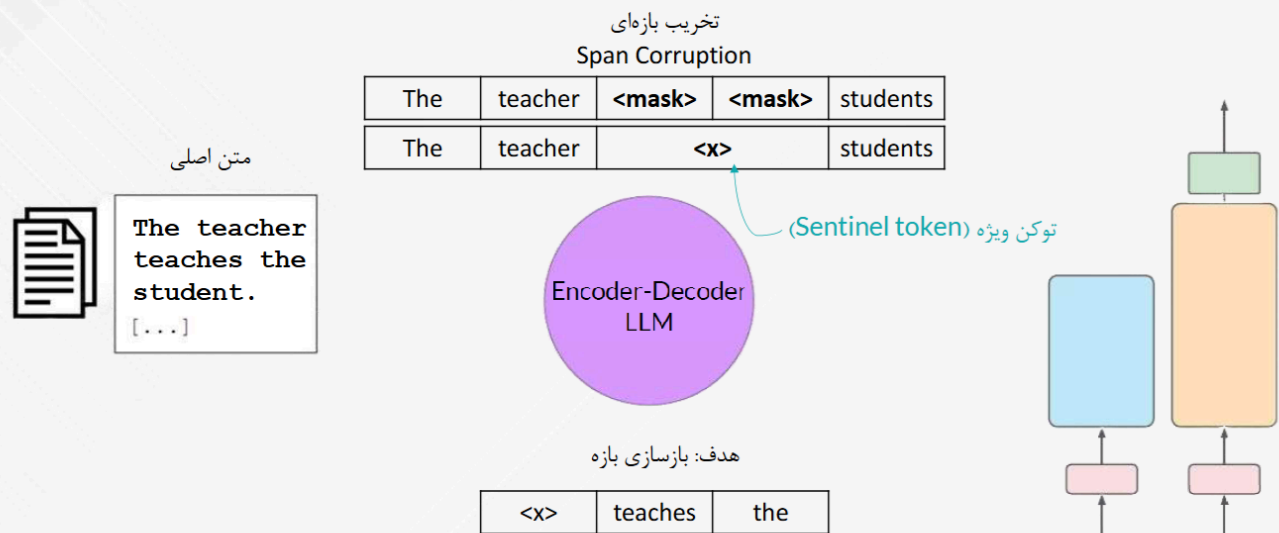
○ Deepseek

○ Llama

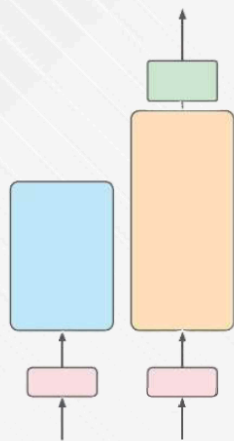
○ Qwen

مدل‌ها رمزگذار - رمز گشا

مدل‌های توالی به توالی (Sequence-to-sequence)



مدل‌های توالی به توالی (Sequence-to-sequence)



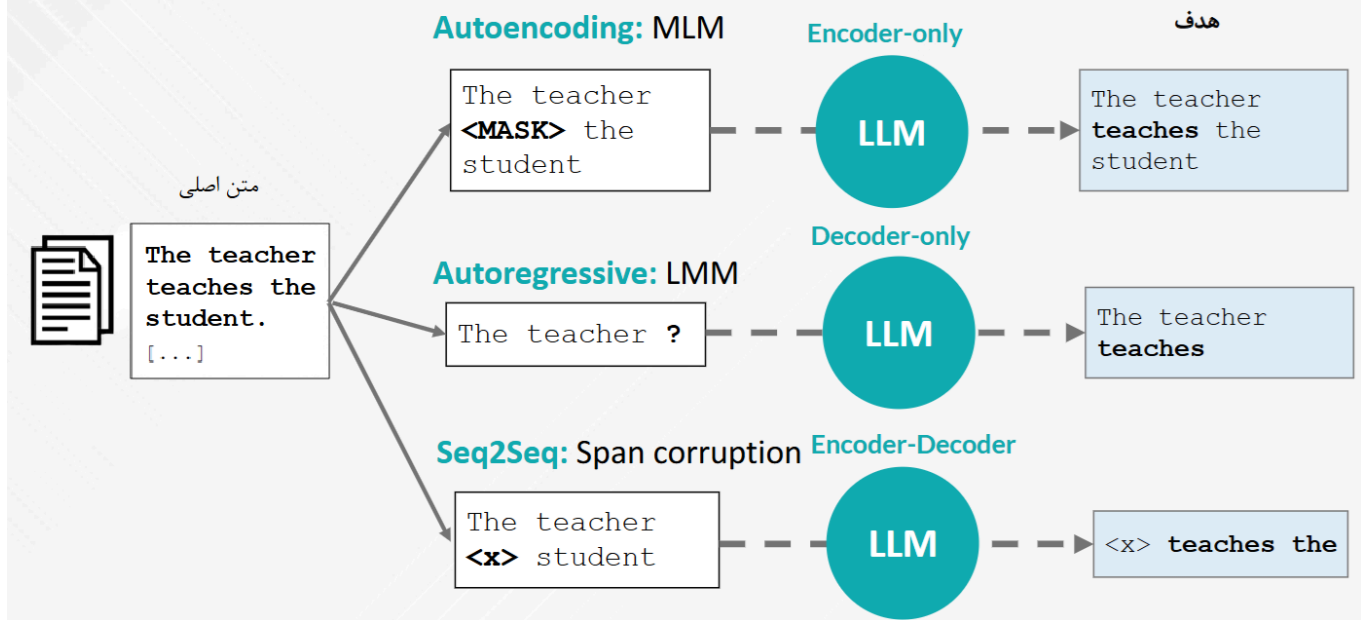
❑ موارد استفاده مناسب:

- ✓ ترجمه
- ✓ خلاصه‌سازی متن
- ✓ پاسخگویی به سوالات

❑ نمونه مدل‌ها:

- T5 (Text-to-Text Transfer Transformer)
- BART (Bidirectional and Auto-Regressive Transformers)

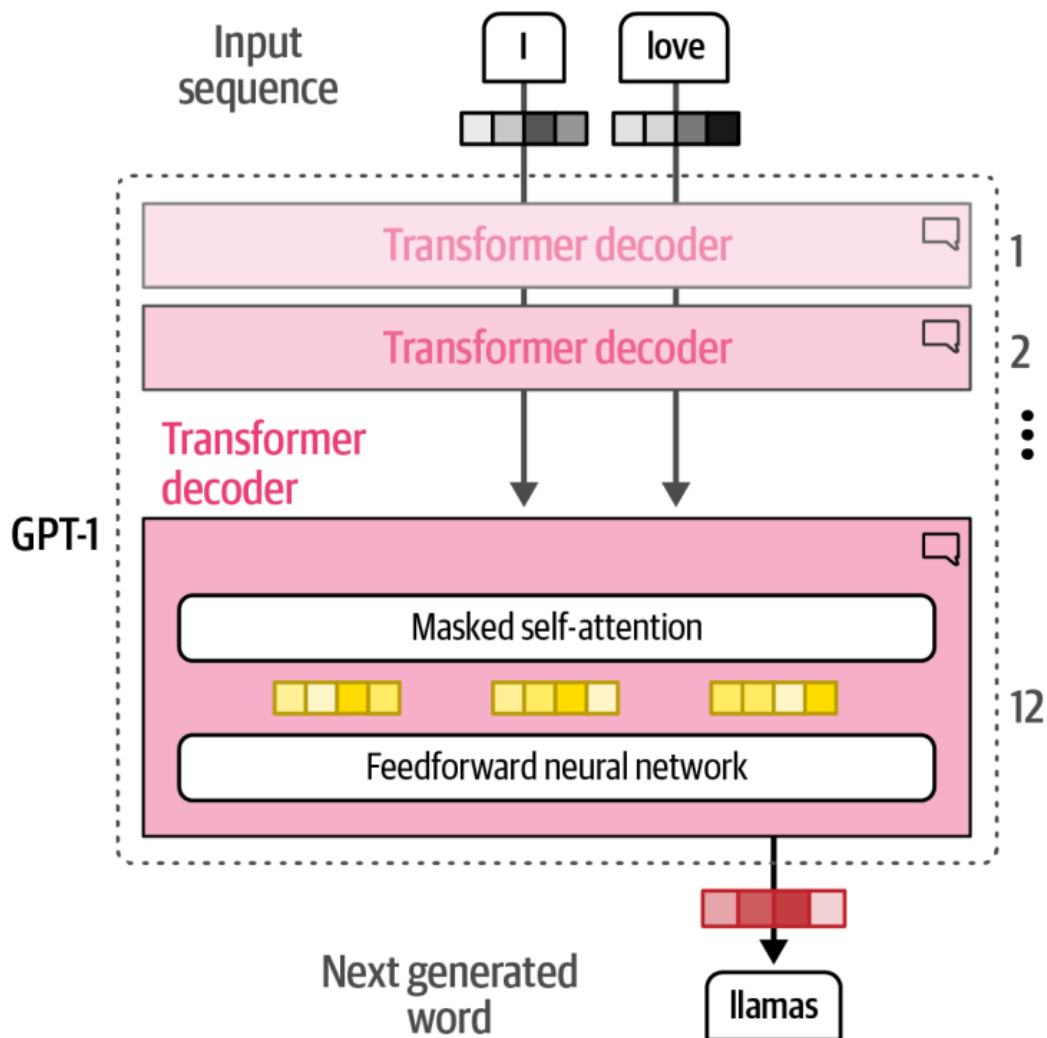
معماری‌ها و اهداف پیش‌آموزش مدل‌ها



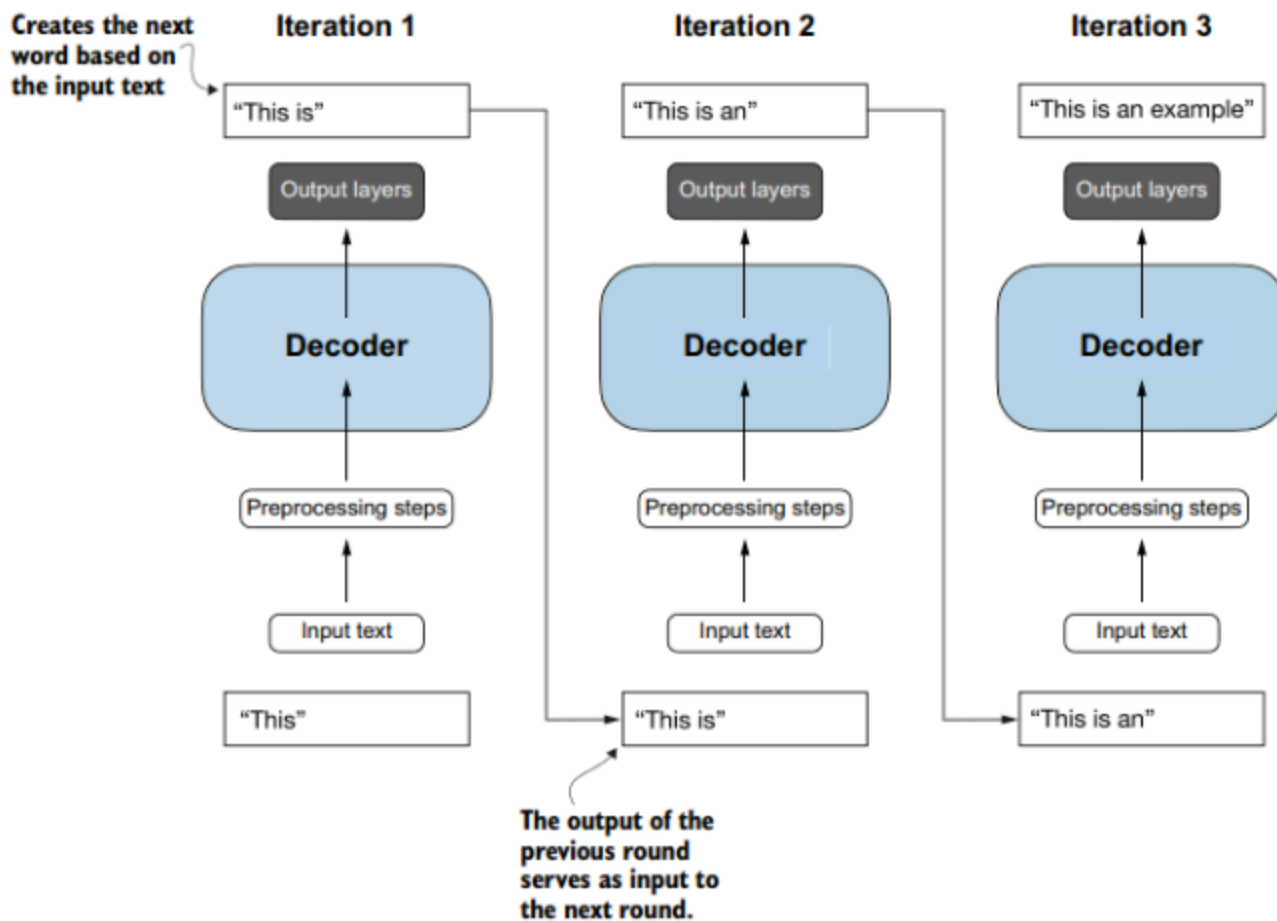
مدل‌های تولیدی: مدل‌های فقط رمزگشا (Decoder-Only)



شبيه به معماری فقط رمزگذار BERT، در سال 2018 یک معماری فقط رمزگشا (decoder-only) برای وظایف تولیدی (generative tasks) پیشنهاد شد. این معماری تحت عنوان ترنسفورماتور پیش‌آموزش‌شده تولیدی (Generative Pre-trained Transformer) (GPT) معرفی شد تا قابلیت‌های تولیدی آن را نشان دهد (و به‌منظور تمایز از نسخه‌های بعدی، اکنون به GPT-1 معروف است). همانطور که در شکل زیر نشان داده شده است، این معماری شامل بلوک‌های رمزگشا است که مشابه معماری رمزگذار-پشته‌ای BERT است.



معماری جی پی تی (GPT) تنها از بخش رمزگشا (Decoder) در ساختار اصلی ترنسفورمر (Transformer) استفاده می کند. این مدل به گونه ای طراحی شده است که پردازش را به صورت تک جهت و از ابتدای متن به انتهای آن انجام دهد، که آن را برای وظایفی مانند تولید متن (Text Generation) و پیش بینی کلمه بعدی (Next-Word Prediction) ایده آل می سازد. این رویکرد به مدل امکان می دهد متن را به صورت تدریجی و کلمه به کلمه تولید کند. (اتورگرسیو)



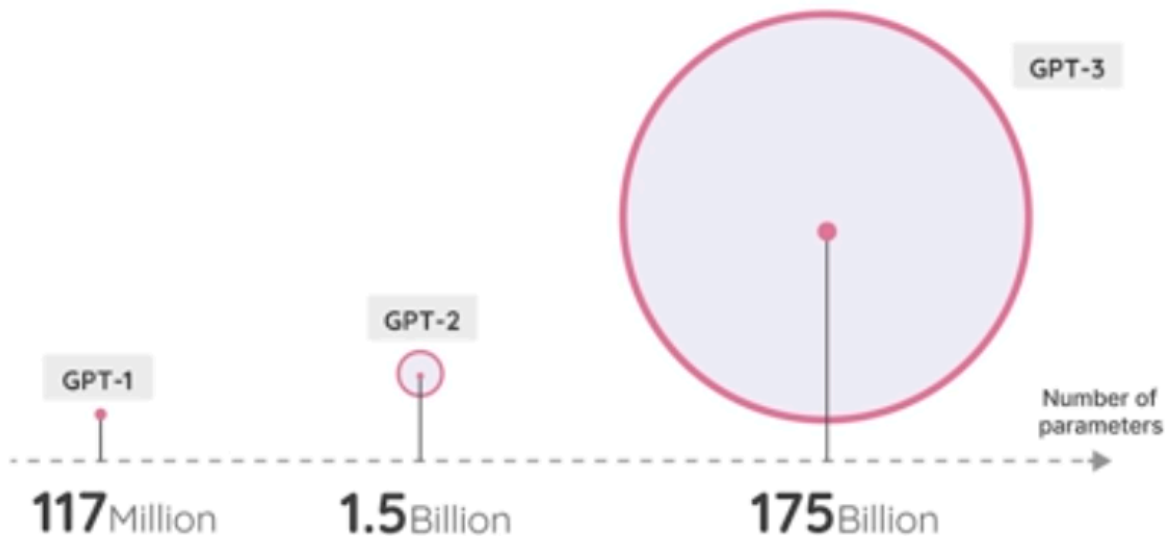
مدل GPT-1 بر روی یک مجموعه داده از 7000 کتاب و Common Crawl (یک مجموعه داده بزرگ از صفحات وب) آموزش داده شد. مدل نهایی دارای 117 میلیون پارامتر بود.

هر پارامتر یک مقدار عددی است که نشان‌دهنده درک مدل از زبان است.

اگر همه چیز ثابت بماند، انتظار می‌رود که پارامترهای بیشتر تأثیر زیادی بر توانایی‌ها و عملکرد مدل‌های زبانی داشته باشد.

با در نظر گرفتن این نکته، شاهد انتشار مدل‌های بزرگ‌تر و بزرگ‌تر در یک روند ثابت بودیم. همانطور که در شکل بعدی نشان داده شده است، GPT-2 دارای 1.5 میلیارد پارامتر و GPT-3 از 175 میلیارد پارامتر استفاده می‌کند که به سرعت منتشر شدند.

Parameters



این مدل‌های تولیدی فقط رمزگشا، به‌ویژه مدل‌های بزرگ‌تر، معمولاً به عنوان مدل‌های زبان بزرگ (Large Language Models) (LLMs) شناخته می‌شوند.

همانطور که در ادامه این فصل خواهیم دید، اصطلاح LLM نه تنها برای مدل‌های تولیدی (فقط رمزگشا) بلکه برای مدل‌های نمایشی (فقط رمزگذار) نیز استفاده می‌شود.

مدل‌های تولیدی LLM به‌عنوان ماشین‌های دنباله به دنباله (Sequence-to-Sequence Machines) عمل کرده و متنی را دریافت کرده و سعی می‌کنند آن را تکمیل کنند (autocomplete).

اگرچه این ویژگی کاربردی است، قدرت واقعی این مدل‌ها از آموزش به‌عنوان یک ربات چت نمایان شد.

حال فرض کنید به جای تکمیل یک متن، چه می‌شود اگر آنها برای پاسخ دادن به سوالات (answer questions) آموزش داده شوند؟ با تنظیم دقیق این مدل‌ها، می‌توان مدل‌های دستورالعمل یا چت (instruct or chat) ایجاد کرد که قادر به پیروی از دستورالعمل‌ها (follow directions) باشند.

همانطور که در شکل زیر نشان داده شده است، مدل حاصل می‌تواند یک پرسش از کاربر (ورودی) را دریافت کرده و پاسخی که احتمالاً بعد از آن می‌آید، تولید کند. از این‌رو، شما اغلب خواهید شنید که مدل‌های تولیدی به‌عنوان مدل‌های تکمیل (Completion Models) شناخته می‌شوند.

User query
(prompt)

Tell me something about llamas

Generative LLM

Task: complete the input

Output
(completion)

Llamas are domesticated South American camelids, widely used as pack animals by Andean cultures since pre-Hispanic times. With their fluffy coat, long neck, and distinctive facial features...

Input
Features

Thou

shalt

Trained Language Model

Task:

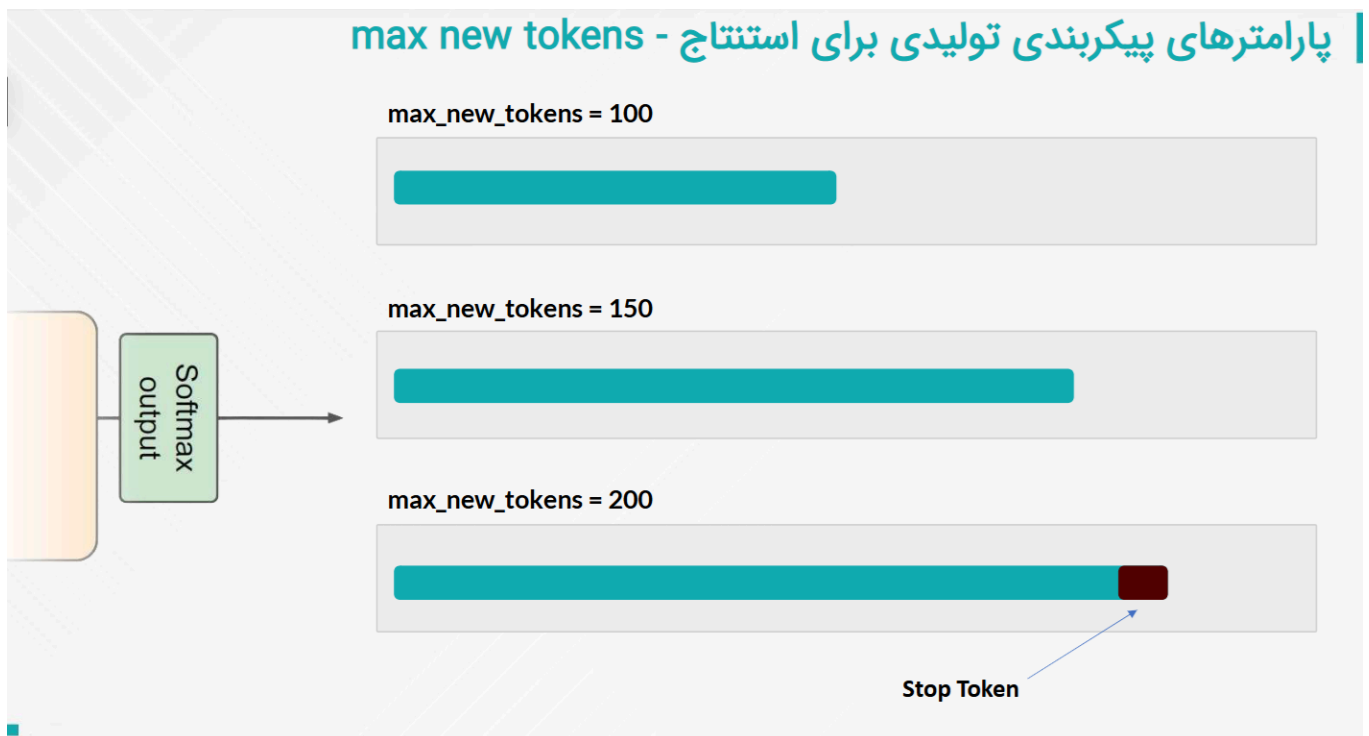
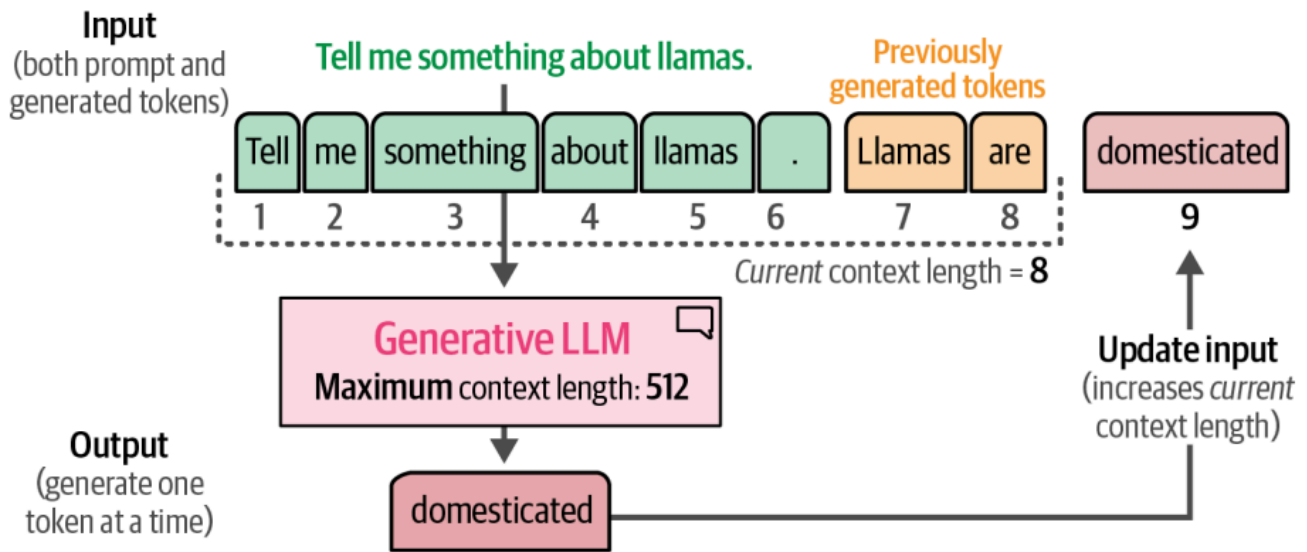
Predict the next word

Output
Prediction

0%	aardvark
0%	aarhus
0.1%	aaron
...	
40%	not
...	
0.01	zyzzyva

یک بخش حیاتی از این مدل‌های تکمیل کننده متن (completion models)، چیزی به نام طول زمینه (Context Length) یا پنجره زمینه (Context Window) است.

طول زمینه نمایانگر حداکثر تعداد توکن‌هایی است که مدل می‌تواند پردازش کند، همانطور که در شکل زیر نشان داده شده است. پنجره زمینه بزرگ‌تر اجازه می‌دهد تا مستندات کامل به LLM ارسال شوند. توجه داشته باشید که به دلیل ماهیت اتورگرسیو (Autoregressive) این مدل‌ها (تولید کلمه به کلمه متن خروجی و افزودن آن به متن ورودی)، طول زمینه فعلی با تولید توکن‌های جدید افزایش خواهد یافت. یعنی طول زمینه مجموع توکن‌های ورودی و خروجی است



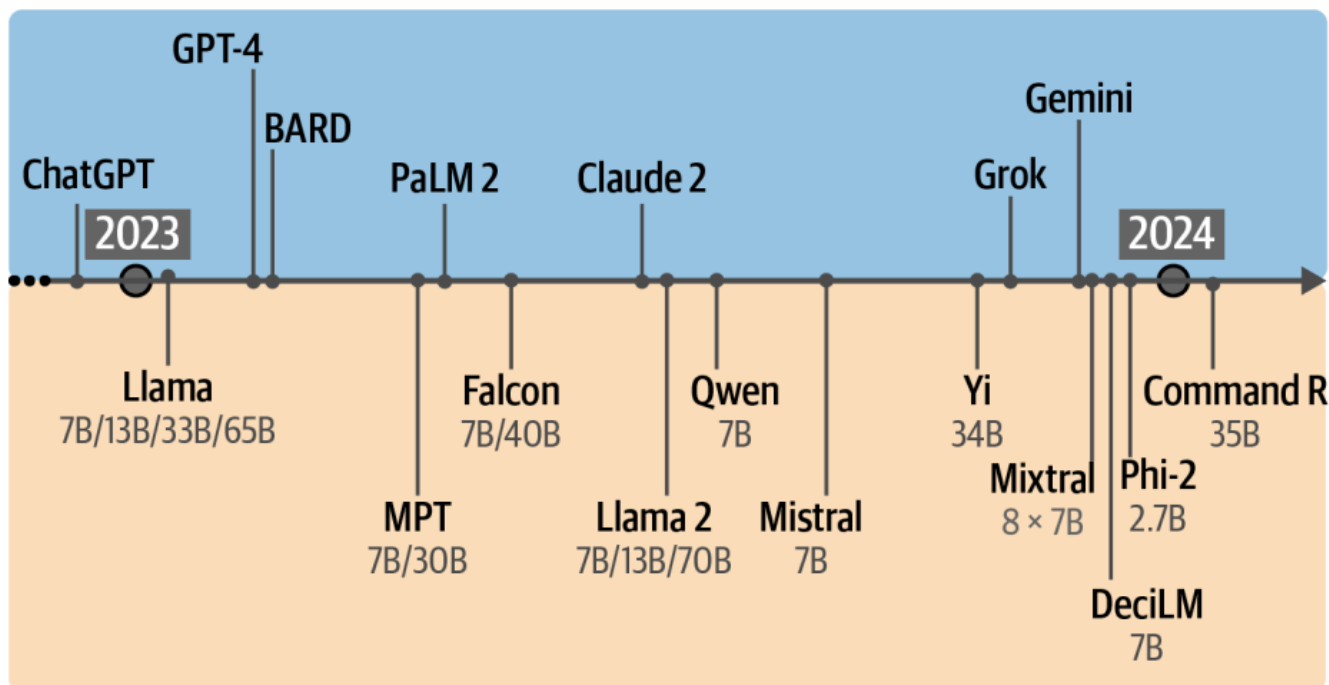
تاریخچه هوش مصنوعی مولد

مدل‌های زبان بزرگ (LLMها) تأثیر عظیمی بر این حوزه گذاشتند و برخی از افراد 2023 را سال هوش مصنوعی مولد نامیدند، با انتشار، پذیرش و پوشش رسانه‌ای چت‌جی‌پی‌تی (ChatGPT) (GPT-3.5). وقتی به چت‌جی‌پی‌تی اشاره می‌کنیم، در واقع منظورمان محصول است نه مدل زیربنایی آن. زمانی که چت‌جی‌پی‌تی برای اولین بار منتشر شد، از مدل LLM GPT-3.5 پشتیبانی می‌کرد و از آن زمان تا کنون نسخه‌های بهینه‌تر آن مانند GPT-4 و GPT-5 به آن اضافه شده است.

مدل GPT-3.5 تنها مدلی نبود که در سال هوش مصنوعی مولد تأثیرگذار بود. همانطور که در شکل زیر نشان داده شده است، مدل‌های LLM منبع‌باز (open source) و اختصاصی با سرعت بی‌نظیری به دست عموم مردم رسیدند. این مدل‌های پایه منبع‌باز اغلب به عنوان مدل‌های بنیادین (Foundation Models) شناخته

می‌شوند و می‌توانند برای وظایف خاصی (specific tasks) مانند دنبال کردن دستورالعمل‌ها تنظیم دقیق (fine-tuned) شوند.

Proprietary models



Open models

علاوه بر معماری محبوب ترنسفورمر (Transformer)، معماری‌های نوآورانه دیگری مانند مامبا (Mamba) و RWKV ظهور کرده‌اند. این معماری‌های جدید سعی دارند به عملکرد سطح ترنسفورمر دست یابند و مزایای اضافی مانند **پنجره‌های زمینه بزرگتر** یا **استنتاج سریع‌تر** را ارائه دهند.

این تحولات نمایانگر تکامل این حوزه هستند و سال 2023 را به سالی واقعاً شلوغ و پرتحول در زمینه هوش مصنوعی تبدیل کرده‌اند. برای ما، فقط پیگیری تحولات متعدد درون و بیرون از هوش مصنوعی زبان کافی بود.

از این رو، این نوشتار بیشتر از آخرین مدل‌های LLM را مورد بررسی قرار می‌دهد. ما بررسی خواهیم کرد که چگونه مدل‌های دیگر، مانند مدل‌های ام‌بدینگ، مدل‌های تنها انکودر و حتی کیسه کلمات (Bag-of-Words) می‌توانند به تقویت LLM‌ها کمک کنند.

در تاریخچه اخیر هوش مصنوعی زبان، مشاهده کردیم که به طور عمده مدل‌های مولد دکودر-تنها (ترنسفورمر) (generative decoder-only (Transformer)) به طور رایج به عنوان مدل‌های زبان بزرگ شناخته می‌شوند. به ویژه اگر این مدل‌ها به عنوان "بزرگ" در نظر گرفته شوند. در عمل، این تعریف به نظر محدودکننده می‌آید!

حال چه می‌شود اگر مدلی با همان قابلیت‌های GPT-3 بسازیم اما ۱۰ برابر کوچک‌تر؟ آیا چنین مدلی از دسته‌بندی "مدل زبان بزرگ" خارج خواهد بود؟

به طور مشابه، چه می‌شود اگر مدلی با اندازه مشابه GPT-4 را منتشر کنیم که قادر به انجام طبقه‌بندی متنی دقیق است اما هیچ قابلیت مولدی ندارد؟ آیا همچنان می‌توان آن را یک مدل "زبان بزرگ" نامید اگر عملکرد اصلی آن تولید زبان نباشد، حتی اگر هنوز متن را نمایش دهد (represents text)؟

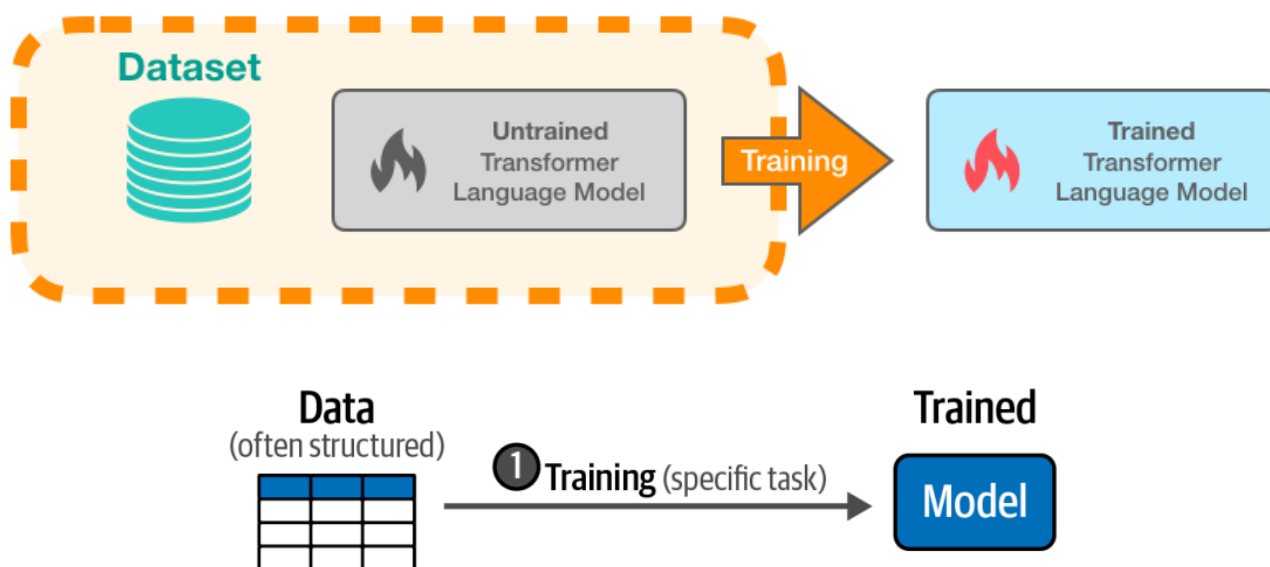
مشکل این گونه تعاریف این است که مدل‌های توانمندی را از دایره خارج می‌کنیم. اینکه چه نامی به یک مدل یا دیگری بدهیم، رفتار آن را تغییر نمی‌دهد.

از آنجا که تعریف اصطلاح "مدل زبان بزرگ" تمایل دارد با انتشار مدل‌های جدید تکامل یابد، ما می‌خواهیم در این نوشتار به وضوح بگوییم که برای ما واژه "بزرگ" یک نماد و یا واژه دلخواه است و آنچه که امروز به عنوان یک مدل بزرگ در نظر گرفته می‌شود، ممکن است فردا کوچک باشد. در حال حاضر، نام‌های زیادی برای همان مفهوم وجود دارند.

از این رو، علاوه بر پوشش مدل‌های مولد، این نوشتار همچنین مدل‌هایی با کمتر از یک میلیارد پارامتر را که متن تولید نمی‌کنند، مورد بررسی قرار خواهد داد. ما بررسی خواهیم کرد که چگونه مدل‌های دیگر، مانند مدل‌های تعبیه متن یا همان ام‌بدینگ (embedding)، مدل‌های نمایشی (representation models)، و حتی مدل کیسه کلمات (n bag-of-words) می‌توانند برای تقویت LLMها به کار گرفته شوند.

الگوی آموزش مدل‌های زبان بزرگ

یادگیری ماشین سنتی به طور معمول شامل آموزش مدل برای یک وظیفه خاص مانند طبقه‌بندی است. همانطور که در شکل زیر نشان داده شده است، این فرایند یک مرحله‌ای است.



یادگیری ماشین سنتی شامل یک مرحله است: آموزش مدل برای یک وظیفه هدف خاص، مانند طبقه‌بندی یا رگرسیون.

در مقابل، ایجاد مدل‌های زبان بزرگ معمولاً شامل حداقل دو مرحله است:

- مرحله اول - مدل‌سازی زبان (Language modeling):

اولین مرحله که به آن پیش‌آموزش (pretraining) گفته می‌شود، بیشترین میزان محاسبات و زمان آموزش را به خود اختصاص می‌دهد. یک LLM بر روی یک مجموعه داده وسیع از متن‌های اینترنتی به شکل خودنظارتی (self supervised) آموزش داده می‌شوند آموزش داده می‌شود تا مدل قادر به یادگیری دستور زبان (grammar)، زمینه (context) و الگوهای زبانی (language patterns) باشد. این مرحله از آموزش هنوز به سمت وظایف یا برنامه‌های خاص (specific tasks) فراتر از پیش‌بینی کلمه بعدی نمی‌رود. (یعنی فقط بلد است کلمه بعدی را پیش‌بینی کند) مدل حاصل معمولاً به عنوان مدل پایه (base model) یا مدل بنیادین

(foundation model) شناخته می‌شود. این مدل‌ها عموماً از دستورالعمل‌ها (instructions) پیروی نمی‌کنند. ولی می‌توان آن‌ها را با تنظیم دقیق و یا استفاده از دستورها (پرومپت) برای وظایف مختلف زبانی به کار گرفت.

The model is simply trained to predict the next word

مدل‌های پیش‌آموزش‌دیده، به‌عنوان «مدل پایه» شناخته می‌شوند و می‌توان آن‌ها را با تغییرات کوچکی برای کارهای مختلف استفاده کرد. به همین دلیل، روش کار در پردازش زبان طبیعی تغییر زیادی کرده است. حالا دیگر لازم نیست برای هر کار زبانی، یک مدل جداگانه و با داده‌ی زیاد بسازیم. کافی است یک مدل پایه را کمی تغییر دهیم تا برای آن کار مناسب شود.

وقتی درباره‌ی پیش‌آموزش مدل‌های زبانی صحبت می‌کنیم، معمولاً با دو نوع مسئله اصلی روبه‌رو هستیم:

۱. مدل‌سازی یا رمزگذاری دنباله‌های ورودی (sequence modeling / encoding)

۲. تولید دنباله (sequence generation)

با اینکه این دو مسئله شکل‌های متفاوتی دارند، ولی برای ساده‌تر شدن توضیح، هر دو را با یک مدل کلی نشان می‌دهیم:

$$o = g(x_0, x_1, \dots, x_m; \theta)$$

در این فرمول:

- دنباله‌ای از توکن‌ها یا کلمات ورودی هستند $\{x_0, x_1, \dots, x_m\}$.
- است که در ابتدای جمله یا متن قرار می‌گیرد (CLS) یا $\langle s \rangle$ (مثل) یک توکن ویژه x_0 .
- نشان داده می‌شود θ یک شبکه عصبی است که پارامترهای آن با g .
- خروجی مدل است که بسته به نوع مسئله می‌تواند متفاوت باشد o .

مثلاً:

- اگر هدف پیش‌بینی توکن باشد (مثل مدل‌های زبانی)، o یک توزیع احتمالاتی روی کلمات واژگان خواهد بود.
- اگر هدف، رمزگذاری جمله باشد، o یک نمایش عددی از کل جمله خواهد بود (مثلاً یک یا چند بردار عددی).

در پیش‌آموزش مدل‌های زبانی، دو موضوع اصلی مطرح است:

۱. آموزش مدل (یادگیری پارامترهای θ) با استفاده از یک وظیفه‌ی پیش‌آموزشی:

برخلاف روش‌های سنتی که مدل را برای یک کار خاص (مثلاً ترجمه یا دسته‌بندی) آموزش می‌دهند، در پیش‌آموزش ما کاری کلی انجام می‌دهیم. یعنی هدف این است که مدلی بسازیم که در آینده بتواند برای کارهای مختلف استفاده شود، نه فقط یک کار مشخص.

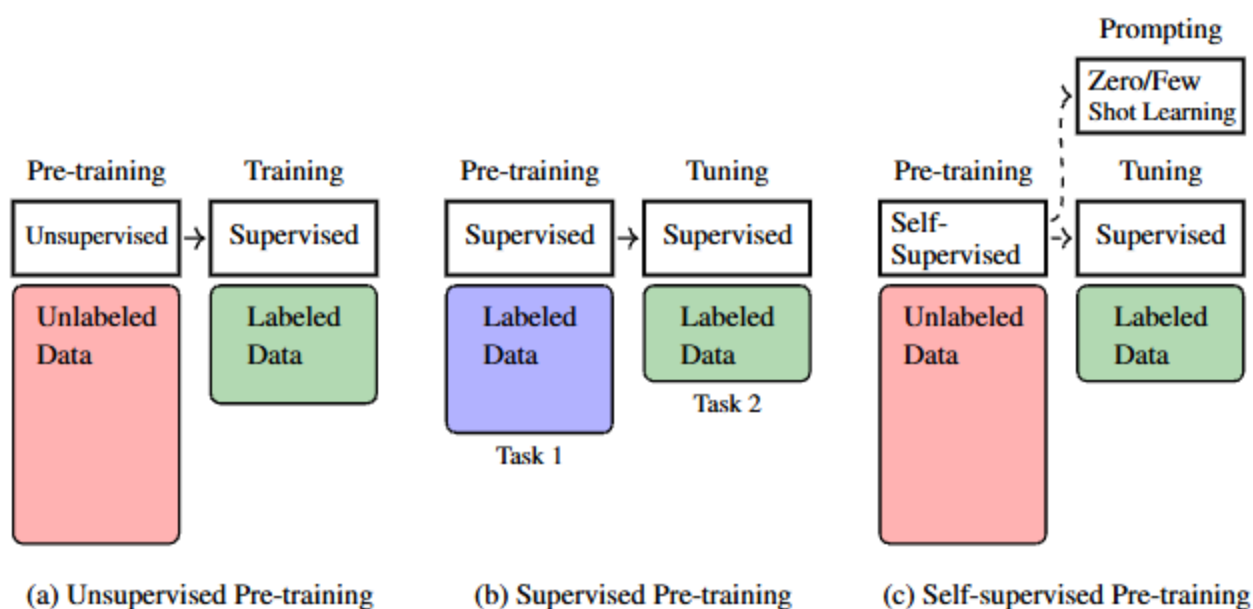
۲. استفاده از مدل آموزش دیده برای حل وظایف مشخص:

بعد از اینکه مدل با پیش آموزش آماده شد، باید بتوان آن را برای وظایف واقعی مثل پاسخ گویی به سؤال، خلاصه سازی، یا تحلیل احساسات تطبیق داد. این کار معمولاً به یکی از دو روش انجام می شود: یا با مقدار کمی داده برچسب دار، پارامترهای مدل را کمی تنظیم می کنیم (fine-tuning)، یا فقط با دادن توضیح و دستور، مدل را راهنمایی می کنیم (prompting).

انواع روش های پیش آموزش: بدون نظارت، با نظارت و خودنظارتی

پیش آموزش در یادگیری عمیق، به فرایند آموزش اولیه ی یک مدل پیش از به کارگیری در یک وظیفه ی خاص گفته می شود. هدف از این مرحله، یادگیری ویژگی ها و ساختارهای عمومی داده هاست، به طوری که بتوان مدل را برای وظایف گوناگون با تلاش کمتر و داده های محدودتر تطبیق داد.

سه رویکرد اصلی برای پیش آموزش وجود دارد:



۱. پیش آموزش بدون نظارت (Unsupervised Pre-training)

در این روش، مدل با داده های خام و بدون برچسب آموزش می بیند. برای مثال، در یک شبکه ی عصبی، می توان از مدل خواست ورودی خود را بازسازی کند (مانند خودرمزگذارها). این روش، در گذشته به عنوان مرحله ای

مقدماتی برای تقویت آموزش با نظارت استفاده می‌شد و به یافتن تنظیمات بهینه‌تر و پایدارتر کمک می‌کرد.



Unsupervised Pre-training

Untrained
GPT-3

GPT-3

Expensive training on massive datasets

Dataset: 300 billion tokens of text

Objective: Predict the next word

Example:

a robot must ?

Text: Second Law of Robotics: A robot must obey the orders given it by human beings



Generated training examples

Example #

Input (features)

Correct output
(labels)

1 Second law of robotics :

a

2 Second law of robotics : a

robot

3 Second law of robotics : a robot

must

۲. پیش‌آموزش با نظارت (Supervised Pre-training)

در این رویکرد، مدل با داده‌های برچسب‌دار برای یک وظیفه‌ی مشخص آموزش داده می‌شود؛ برای نمونه، آموزش یک مدل متنی برای تشخیص احساس جمله (مثبت یا منفی). سپس از همین مدل در وظیفه‌ی مشابهی استفاده می‌شود، مانند تشخیص جملات عینی یا ذهنی. این روش از نظر فنی ساده است، اما به داده‌ی برچسب‌دار فراوان نیاز دارد که در مقیاس بزرگ، ممکن است در دسترس نباشد.

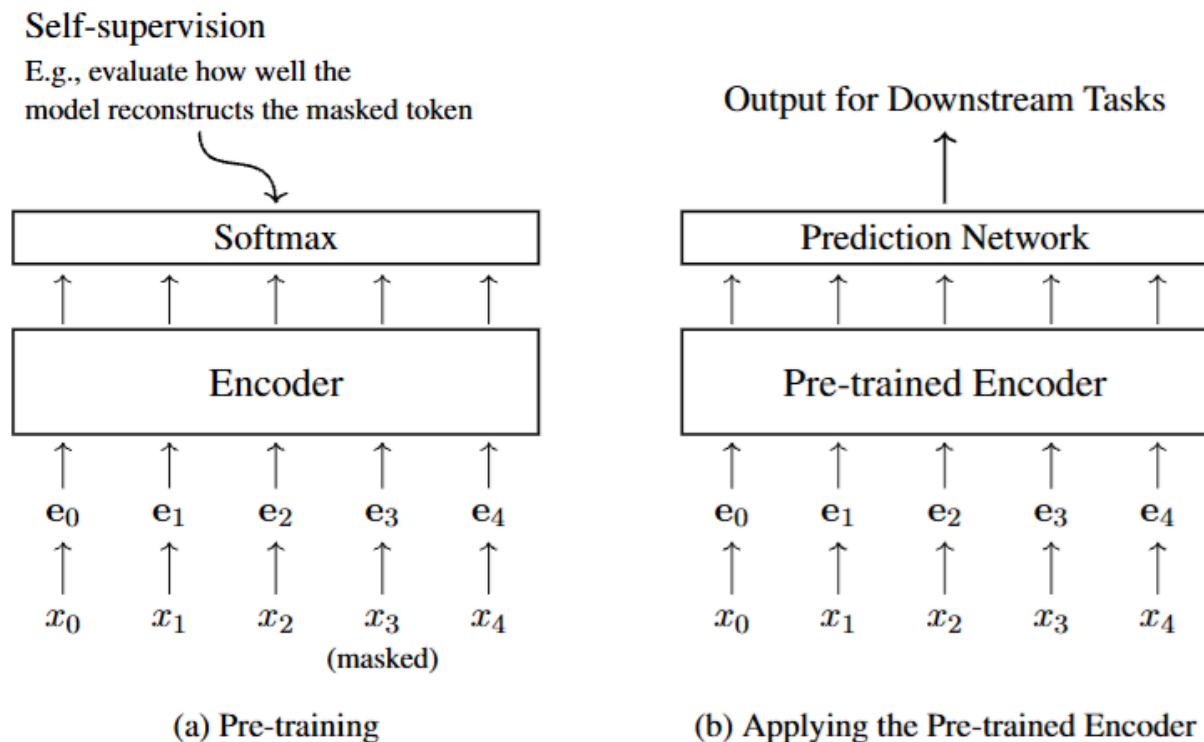
۳. پیش‌آموزش خودنظارتی (Self-supervised Pre-training) مدلی که بدون برچسب انسانی از طریق اطلاعات درون‌متنی آموزش می‌بیند

در یادگیری خودنظارتی، مدل از داده‌های بدون برچسب استفاده می‌کند، اما وظایف آموزشی را به صورت خودکار از دل داده‌ها استخراج می‌کند. برای مثال، در یک جمله مانند «دیروز به ___ رفتم»،

کلمه‌ی حذف‌شده را مدل باید حدس بزند. این روش نیاز به برچسب‌گذاری انسانی ندارد و امکان آموزش با حجم عظیمی از داده‌های متنی را فراهم می‌سازد.

پیش‌آموزش خودنظارتی، به‌ویژه در مدل‌های زبانی مانند BERT و GPT، بسیار موفق عمل کرده و امروزه، مبنای اکثر مدل‌های پیشرفته‌ی NLP به‌شمار می‌رود و به طور کلی به صورت زیر تقسیم بندی می‌گردد:

- الف - مدل‌های فقط رمزگشا (Decoder-only Pre-training) مانند GPT
در این معماری، مدل بر اساس کلمات قبلی، کلمه بعدی را پیش‌بینی می‌کند. فرآیند آموزش شامل بهینه‌سازی پارامترهای مدل برای کاهش اختلاف بین پیش‌بینی‌های مدل و کلمات واقعی است. برای این منظور، معمولاً از تابع خطای «کراس انتروپی» استفاده می‌شود که میزان خطا را به صورت عددی اندازه‌گیری می‌کند. با کمینه‌سازی این خطا روی مجموعه‌ای بزرگ از داده‌های متنی، مدل به تدریج توانایی درک زبان و تولید متن‌های معنادار را کسب می‌کند.
- ب - مدل‌های فقط رمزگذار (Encoder-only Pre-training) مانند BERT
رمزگذار (Encoder) یک مدل است که کل جمله را می‌گیرد و آن را به یک مجموعه بردار تبدیل می‌کند که میتوان بعداً از آن در تسک های مختلف مثل طبقه بندی و .. استفاده کرد.



از طرف دیگر از آنجایی که نمی‌توانیم مستقیم خروجی این بردارها را بسنجیم، برای آموزش مدل، بخشی از کلمات جمله را مخفی (ماسک) می‌کنیم و از مدل می‌خواهیم آن‌ها را حدس بزند. در این روش، مدل همه کلمات جمله را همزمان می‌بیند و تلاش می‌کند کلمات حذف‌شده را بازسازی کند. پس از آموزش، مدل را با

داده‌های برچسب‌دار تنظیم می‌کنیم تا در کارهای خاص مثل طبقه‌بندی متن کاربردی شود. مدل‌سازی زبان ماسک‌شده (Masked Language Modeling) یکی از روش‌های کلیدی پیش‌آموزش مدل‌های رمزگذار است که در مدل‌های معروفی مانند BERT استفاده می‌شود. در این روش، برخی کلمات متن به صورت تصادفی با علامت ویژه [MASK] جایگزین می‌شوند و مدل با دیدن بقیه کلمات باید کلمات حذف‌شده را حدس بزند.

More formally, for an input sequence $\mathbf{x} = x_0 \dots x_m$, suppose that we mask the tokens at positions $\mathcal{A}(\mathbf{x}) = \{i_1, \dots, i_u\}$. Hence we obtain a masked token sequence $\bar{\mathbf{x}}$ where the token at each position in $\mathcal{A}(\mathbf{x})$ is replaced with a special symbol [MASK]. For example, for the following sequence

The early bird catches the worm

we may have a masked token sequence like this

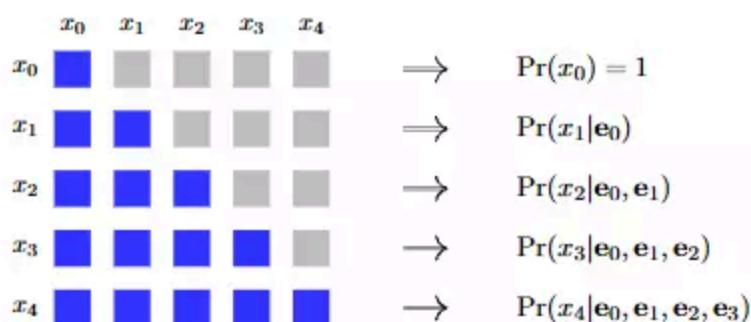
The [MASK] bird catches the [MASK]

where we mask the tokens *early* and *worm* (i.e., $i_1 = 2$ and $i_2 = 6$).

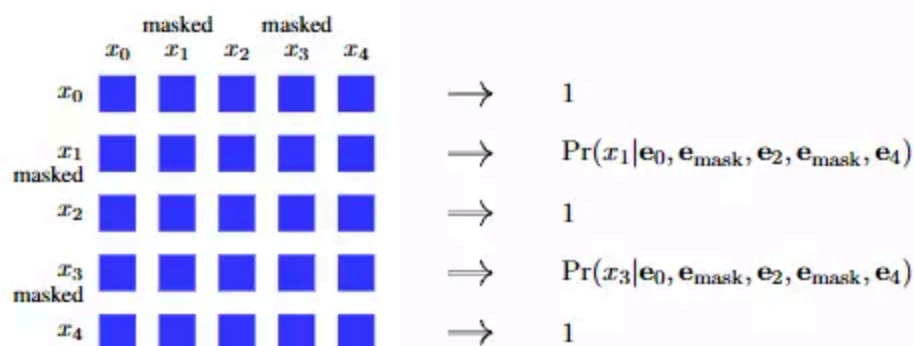
در مدل ماسک‌شده، توکن‌های خاصی به نام [MASK] در متن جایگزین می‌شوند و مدل باید آن‌ها را حدس بزند.

این باعث می‌شود مدل هنگام آموزش چیزی می‌بیند که در زمان استفاده واقعی وجود ندارد و همچنین پیش‌بینی هر توکن ماسک‌شده به صورت جداگانه و بدون توجه به دیگر توکن‌های ماسک‌شده انجام

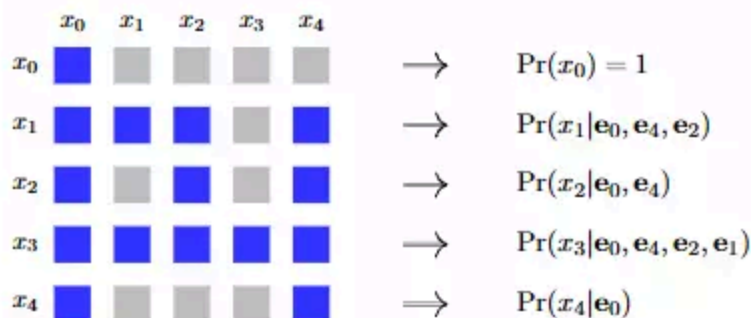
می‌شود.



(a) Causal Language Modeling (order: $x_0 \rightarrow x_1 \rightarrow x_2 \rightarrow x_3 \rightarrow x_4$)



(b) Masked Language Modeling (order: $x_0, [\text{MASK}], x_2, [\text{MASK}], x_4 \rightarrow x_1, x_3$)



(c) Permuted Language Modeling (order: $x_0 \rightarrow x_4 \rightarrow x_2 \rightarrow x_1 \rightarrow x_3$)

در مدل‌های زبانی مختلف، نحوه‌ی «ماسک‌گذاری در مکانیزم خودتوجهی (self-attention)» نقش مهمی در شکل‌گیری درک مدل از توالی دارد.

پایه‌سازی این روش در ترنسفورمرها با تنظیم ماسک‌های مناسب برای مکانیزم self-attention انجام می‌شود تا توجه‌ها فقط به توکن‌های مجاز محدود شود و ترتیب جابه‌جاشده حفظ شود. این رویکرد به مدل کمک می‌کند تا وابستگی‌های پیچیده‌تر بین توکن‌ها را یاد بگیرد و عملکرد بهتری داشته باشد.

در اینجا، سه رویکرد رایج با یکدیگر مقایسه می‌شوند:

1. مدل زبانی علی (Causal Language Modeling):

در این روش که بیشتر در مدل‌های تولیدی مانند GPT استفاده می‌شود، هر توکن فقط به توکن‌های

قبل از خودش (چپ به راست) توجه می‌کند. این کار باعث می‌شود مدل فقط از اطلاعات قبلی برای پیش‌بینی استفاده کند، مشابه شرایط واقعی تولید متن.

2. مدل زبانی ماسک گذاری شده یا نقاب‌دار (Masked Language Modeling): در این روش که در مدل‌هایی مانند BERT به کار می‌رود، برخی از توکن‌ها در ورودی با MASK جایگزین می‌شوند، و مدل باید آن‌ها را با توجه به سایر توکن‌های موجود در جمله (قبل و بعد از توکن ماسک شده) پیش‌بینی کند. این منجر به شکل‌گیری مدلی دوسویه می‌شود که از کل متن استفاده می‌کند.

3. مدل زبانی ترتیب‌درهم (Permuted Language Modeling): این رویکرد که در مدل‌هایی مانند XLNet استفاده شده، به جای ترتیب طبیعی واژه‌ها، ترتیب جدیدی برای پیش‌بینی انتخاب می‌شود و مدل طبق آن ترتیب، توکن‌ها را پیش‌بینی می‌کند. با این روش، امکان استفاده از زمینه چپ و راست به شکل منعطف‌تر فراهم می‌شود، بدون نیاز به استفاده از MASK.

در تصویر مقایسه‌ای فوق این سه رویکرد، خانه‌های آبی نشان‌دهنده امکان توجه یک توکن به توکن دیگر هستند و خانه‌های خاکستری بیانگر محدودیت در توجه هستند. این تفاوت در ماسک‌گذاری باعث شکل‌گیری رفتارهای متفاوت یادگیری در هر مدل می‌شود.

در پیش‌پردازش خودنظارتی، یکی از روش‌های رایج برای آموزش رمزگذار (Encoder)، تعریف مسائل طبقه‌بندی ساختگی بر اساس متن‌های بدون برچسب است. دو نمونه معروف از این نوع آموزش عبارت‌اند از:

- **پیش‌بینی جمله بعدی (NSP (Next Sentence Prediction))**: این روش در مدل BERT معرفی شده است. ایده اصلی آن است که مدل باید تشخیص دهد آیا جمله دوم واقعاً ادامه‌ای برای جمله اول هست یا نه. برای این کار، دو جمله پشت‌سرهم (یا دو جمله تصادفی) به مدل داده می‌شوند و خروجی رمزگذار برای توکن آغازین [CLS] به لایه Softmax داده می‌شود تا مدل یک برچسب دودویی (در ادامه هست / نیست) پیش‌بینی کند. این وظیفه به صورت مکمل در کنار مدل‌سازی زبان نقاب‌دار (Masked Language Modeling (MLM)) برای آموزش به کار می‌رود.
- **تشخیص واژه جایگزین (ELECTRA)**: در این روش، ابتدا یک مدل کوچک‌تر (Generator) مدلی که توکن‌های ماسک‌شده را تولید یا جایگزین می‌کند با (Masked Language Modeling (MLM)) آموزش می‌بیند تا برخی واژه‌های ماسک‌شده را حدس بزند. سپس یک رمزگذار دیگر (Discriminator) مدلی که وظیفه‌اش تشخیص تغییر یا عدم تغییر در توکن‌هاست) آموزش می‌بیند تا تشخیص دهد کدام واژه‌ها در جمله تولیدی با واژه اصلی متفاوت هستند. برای هر توکن، مدل باید پیش‌بینی کند که "اصلی" است یا "جایگزین شده". پس از آموزش، فقط رمزگذار Discriminator به عنوان مدل پیش‌آمخته برای کارهای بعدی استفاده می‌شود. (به جای پیش‌بینی کلمات ماسک‌شده، مدل باید تشخیص دهد کدام کلمات در متن تغییر کرده‌اند). هر دو روش بر پایه یادگیری طبقه‌بندی عمل می‌کنند و کمک می‌کنند رمزگذار زبان را بهتر درک کند، بدون نیاز به داده‌های برچسب‌خورده.
- ج - مدل‌های رمزگذار-رمزگشا (Encoder-Decoder) مانند T5 و Transformer

در معماری **Encoder-Decoder** که در مسائل دنباله‌به‌دنباله (sequence-to-sequence) مثل ترجمه ماشینی یا پاسخ به پرسش استفاده می‌شود، هم ورودی و هم خروجی مدل، متن هستند. ایده‌ی اصلی در پیش‌پردازش این معماری، آموزش مدلی است که بتواند هر مسئله‌ی NLP را به شکل تبدیل یک متن به متن دیگر انجام دهد.

به عنوان مثال:

ورودی: «This movie was amazing»!

خروجی: «positive»

در این حالت، مدل یک مسئله‌ی **تحلیل احساسات** را به صورت تبدیل متن به متن یاد می‌گیرد.

در این روش ابتدا مدل با داده‌های بزرگ و بدون برچسب، به صورت **خودنظارتی (self-supervised)** آموزش می‌بیند. سپس با داده‌های خاص‌تر و دارای برچسب، برای یک **وظیفه‌ی خاص تنظیم دقیق (fine-tuning)** می‌شود.

این رویکرد پایه‌گذار مدل‌هایی مانند **T5 (Text-to-Text Transfer Transformer)** و **BART** است که همه‌ی وظایف را به یک فرمول‌بندی «متن به متن» تبدیل می‌کنند.

در مدل‌های Encoder-Decoder مثل T5، تمام مسائل به شکل «متن به متن» تبدیل می‌شوند. یعنی مدل با دریافت یک متن ورودی (مثلاً جمله یا سوال) باید یک متن خروجی (مثلاً ترجمه، پاسخ یا خلاصه) تولید کند.

برای آموزش، دو روش اصلی وجود دارد:

1. **پیشوند زبانی (Prefix Language Modeling)**: مدل بخش اول جمله را می‌گیرد (پیشوند) و باید ادامه‌ی آن را حدس بزند. مثلاً:

• ورودی: "The puppies are frolicking"

• خروجی: "outside the house".

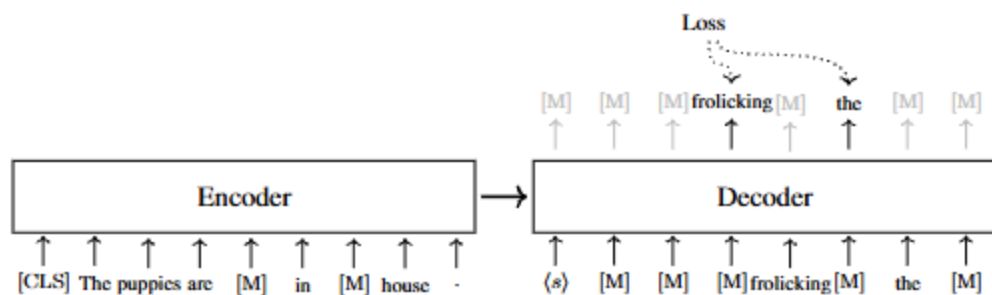
2. **بازسازی نویزی (Denoising Autoencoding)**: بعضی کلمات داخل جمله با [MASK] جایگزین می‌شوند و مدل باید جمله اصلی را بازسازی کند. مثلاً:

• ورودی: "The puppies are [MASK] outside [MASK] house"

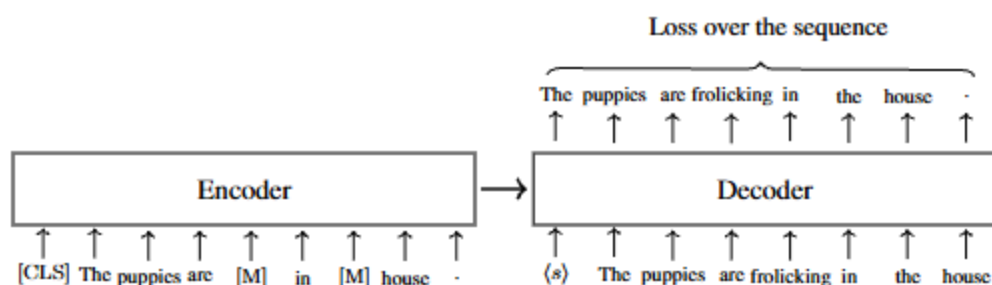
• خروجی: "The puppies are frolicking outside the house"

این آموزش باعث می‌شود مدل هم زبان را خوب بفهمد (encoder) و هم متن تولید کند (decoder). همچنین با داده‌های چندزبانه، مدل توانایی ترجمه و درک زبان‌های مختلف را پیدا می‌کند. در نهایت، مدل آموزش‌دیده را می‌توان برای کارهای مختلف زبان طبیعی با تنظیم‌های جزئی (fine-tuning) به کار برد.

روش‌های درک جملات خراب و بازسازی جمله اصلی در روش رمزگذار و رمز گشا



(a) Training an encoder-decoder model with BERT-style masked language modeling



(b) Training an encoder-decoder model with denoising autoencoding

- **روش BERT-style:** ورودی به encoder جملاتی است که بعضی کلماتشان با [MASK] جایگزین شده. سپس decoder فقط باید همان کلمات ماسک شده را پیش بینی کند و فقط خطاهای این کلمات محاسبه می شود.
- **روش denoising autoencoding:** باز هم ورودی به encoder جملات ماسک شده است، ولی این بار decoder باید کل جمله اصلی را به ترتیب پیش بینی کند و خطا برای همه کلمات محاسبه می شود، مثل آموزش مدل های زبان معمولی.

آموزش به روش Denoising

آموزش به روش Denoising به معنای آموزش مدل های Encoder-Decoder به گونه ای است که بتوانند جملات اصلی و صحیح را از ورودی هایی که دچار اختلال یا تغییر شده اند، بازسازی کنند.

برای ایجاد اختلال در ورودی، روش های متعددی وجود دارد که مهم ترین آنها عبارتند از:

- **ماسک گذاری کلمات (Token Masking):** برخی از کلمات به صورت تصادفی با نماد [MASK] جایگزین می شوند.
- **حذف کلمات (Token Deletion):** کلمات منتخب به طور کامل از متن حذف می گردند، بدون اینکه جایگزین شوند.
- **ماسک گذاری بخش های متوالی (Span Masking):** بخش هایی از متن که شامل چند کلمه پشت سر هم است، به صورت یکجا با نماد [MASK] جایگزین می شوند.
- **تغییر ترتیب جملات (Sentence Reordering):** جملات داخل متن به صورت تصادفی جابه جا می شوند تا مدل بتواند توانایی تشخیص ترتیب درست جملات را بیاموزد.

- **چرخش متن (Document Rotation):** بخشی از متن به صورت تصادفی انتخاب شده و متن به گونه‌ای چرخش داده می‌شود که آن بخش به ابتدای متن منتقل گردد.

در فرآیند آموزش، معمولاً یکی از این روش‌ها به صورت تصادفی برای هر نمونه به کار گرفته می‌شود تا مدل بتواند مهارت بالاتری در درک و بازسازی متن کسب کند. انتخاب روش مناسب ایجاد اختلال در ورودی، تأثیر بسزایی بر کیفیت مدل نهایی دارد و نیازمند آزمایش و تنظیم دقیق است.

سازگارسازی مدل‌های پیش‌آموزش‌دیده

همان‌طور که قبلاً نیز اشاره شد، در پیش‌آموزش مدل‌های زبانی، دو نوع اصلی از مدل‌ها به کار می‌روند:

- **مدل‌های رمزگذار دنباله (Sequence Encoding Models)**

این مدل‌ها، با دریافت یک دنباله از کلمات یا توکن‌ها، آن را به یک بردار عددی (یا مجموعه‌ای از بردارها) تبدیل می‌کنند. این بردار، نماینده‌ی معنایی دنباله‌ی ورودی است و معمولاً به عنوان ورودی به مدل دیگری داده می‌شود؛ برای مثال، در سیستم‌های دسته‌بندی جملات یا تحلیل احساس.

- **مدل‌های تولید دنباله (Sequence Generation Models)**

در این نوع مدل‌ها، هدف تولید یک دنباله‌ی جدید از توکن‌ها بر اساس یک زمینه‌ی مشخص است. مفهوم «زمینه» بسته به کاربرد متفاوت است؛ در مدل‌سازی زبان، منظور توکن‌های قبلی است و در ترجمه‌ی ماشینی، زمینه به جمله‌ی زبان مبدا اشاره دارد. این مدل‌ها برای کارهایی مانند تولید متن، پاسخ به سوال، یا ترجمه مورد استفاده قرار می‌گیرند.

پس از مرحله‌ی پیش‌آموزش، برای استفاده از این مدل‌ها در وظایف مختلف پردازش زبان (وظایف پایین‌دستی)، باید آن‌ها را با روش‌هایی خاص سازگار کرد. در ادامه، دو شیوه‌ی رایج برای این سازگارسازی را بررسی خواهیم کرد.

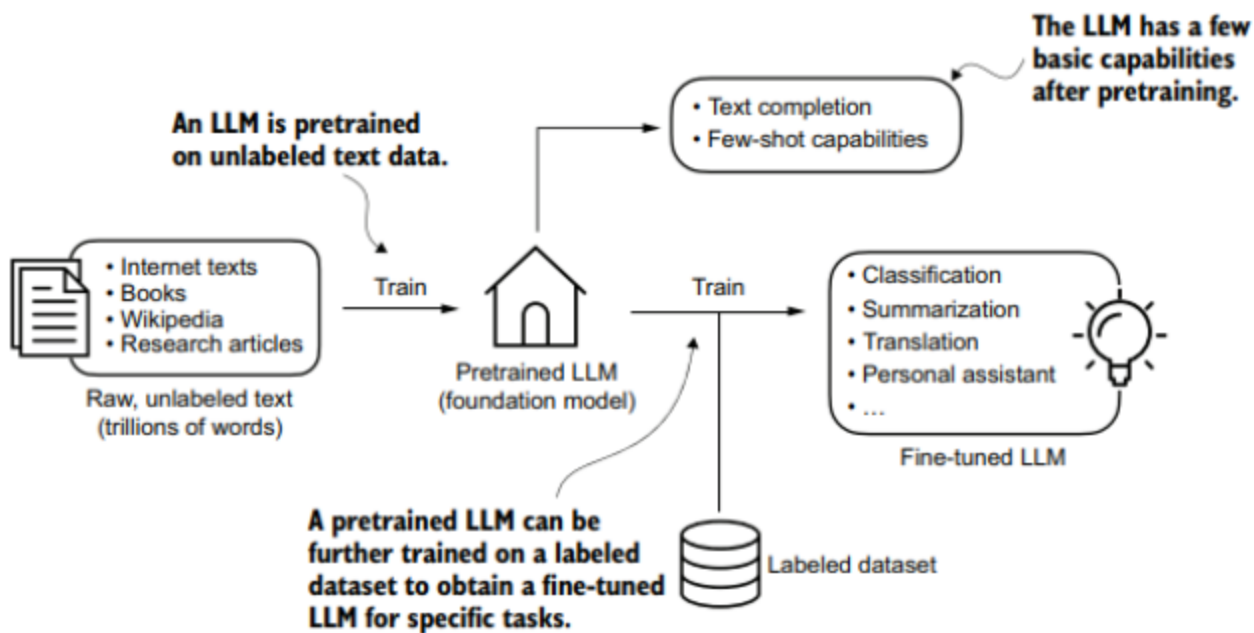
تا اینجا، چند روش مختلف برای پیش‌آموزش مدل‌های زبان را بررسی کردیم. چون یک هدف آموزشی (مثل ماسک‌گذاری توکن‌ها) می‌تواند روی مدل‌های مختلفی (مثل فقط انکودر یا انکودر-دکودر) استفاده شود، بهتر است وظایف پیش‌آموزشی را براساس هدف و نحوه آموزش‌شان دسته‌بندی کرد (نه فقط معماری مدل).

- **مدل‌سازی زبان (Language Modeling):** پیش‌بینی کلمه بعدی بر اساس کلمات قبلی.
- **مدل‌سازی زبان با ماسک (Masked Language Modeling):** پنهان کردن کلماتی در متن و حدس زدن آنها توسط مدل.
- **مدل‌سازی جابه‌جاشده (Permuted Language Modeling):** پیش‌بینی کلمات به ترتیب متفاوت از متن اصلی.
- **آموزش تفکیکی (Discriminative Training):** آموزش مدل برای بهبود عملکرد در کارهای طبقه‌بندی.

- **رمزگذار حذف نویز خودکار (Denoising Autoencoding):** خراب کردن ورودی و آموزش مدل برای بازسازی متن اصلی.

این روش‌ها هر کدام ویژگی‌ها و کاربردهای خاص خودشان را دارند و می‌توانند در مدل‌های مختلف و مسائل متفاوت به کار روند. پژوهش‌های زیادی انجام شده که کارایی و مزایای هر روش را در شرایط مختلف بررسی کرده‌اند.

به طور خلاصه هدف از فرآیندپیش‌آموزش (Pretraining) یک مدل زبانی بزرگ (LLM) بر روی مجموعه داده‌های متنی عظیم، پیش‌بینی کلمه بعدی است. پس از پیش‌آموزش، این مدل را می‌توان با استفاده از یک مجموعه داده برچسب‌گذاری‌شده کوچک‌تر تنظیم دقیق (Fine-tuning) کرد و یا با استفاده از پرامپت (مهندسی درخواست) از آن استفاده کرد.



- **مرحله دوم - بخش اول: تنظیم دقیق (Fine-tuning):**

مرحله دوم که به آن تنظیم دقیق (fine-tuning) یا گاهی اوقات آموزش پسین (post-training) گفته می‌شود، یکی از روش‌های اصلی برای استفاده از مدل‌های پیش‌آموزش‌دیده و آموزش بیشتر آن برای یک وظیفه محدودتر (narrower task) است.

این کار به مدل اجازه می‌دهد تا به وظایف خاص (specific tasks) سازگار شود یا رفتار مطلوبی از خود نشان دهد. به عنوان مثال، می‌توانیم یک مدل پایه (base model) را برای اجرای خوب یک وظیفه طبقه‌بندی (classification task) یا برای دنبال کردن دستورالعمل‌ها (follow instructions) تنظیم دقیق (fine-tune) کنیم.

این مرحله از آموزش باعث صرفه جویی زیادی در منابع خواهد شد زیرا مرحله پیش‌آموزش بسیار پرهزینه است و معمولاً نیازمند داده‌ها و منابع محاسباتی است که فراتر از دسترس بسیاری از افراد و سازمان‌ها است. به عنوان مثال، Llama 2 بر روی مجموعه داده‌ای حاوی ۲ تریلیون توکن آموزش داده شده است. تصور کنید چه محاسباتی برای ایجاد این مدل لازم است!

در این روش، مدل ابتدا روی داده‌های عمومی و بدون برچسب آموزش دیده و سپس با مقدار کمی داده‌ی برچسب‌دار برای یک کار مشخص، به‌صورت دقیق‌تر تنظیم می‌شود. یعنی می‌توان بدون نیاز به آموزش از ابتدا، مدل را برای کارهای مختلف به‌خوبی تنظیم کرد.

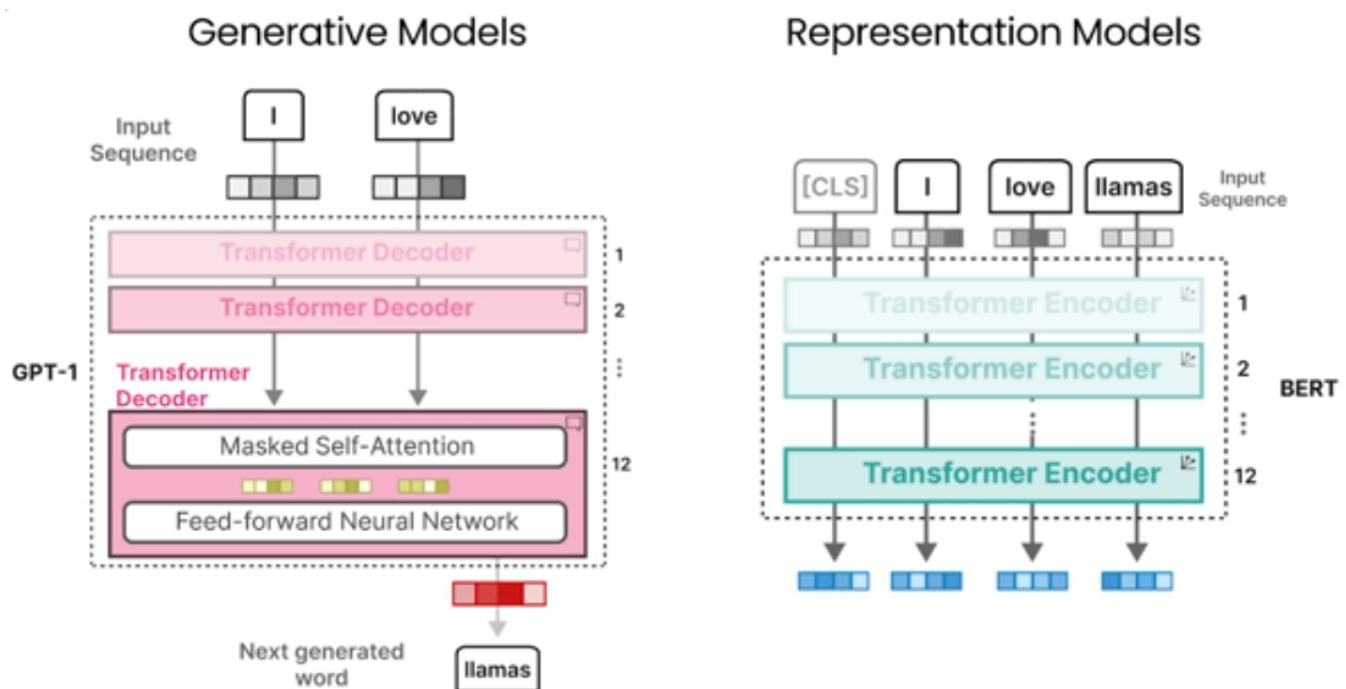
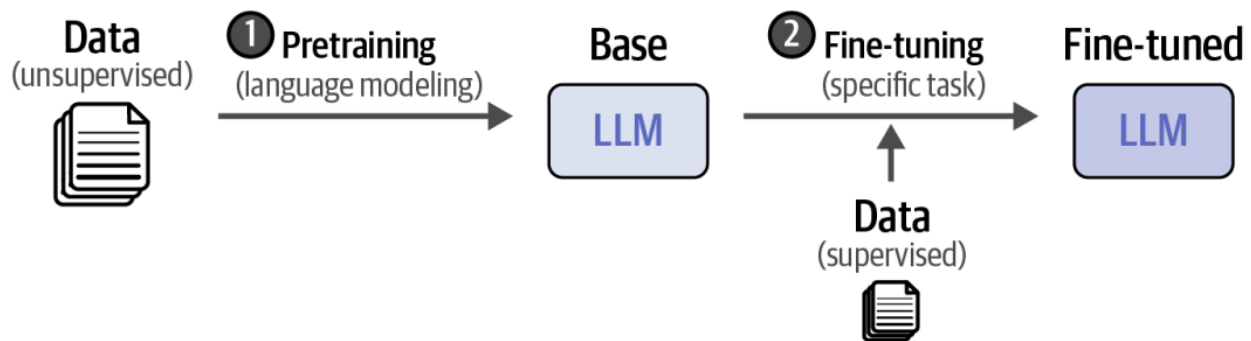
به‌عنوان مثال، یک رمزگذار (مثل BERT) که قبلاً آموزش دیده، می‌تواند برای کارهایی مثل تشخیص احساسات، دسته‌بندی متن یا پاسخ به سوال، با افزودن یک لایه‌ی ساده (مثل دسته‌بند) دوباره آموزش داده شود.

دو شیوه برای تنظیم دقیق وجود دارد:

1. آموزش کل مدل (هم رمزگذار و هم دسته‌بند)
2. فریز کردن رمزگذار و آموزش فقط بخش دسته‌بندی

در فصل‌های آتی، ما چندین روش برای تنظیم دقیق مدل‌های بنیادین بر روی مجموعه داده را بررسی خواهیم کرد.

هر مدلی که از مرحله اول یعنی پیش‌آموزش (pretraining) عبور کند، ما آن را یک مدل پیش‌آموزش‌شده (pretrained model) می‌نامیم، که شامل مدل‌های تنظیم دقیق‌شده (fine-tuned models) نیز می‌شود. این رویکرد دو مرحله‌ای آموزش در شکل زیر نشان داده شده است.



می‌توان مراحل تنظیم دقیق اضافی را برای همراستا کردن بیشتر مدل با ترجیحات کاربر (e user's preferences) اضافه کرد، که آنرا در فصل‌های آتی بررسی خواهیم کرد.

- مرحله دوم - بخش دوم: استفاده از مهندسی درخواست (Prompting) در مدل‌های پیش‌آموزش‌دیده

در حالی که مدل‌های رمزگذار (Encoding Models) معمولاً به ماژول‌های اضافی برای انجام کار نیاز دارند، **مدل‌های تولید دنباله** (Sequence Generation Models) مانند مدل‌های زبانی بزرگ (LLMs)، اغلب به‌تنهایی برای وظایفی مانند ترجمه، پاسخ به سوال و تولید متن کافی هستند. این مدل‌ها معمولاً با پیش‌بینی توکن بعدی بر اساس توکن‌های قبلی آموزش دیده‌اند.

با وجود سادگی این وظیفه (پیش‌بینی توکن بعدی)، تکرار آن در مقیاس بسیار وسیع باعث شده است مدل‌های زبانی بزرگ، دانش زبانی عمیقی کسب کنند. این موضوع امکان جدیدی را فراهم کرده: می‌توان بسیاری از مسائل NLP را تنها با ارائه یک **پرامپت (Prompt)** به صورت تولید متن حل کرد، بدون نیاز به آموزش دوباره‌ی مدل.

مثال ساده:

فرض کنید جمله زیر را داریم:

I love the food here. It's amazing! I'm...

اگر مدل در ادامه‌ی این جمله واژه‌هایی مانند *happy* یا *satisfied* تولید کند، می‌توان نتیجه گرفت که احساس جمله مثبت است. در این روش، با افزودن عبارتی مانند "I'm" به انتهای متن، مدل به شکلی طبیعی ادامه جمله را تولید می‌کند که نشانه‌ی نوع احساس است.

استفاده از دستور متنی (Instruction Prompting)

با استفاده از یک توضیح کوتاه، می‌توان مدل را به انجام کار هدایت کرد:

فرض کنید احساس یک متن یکی از برچسب‌های {مثبت، منفی، خنثی} است. احساس متن زیر را مشخص کنید.

Input: I love the food here. It's amazing!

Polarity: ...

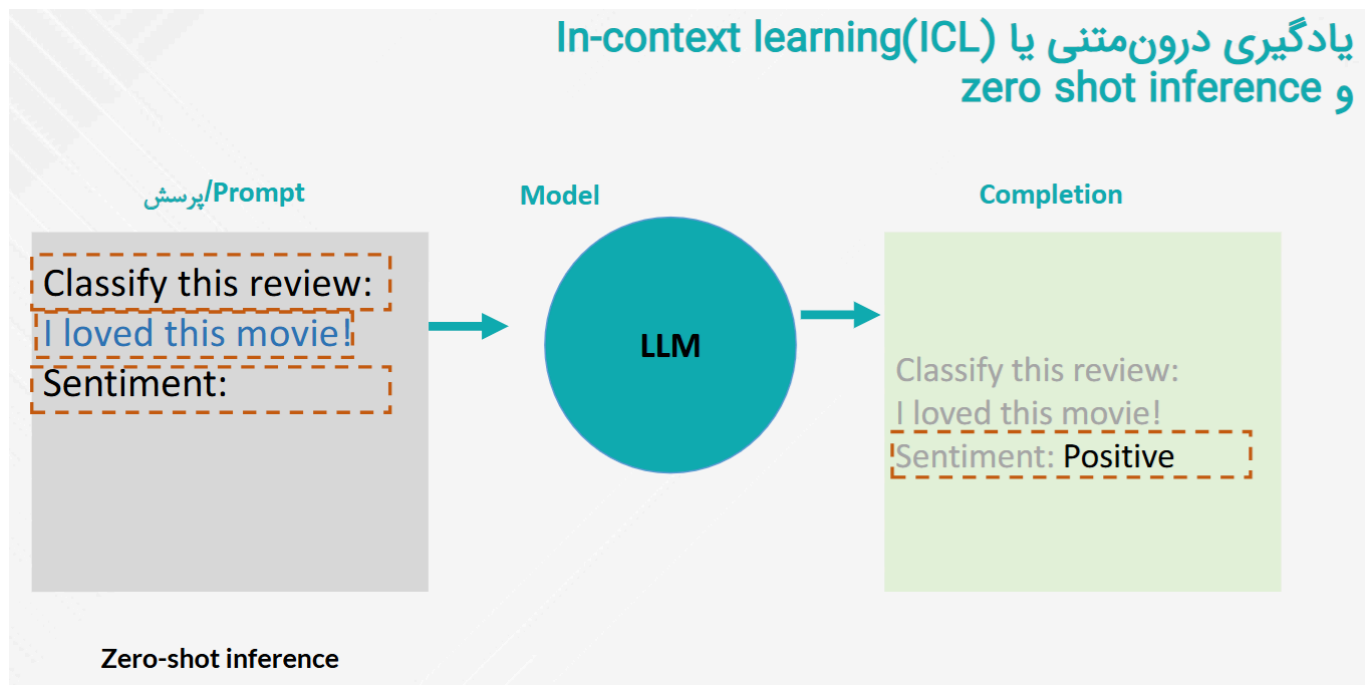
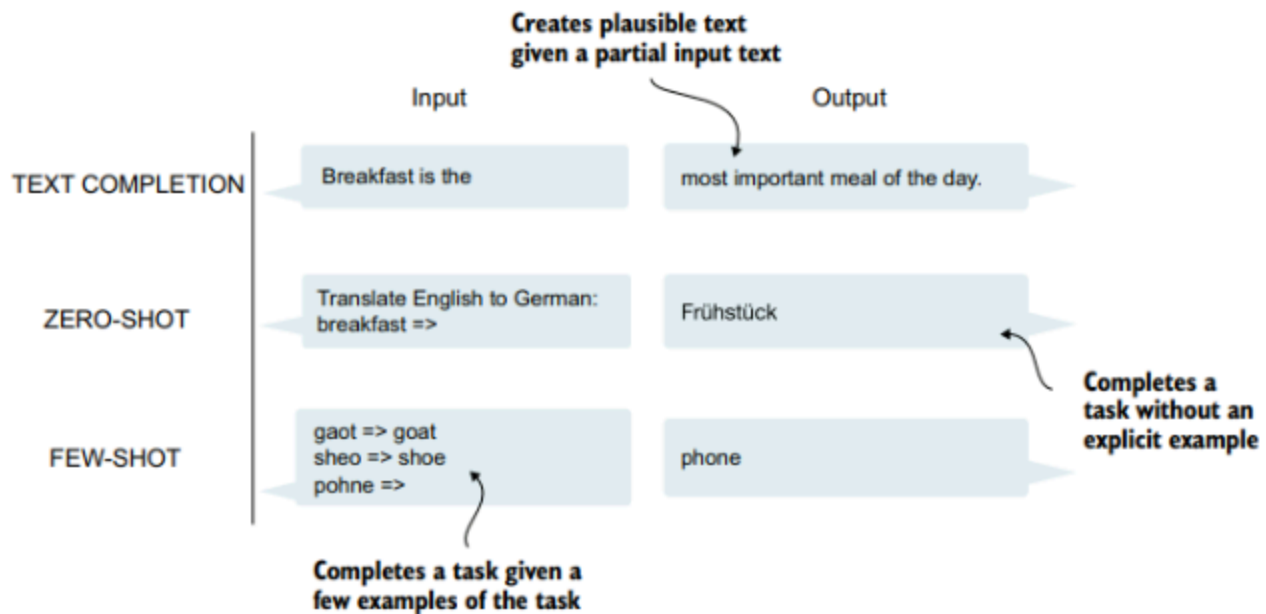
مدل با درک این دستور، برچسب مناسب را به‌صورت ادامه‌ی متن تولید می‌کند. این روش نشان‌دهنده‌ی توانایی **یادگیری بدون نمونه (Zero-shot learning)** در مدل‌های زبانی بزرگ است.

یادگیری با چند نمونه (Few-shot Learning)

گاهی برای آموزش بهتر مدل، چند مثال مشابه را در قالب پرامپت ارائه می‌دهیم:

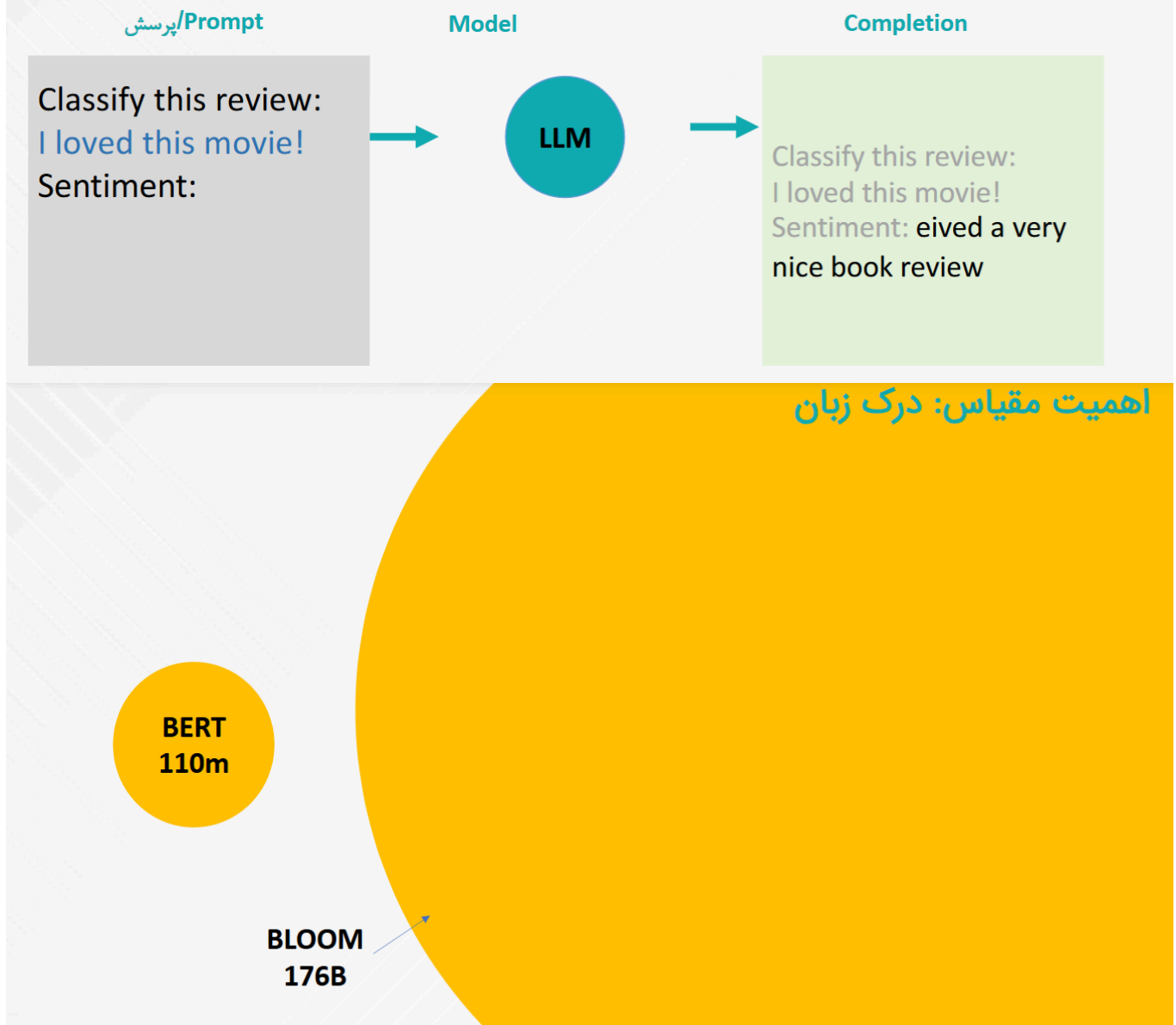
Input: The traffic is terrible during rush hours. Polarity: Negative
 Input: The weather here is wonderful. Polarity: Positive
 Input: I love the food here. It's amazing! Polarity: ...

در این روش، مدل با دیدن چند مثال مشابه در متن، یاد می‌گیرد چگونه ورودی‌ها را به خروجی‌ها ارتباط دهد. به این شیوه یادگیری در متن (In-context learning) گفته می‌شود.



مدل کوچک نمیتواند این کار را به هوبی انجام دهد

یادگیری درون‌متنی یا (ICL) و zero shot inference



برای مدل کوچک باید مثال زد تا بفهمد...

یادگیری درون‌متنی یا In-context learning (ICL) و one shot inference

پرسش / Prompt

```
Classify this review:  
I loved this movie!  
Sentiment: Positive  
Classify this review:  
I don't like this chair.  
Sentiment:
```

Model

LLM

Completion

```
Classify this review:  
I loved this movie!  
Sentiment: Positive  
Classify this review:  
I don't like this chair.  
Sentiment: Negative
```

یادگیری درون‌متنی یا In-context learning (ICL) و few shot inference

پرسش / Prompt

```
Classify this review:  
I loved this DVD!  
Sentiment: Positive  
Classify this review:  
I don't like this chair.  
Sentiment: Negative  
Classify this review:  
This is not great.  
Sentiment:
```

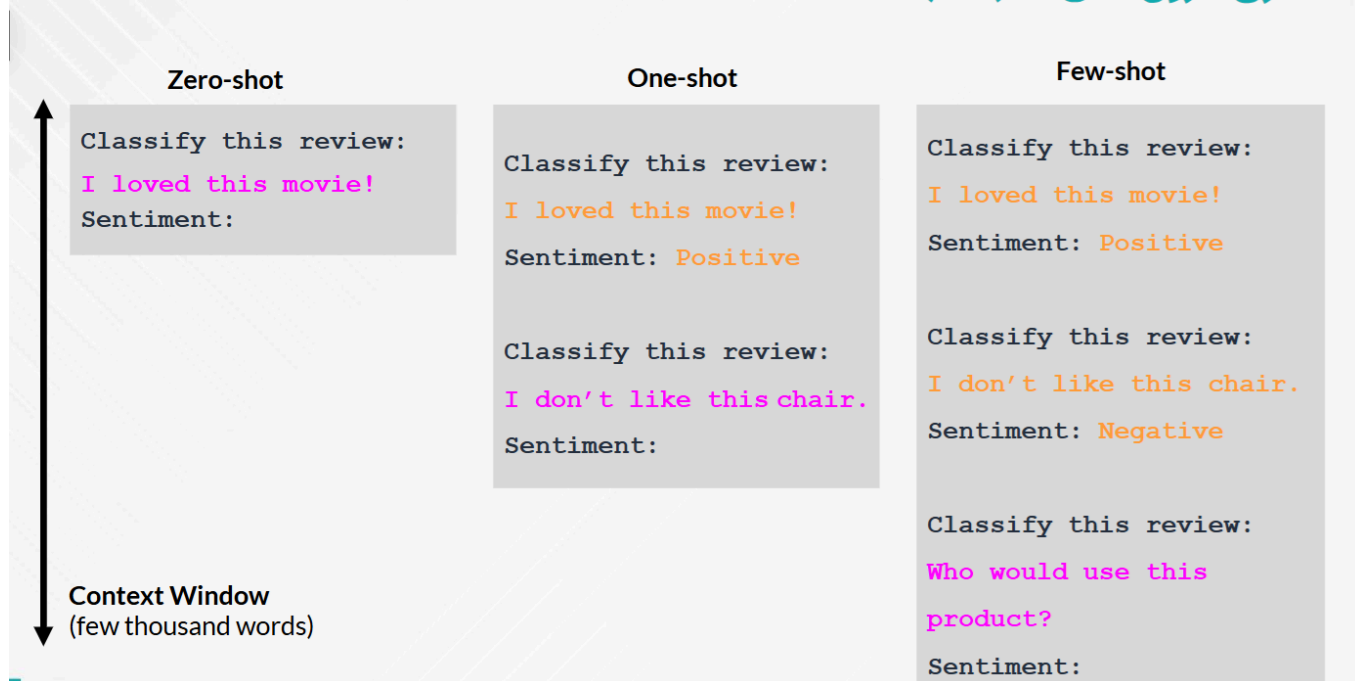
Model

LLM

Completion

```
Classify this review:  
I loved this DVD!  
Sentiment: Positive  
Classify this review:  
I don't like this chair.  
Sentiment: Positive  
Classify this review:  
This is not great.  
Sentiment: Negative
```

یادگیری درون‌متنی یا (ICL) In-context learning



یادگیری درون‌متنی یا (ICL) In-context learning



چند مثال کاربردی

ماهیت مدل‌های زبان بزرگ (LLMs) آن‌ها را برای انجام گستره‌ی وسیعی از وظایف مناسب می‌کند. با تولید متن (text generation) و استفاده از پرامپت‌ها (prompting)، تقریباً به نظر می‌رسد تنها محدودیت شما، تخیلتان باشد. برای روشن‌تر شدن مطلب، بیایید برخی از وظایف و تکنیک‌های رایج را بررسی کنیم:

تشخیص مثبت یا منفی بودن بازخورد مشتری

این یک دسته‌بندی (طبقه‌بندی) نظارت‌شده (*supervised*) است و می‌توان آن را با مدل‌های فقط کدگذار (encoder-only) یا فقط رمزگشا (decoder-only) انجام داد، چه با استفاده از مدل‌های از پیش‌آموزش‌دیده و چه با تنظیم دقیق (fine-tuning) مدل‌ها.

توسعه سیستمی برای یافتن موضوعات مشترک در مسائل تیکت‌ها

این یک دسته‌بندی (طبقه‌بندی) غیرنظارت‌شده (*unsupervised*) است که در آن برچسب‌های از پیش تعریف شده‌ای نداریم. می‌توانیم از مدل‌های فقط کدگذار (encoder-only) برای انجام طبقه‌بندی خود استفاده کنیم و از مدل‌های فقط رمزگشا (decoder-only) برای برچسب‌گذاری موضوعات بهره ببریم.

ساخت سیستمی برای بازیابی و بررسی اسناد مرتبط

یکی از اجزای اصلی سیستم‌های مدل زبان، توانایی آن‌ها در افزودن منابع خارجی اطلاعات است.

با استفاده از جستجوی معنایی، می‌توانیم سیستم‌هایی بسازیم که امکان دسترسی آسان و یافتن اطلاعات را برای استفاده مدل زبان بزرگ فراهم کنند. برای بهبود سیستم خود، می‌توانیم یک مدل تعبیه (ام‌دینگ) سفارشی بسازیم یا مدل خود را تنظیم دقیق کنیم (fine-tuning a custom embedding model).

ساخت یک چت‌بات مدل زبان بزرگ (LLM) که بتواند از منابع خارجی مانند ابزارها و اسناد بهره‌بردار

این ترکیبی از تکنیک‌ها است که نشان می‌دهد قدرت واقعی مدل‌های زبان بزرگ از طریق اجزای اضافی نمایان می‌شود. روش‌هایی مانند مهندسی پرامپت یا همان مهندسی درخواست، تولید تقویت‌شده با بازیابی اطلاعات (RAG) و تنظیم دقیق مدل زبان بزرگ همگی بخش‌هایی از معماری مدل زبان بزرگ هستند.

ساخت یک مدل زبان بزرگ قادر به نوشتن دستور پخت بر اساس تصویری که محصولات داخل یخچالتان را نشان می‌دهد

مدل‌های زبان بزرگ در حال تطبیق با دیگر مدالیت‌ها مانند بینایی هستند.

این یک وظیفه چندرسانه‌ای (مولتی‌مودال) که در آن مدل زبان بزرگ یک تصویر را دریافت کرده و در مورد آنچه که می‌بیند استدلال می‌کند. مدل‌های زبان بزرگ به گونه‌ای سازگار شده‌اند که می‌توانند از دیگر مدالیت‌ها، مانند بینایی، استفاده کنند که درهای دامنه وسیعی از کاربردهای جالب را باز می‌کند.

مدل‌های زبان بزرگ بسیار رضایت‌بخش هستند زیرا بخشی از محدودیت‌های آن‌ها به تصورات شما بستگی دارد. با دقت بیشتر این مدل‌ها، استفاده از آن‌ها در کاربردهای خلاقانه مانند بازی‌های نقش‌آفرینی و نوشتن کتاب‌های کودکان فقط جذاب‌تر می‌شود.

توسعه و استفاده مسئولانه از مدل‌های زبان بزرگ (LLM)

تأثیر مدل‌های زبان بزرگ بسیار گسترده بوده و احتمالاً همچنان ادامه دارد، به دلیل پذیرش وسیع آن‌ها. در حالی که قابلیت‌های شگفت‌انگیز این مدل‌ها را بررسی می‌کنیم، مهم است که پیامدهای اجتماعی و اخلاقی آن‌ها را نیز مد نظر داشته باشیم. چند نکته کلیدی برای توجه:

سوگیری (تعصب) و انصاف

مدل‌های زبان بزرگ بر روی حجم عظیمی از داده‌ها آموزش داده می‌شوند که ممکن است شامل تعصباتی باشند. این مدل‌ها ممکن است این تعصبات را یاد گرفته، بازتولید کرده و حتی تقویت کنند. از آنجا که داده‌های آموزشی معمولاً به اشتراک گذاشته نمی‌شوند، مشخص نیست چه تعصباتی ممکن است در آن‌ها وجود داشته باشد مگر اینکه خودتان مدل را آزمایش کنید.

شفافیت و مسئولیت‌پذیری

به دلیل قابلیت‌های شگفت‌انگیز مدل‌های زبان بزرگ، همیشه مشخص نیست که با یک انسان صحبت می‌کنید یا یک مدل. بنابراین، استفاده از این مدل‌ها در تعامل با انسان‌ها می‌تواند پیامدهای ناخواسته‌ای داشته باشد اگر انسان در فرایند نظارت نداشته باشد. به عنوان مثال، برنامه‌های مبتنی بر LLM که در حوزه پزشکی استفاده می‌شوند ممکن است به عنوان دستگاه‌های پزشکی تحت مقررات قرار بگیرند زیرا می‌توانند بر سلامت بیمار تأثیر بگذارند.

تولید محتوای مضر

یک مدل زبان بزرگ لزوماً محتوای صحیح و مستند تولید نمی‌کند و ممکن است با اعتماد به نفس متن نادرست ارائه دهد. همچنین، این مدل‌ها می‌توانند برای تولید اخبار جعلی، مقالات و منابع اطلاعاتی گمراه‌کننده به کار روند.

مالکیت معنوی

آیا خروجی مدل زبان بزرگ، مالکیت معنوی شماست یا سازنده مدل؟ وقتی خروجی مشابه عبارتی در داده‌های آموزشی باشد، آیا مالکیت معنوی به نویسنده آن عبارت تعلق دارد؟ بدون دسترسی به داده‌های آموزشی، مشخص نیست که آیا مواد دارای حق نشر توسط مدل استفاده شده‌اند یا خیر.

مقررات

آیا خروجی مدل زبان بزرگ، مالکیت معنوی شماست یا سازنده مدل؟ وقتی خروجی مشابه عبارتی در داده‌های آموزشی باشد، آیا مالکیت معنوی به نویسنده آن عبارت تعلق دارد؟ بدون دسترسی به داده‌های آموزشی، مشخص نیست که آیا مواد دارای حق نشر توسط مدل استفاده شده‌اند یا خیر.

هنگام توسعه و استفاده از مدل‌های زبان بزرگ، می‌خواهیم بر اهمیت ملاحظات اخلاقی تأکید کرده و شما را تشویق کنیم که بیشتر در مورد استفاده ایمن و مسئولانه از مدل‌های زبان بزرگ و سیستم‌های هوش مصنوعی به طور کلی بیاموزید.

منابع کافی، تنها چیزی است که به آن نیاز دارید!

منابع محاسباتی که تاکنون به آن‌ها اشاره کرده‌ایم معمولاً به GPUهایی اشاره دارند که در سیستم شما موجود است. یک GPU قدرتمند (کارت گرافیک) باعث می‌شود که هم آموزش و هم استفاده از مدل‌های زبان بزرگ بسیار کارآمدتر و سریع‌تر انجام شود.

در انتخاب یک GPU، یک جزء مهم مقدار VRAM (حافظه دسترسی تصادفی ویدیویی) است که شما در دسترس دارید. این به میزان حافظه‌ای اشاره دارد که شما بر روی GPU خود دارید. هر چه VRAM

شما بیشتر باشد، بهتر است. دلیل این امر این است که اگر VRAM کافی نداشته باشید برخی از مدل‌ها اصلاً نمی‌توانند استفاده شوند.

از آنجا که آموزش و تنظیم دقیق مدل‌های زبان بزرگ می‌تواند فرایندی پرهزینه از نظر GPU باشد، افرادی که GPU قدرتمندی ندارند اغلب به عنوان GPU-poor (فقر GPU) شناخته می‌شوند. این نشان‌دهنده رقابت برای منابع محاسباتی برای آموزش این مدل‌های بزرگ است. برای مثال، برای ایجاد خانواده مدل‌های Llama 2، Meta از GPUهای 80 GB A100 استفاده کرده است. فرض کنید اجاره چنین GPU به قیمت 1.50 دلار در ساعت باشد، هزینه کل ایجاد این مدل‌ها بیشتر از 5 میلیون دلار خواهد شد!

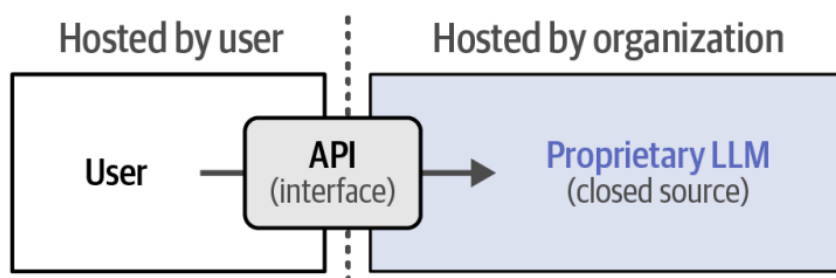
متأسفانه، هیچ قاعده خاصی برای تعیین دقیق میزان VRAM مورد نیاز برای یک مدل خاص وجود ندارد. این امر به معماری و اندازه مدل، تکنیک فشرده‌سازی، اندازه زمینه، بک‌اند برای اجرای مدل و غیره بستگی دارد.

این نوشتار برای GPU-فقرها است! ما از مدل‌هایی استفاده خواهیم کرد که کاربران می‌توانند آن‌ها را بدون نیاز به گران‌ترین GPU ها یا بودجه کلان اجرا کنند. برای انجام این کار، ما تمام کدها را در Google Colab در دسترس قرار خواهیم داد. در زمان نوشتن، یک نمونه رایگان از Google Colab به شما یک GPU T4 با 16 گیگابایت VRAM خواهد داد که حداقل میزان VRAM پیشنهادی ما است.

مدل‌های اختصاصی و خصوصی

مدل‌های زبان بزرگ با کد بسته (Closed source) مدل‌هایی هستند که وزن‌ها و معماری آن‌ها به صورت عمومی به اشتراک گذاشته نمی‌شود. این مدل‌ها توسط سازمان‌های خاصی توسعه یافته‌اند و کد پایه آن‌ها محرمانه نگه داشته شده است. نمونه‌هایی از چنین مدل‌هایی شامل GPT-4 شرکت OpenAI و Claude شرکت Anthropic هستند. این مدل‌های اختصاصی معمولاً توسط حمایت‌های تجاری قابل توجهی پشتیبانی می‌شوند و در خدمات آن سازمان‌ها توسعه یافته و ادغام شده‌اند.

می‌توانید از طریق یک رابط که با مدل زبان بزرگ (LLM) ارتباط برقرار می‌کند، به این مدل‌ها دسترسی پیدا کنید که به آن API (رابط برنامه‌نویسی کاربردی) گفته می‌شود، همان‌طور که در شکل cdv نشان داده شده است. به عنوان مثال، برای استفاده از ChatGPT در زبان برنامه‌نویسی پایتون می‌توانید از بسته OpenAI استفاده کنید تا بدون دسترسی مستقیم به مدل، با سرویس ارتباط برقرار کنید.

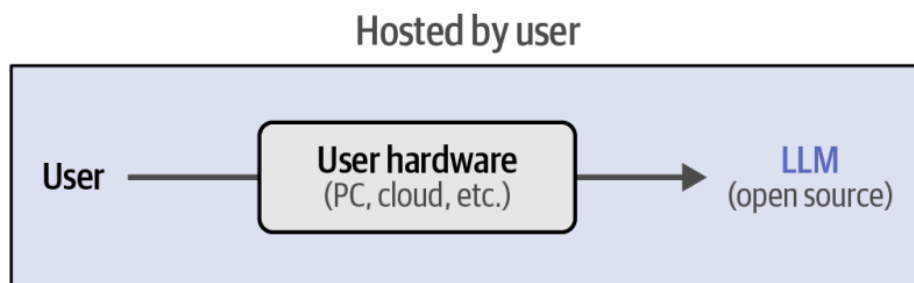


مدل‌های متن باز

مدل‌های متن باز (Open LLMs) مدل‌هایی هستند که وزن‌ها و معماری خود را با عموم به اشتراک می‌گذارند تا قابل استفاده باشند. این مدل‌ها هنوز توسط سازمان‌های خاص توسعه داده می‌شوند، اما اغلب کدهای مربوط به ساخت یا اجرای مدل را به صورت محلی به اشتراک می‌گذارند—با سطوح مختلفی از مجوزها که ممکن است اجازه استفاده تجاری از مدل را بدهند یا ندهند. مدل‌های Cohere's Command R، مدل‌های Mistral، Phi مایکروسافت و مدل‌های Llama متا نمونه‌هایی از مدل‌های باز هستند.

برخی مدل‌های به اشتراک گذاشته شده عمومی دارای مجوز تجاری آسان هستند که به این معنی است که نمی‌توان از این مدل‌ها برای مقاصد تجاری استفاده کرد. برای بسیاری، این تعریف واقعی منبع باز نیست، چرا که در این تعریف استفاده از این مدل‌ها باید هیچ محدودیتی نداشته باشد. به طور مشابه، داده‌هایی که مدل بر اساس آن‌ها آموزش می‌بیند و همچنین کد منبع آن‌ها معمولاً به اشتراک گذاشته نمی‌شود.

شما می‌توانید این مدل‌ها را دانلود کرده و از آن‌ها روی دستگاه خود استفاده کنید، به شرطی که GPU قدرتمندی داشته باشید که بتواند این نوع مدل‌ها را پردازش کند، همانطور که در شکل زیر نشان داده شده است.



مدل‌های زبان بزرگ متن باز (Open source) به صورت مستقیم در اختیار کاربر قرار می‌گیرند. در نتیجه، جزئیات خود مدل از جمله کد و معماری آن با کاربر به اشتراک گذاشته می‌شود.

یکی از مزایای اصلی این مدل‌های محلی این است که شما، به عنوان کاربر، کنترل کامل روی مدل دارید. شما می‌توانید از مدل استفاده کنید بدون اینکه به اتصال API آن وابسته باشید، آن را تنظیم مجدد (fine-tune) کنید و داده‌های حساس را از طریق آن پردازش کنید. شما به هیچ خدماتی وابسته نیستید و شفافیت کامل در مورد فرایندهایی که منجر به خروجی مدل می‌شوند دارید. این مزیت با جوامع بزرگی که این فرایندها را امکان‌پذیر می‌سازند، مانند Hugging Face، تقویت می‌شود که قابلیت‌های تلاش‌های مشترک را به نمایش می‌گذارد.

یکی از معایب این است که برای اجرای این مدل‌ها به سخت‌افزار قدرتمندی نیاز دارید و حتی بیشتر از آن زمانی که بخواهید آن‌ها را آموزش یا تنظیم مجدد کنید. علاوه بر این، برای تنظیم و استفاده از این مدل‌ها نیاز به دانش خاصی دارید (که در طول این نوشتار به آن خواهیم پرداخت).

ما معمولاً ترجیح می‌دهیم از مدل‌های منبع باز استفاده کنیم، زیرا آزادی‌ای که این مدل‌ها برای آزمایش گزینه‌ها، بررسی عملکردهای داخلی و استفاده از مدل به صورت محلی فراهم می‌آورند، به طور کلی بیشتر از استفاده از مدل‌های تجاری مفید است.

چارچوب‌های منبع باز

در مقایسه با مدل‌های منبع بسته، مدل‌های منبع باز نیاز به استفاده از بسته‌های خاصی برای اجرای آن‌ها دارند. در سال 2023، بسیاری از بسته‌ها و چارچوب‌های مختلف منتشر شدند که هرکدام به نوعی با مدل‌های زبان تعامل می‌کنند و از آن‌ها استفاده می‌کنند. گشتن در میان صدها چارچوب ممکن است تجربه‌ای چندان لذت‌بخش نباشد.

به همین دلیل، ممکن است شما حتی چارچوب مورد علاقه خود را در این نوشتار از دست بدهید!

به جای تلاش برای پوشش دادن هر چارچوب مدل زبان موجود (زیرا تعداد آن‌ها زیاد است و همچنان در حال افزایش هستند)، هدف ما این است که یک مبنای مستحکم برای استفاده از مدل‌های زبان فراهم کنیم. ایده این است که پس از خواندن این نوشتار، شما به راحتی می‌توانید سایر چارچوب‌ها را که همگی به‌طور مشابه عمل می‌کنند، یاد بگیرید.

در این نوشتار، ما بیشتر روی بسته‌های پشتیبان (Backend Packages) تمرکز خواهیم کرد. این بسته‌ها بدون رابط گرافیکی کاربری (GUI) هستند و برای بارگذاری و اجرای مؤثر هر مدل زبانی روی دستگاه شما طراحی شده‌اند، مانند llama.cpp، LangChain و بسته بسیاری از چارچوب‌ها، Hugging Face Transformers است.

ما عمدتاً چارچوب‌های مورد استفاده برای تعامل با مدل‌های زبان بزرگ را از طریق کد پوشش خواهیم داد. اگرچه این به شما کمک می‌کند اصول این چارچوب‌ها را بیاموزید، اما گاهی اوقات شما فقط یک رابط مشابه ChatGPT با یک مدل زبان محلی می‌خواهید. خوشبختانه، ز چارچوب‌های شگفت‌انگیز بسیاری برای این کار وجود دارند. چند نمونه از آن‌ها عبارتند از KoboldCpp، text-generation-webui، LM و Studio ...

تولید اولین متن

یکی از اجزای مهم استفاده از مدل‌های زبان، انتخاب آن‌ها است. منبع اصلی برای پیدا کردن و دانلود مدل‌های زبان بزرگ (LLMs) Hugging Face Hub است. Hugging Face سازمانی است که پشت بسته معروف Transformers قرار دارد، که برای سال‌ها توسعه مدل‌های زبان را هدایت کرده است. همانطور که از نامش پیداست، این بسته بر اساس چارچوب transformers ساخته شده است که در فصل "تاریخچه اخیر هوش مصنوعی زبان" به آن پرداخته شد.

در زمان نگارش این نوشتار، شما بیش از ۸۰۰,۰۰۰ مدل را در پلتفرم Hugging Face برای اهداف مختلف خواهید یافت، از مدل‌های زبان بزرگ و مدل‌های بینایی کامپیوتری تا مدل‌هایی که با داده‌های صوتی و جدولی کار می‌کنند. در اینجا شما می‌توانید تقریباً هر مدل زبان منبع باز را پیدا کنید.

اگرچه ما در این نوشتار انواع مدل‌ها را بررسی خواهیم کرد، بیایید اولین خطوط کد خود را با یک مدل مولد آغاز کنیم. مدل مولدی که ما در طول نوشتار استفاده می‌کنیم Phi-3-mini است که یک مدل نسبتاً کوچک (با 3.8 میلیارد پارامتر) اما بسیار کارآمد است. به دلیل اندازه کوچک آن، مدل می‌تواند بر روی دستگاه‌هایی با کمتر از 8 گیگابایت VRAM اجرا شود. اگر فشرده‌سازی (quantization) انجام دهید، که نوعی فشرده‌سازی است که در فصل‌های آتی بیشتر به آن پرداخته خواهد شد، می‌توانید از کمتر از 6 گیگابایت VRAM استفاده کنید.

علاوه بر این، مدل تحت مجوز MIT است که به شما این امکان را می‌دهد که از مدل برای مقاصد تجاری بدون هیچ محدودیتی استفاده کنید!

به یاد داشته باشید که مدل‌های زبان بزرگ جدید و بهبود یافته به طور مداوم منتشر می‌شوند. برای اطمینان از به‌روز بودن این نوشتار، بیشتر مثال‌ها به‌گونه‌ای طراحی شده‌اند که با هر مدل LLM کار کنند. همچنین مدل‌های مختلف موجود در مخزن مرتبط با این نوشتار را برای شما معرفی خواهیم کرد تا امتحان کنید.

شروع کار

زمانی که از مدل زبان بزرگ استفاده می‌کنید، دو مدل بارگذاری می‌شود:

- خود مدل مولد (generative model)
- توکنایزر زیرساختی آن (underlying tokenizer)

توکنایزر مسئول تقسیم متن ورودی به توکن‌ها قبل از ارسال آن به مدل مولد است. شما می‌توانید توکنایزر و مدل را در سایت Hugging Face پیدا کنید و فقط نیاز دارید تا شناسه‌های مربوطه را وارد کنید. در اینجا، ما از مسیر "microsoft/Phi-3-mini-4k-instruct" به عنوان مسیر اصلی مدل استفاده می‌کنیم.

ما می‌توانیم از بسته transformers برای بارگذاری هر دو **توکنایزر و مدل** استفاده کنیم. توجه داشته باشید که ما فرض می‌کنیم شما یک GPU از نوع NVIDIA دارید (با استفاده از "device_map="cuda") اما می‌توانید دستگاه متفاوتی انتخاب کنید. اگر به GPU دسترسی ندارید، می‌توانید از نوت‌بوک‌های رایگان Google Colab که در مخزن این نوشتار قرار داده شده است استفاده کنید:

بارگذاری مدل و توکنایزر

```
from transformers import AutoModelForCausalLM, AutoTokenizer
# Load model and tokenizer
model = AutoModelForCausalLM.from_pretrained(
    "microsoft/Phi-3-mini-4k-instruct",
    device_map="cuda",
    torch_dtype="auto",
    trust_remote_code=True,
)
tokenizer = AutoTokenizer.from_pretrained("microsoft/Phi-3-mini-4k-instruct")
```

اجرای این کد باعث شروع دانلود مدل خواهد شد و بسته به اتصال اینترنت شما ممکن است چند دقیقه طول بکشد.

اگرچه اکنون کافی است که شروع به تولید متن کنیم، یک ترفند جالب در transformers وجود دارد که فرآیند را ساده می‌کند، به نام transformers.pipeline. این متد مدل، توکنایزر و فرآیند تولید متن را در یک تابع واحد ترکیب می‌کند:

استفاده از pipeline برای تولید متن

```
from transformers import pipeline
# Create a pipeline
generator = pipeline(
    "text-generation",
    model=model,
    tokenizer=tokenizer,
    return_full_text=False,
    max_new_tokens=500,
```

```
do_sample=False
)
```

در کد فوق پارامترهای زیر قابل توجه هستند:

- `return_full_text` : متن ورودی باز نمی‌گردد و تنها خروجی مدل ، `False` با تنظیم این گزینه به : نمایش داده می‌شود. (قبلا گفتیم که در مدل های اورکریسیو خروجی به صورت کلمه به کلمه به متن ورودی افزوده می‌شود)
- `max_new_tokens` : حداکثر تعداد توکن‌هایی که مدل تولید خواهد کرد. با تنظیم یک حد، از تولید خروجی طولانی و غیرقابل مدیریت جلوگیری می‌کنیم.
- `do_sample` : مشخص می‌کند که آیا مدل از استراتژی نمونه‌برداری برای انتخاب توکن بعدی استفاده : مدل همیشه توکن با بالاترین احتمال را انتخاب می‌کند. در ، `False` می‌کند یا نه. با تنظیم این گزینه به فصل های آتی چندین پارامتر نمونه‌برداری که موجب خلاقیت در خروجی مدل می‌شود را بررسی خواهیم کرد.

برای تولید اولین متن خود، بیاپید از مدل بخواهیم که یک ضرب المثل (بی مزه) برایمان تعریف کند. برای این کار، درخواست را در قالب یک لیست از دیکشنری‌ها ارسال می‌کنیم، که در آن هر دیکشنری به یک موجودیت در مکالمه مربوط می‌شود. نقش ما "کاربر" است و از کلید `content` برای تعریف درخواست خود استفاده می‌کنیم:

درخواست برای تولید ضرب المثل در مورد مرغ‌ها

```
# request (user input)

messages = [

    {"role": "user", "content": "Create a funny joke about chickens."}

]

# generate output

output = generator(messages)

print(output[0]["generated_text"])
```

خروجی ممکن است به صورت زیر باشد:

چرا مرغ‌ها دوست ندارند به باشگاه بروند؟ چون نمی‌توانند تخم‌مرغ خود را بشکنند!

خلاصه

در اولین فصل از این نوشتار، به تأثیر انقلاب مدل‌های زبان بزرگ (LLM) بر حوزه هوش مصنوعی زبانی پرداخته شد. این مدل‌ها به طور قابل توجهی رویکرد ما را در انجام کارهایی مانند ترجمه، طبقه‌بندی،

خلاصه‌سازی و سایر وظایف تغییر داده‌اند. از طریق تاریخچه‌ای اخیر از هوش مصنوعی زبان، مبنای چندین نوع مدل زبان بزرگ را بررسی کردیم، از نمایه ساده bag-of-words گرفته تا نمایه‌های پیچیده‌تر با استفاده از شبکه‌های عصبی.

ما مکانیزم توجه را به عنوان گامی مهم در جهت کدگذاری متون زمینه در مدل‌ها بررسی کردیم، که جزء حیاتی در آنچه که مدل‌های زبان بزرگ را این‌قدر توانمند می‌سازد است. در ادامه به دو دسته اصلی مدل که از این مکانیزم شگفت‌انگیز استفاده می‌کنند، پرداخته شد: مدل‌های نمایش (فقط کدگذار) مانند BERT و مدل‌های مولد (فقط رمزگشا) مانند خانواده مدل‌های GPT. هر دو دسته در این نوشتار به عنوان مدل‌های زبان بزرگ در نظر گرفته می‌شوند.

در مجموع، این فصل نمایی کلی از منظره هوش مصنوعی زبان را ارائه داد، از جمله کاربردهای آن، تأثیرات اجتماعی و اخلاقی، و منابع مورد نیاز برای اجرای چنین مدل‌هایی. در پایان، اولین متن خود را با استفاده از مدل Phi-3 تولید کردیم، مدلی که در طول نوشتار استفاده خواهد شد.

در فصول بعدی، شما با برخی فرآیندهای اساسی آشنا خواهید شد. ما با بررسی توکنیزاسیون و امبدینگ‌ها در فصل بعد شروع می‌کنیم، دو مؤلفه‌ای که اغلب نادیده گرفته می‌شوند اما برای حوزه هوش مصنوعی زبان بسیار حیاتی هستند. در فصول بعدی نیز نگاهی عمیق به مدل‌های زبان خواهیم داشت که در آن شما روش‌های دقیقی که برای تولید متن استفاده می‌شود را خواهید دید.

"Pasted image 20251228171455.png1" could not be found.

"Pasted image 20251228171503.png1" could not be found.

"Pasted image 20251228171510.png1" could not be found.

"Pasted image 20251228171521.png1" could not be found.

"Pasted image 20251228171531.png1" could not be found.

"Pasted image 20251228171555.png1" could not be found.

"Pasted image 20251228171609.png1" could not be found.

"Pasted image 20251228171625.png1" could not be found.

"Pasted image 20251228171631.png1" could not be found.

"Pasted image 20251228171636.png1" could not be found.

"Pasted image 20251228171647.png1" could not be found.

"Pasted image 20251228171706.png1" could not be found.

"Pasted image 20251228171717.png1" could not be found.

"Pasted image 20251228171727.png1" could not be found.